Lightweight LLM beyond Scaling:
From Business Cost Optimization to Frontier Research

Youngjae Yu

SNU PI Lab

youngjaeyu@snu.ac.kr

Session 1 : A Business-Centric Approach to LLM Cost Reduction and Efficiency

- **Understanding** LLM lightweighting and productization trends among US Big Tech and startups
- **Quantifying** business-oriented efficiency metrics (cost, latency, throughput) for informed decision-making
- **Sharing** architecture-level trade-offs and lightweighting techniques across LLM designs

朝鮮日報

한계 다다른 거대 AI 모델… 가볍고 똑똑한 경량 AI 모델 경쟁

'AI 경량화' 시대…삼성, 온디바이스 AI 고도화 나선다

클라우드 없이 기기에서 AI 구동 가능
300억 파라미터 모델 3GB 메모리로 실행

김승화 기자 업데이트 2025.11.22 17:26 | 댓글 0

최신뉴스

Lightweight LLM powers Japanese enterprise AI deployments

중국발 '딥시크 쇼크' 이후 AI 가격 전쟁 막 올랐다

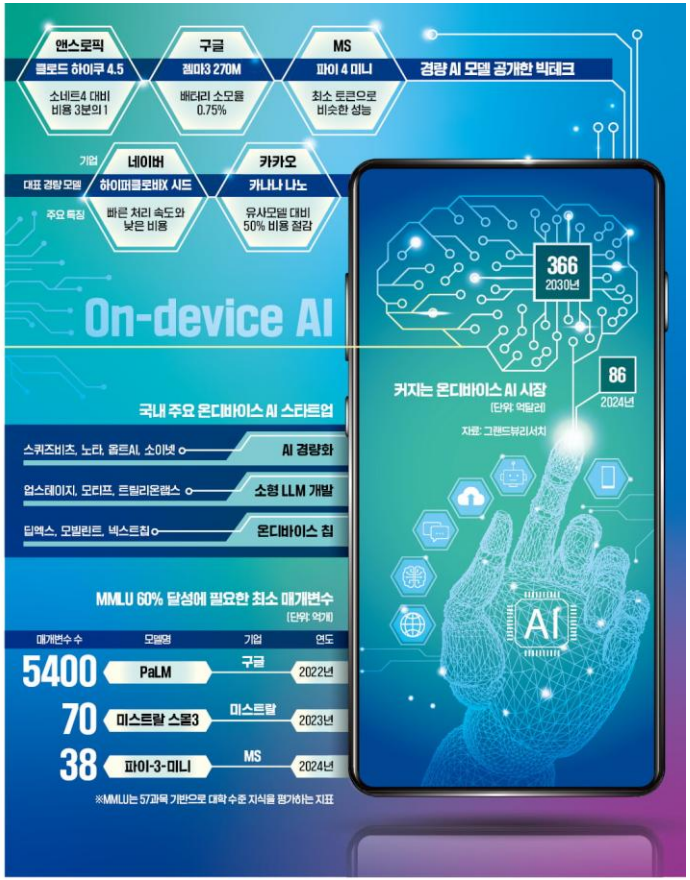
구글, 오픈AI 가격 인하에 중국 기업들 '저가 공세'…"기업들, 효율에 혁신 발목 잡힐라" 시각도

NVIDIA. DEVELOPER Home Blog Forums Docs Downloads Training

Technical Blog Search blog

How Small Language Models Are Key to Scalable Agentic AI

"전력 소모·비용 확 줄여라"…몸 키우던 AI, 이젠 '소형 경쟁'

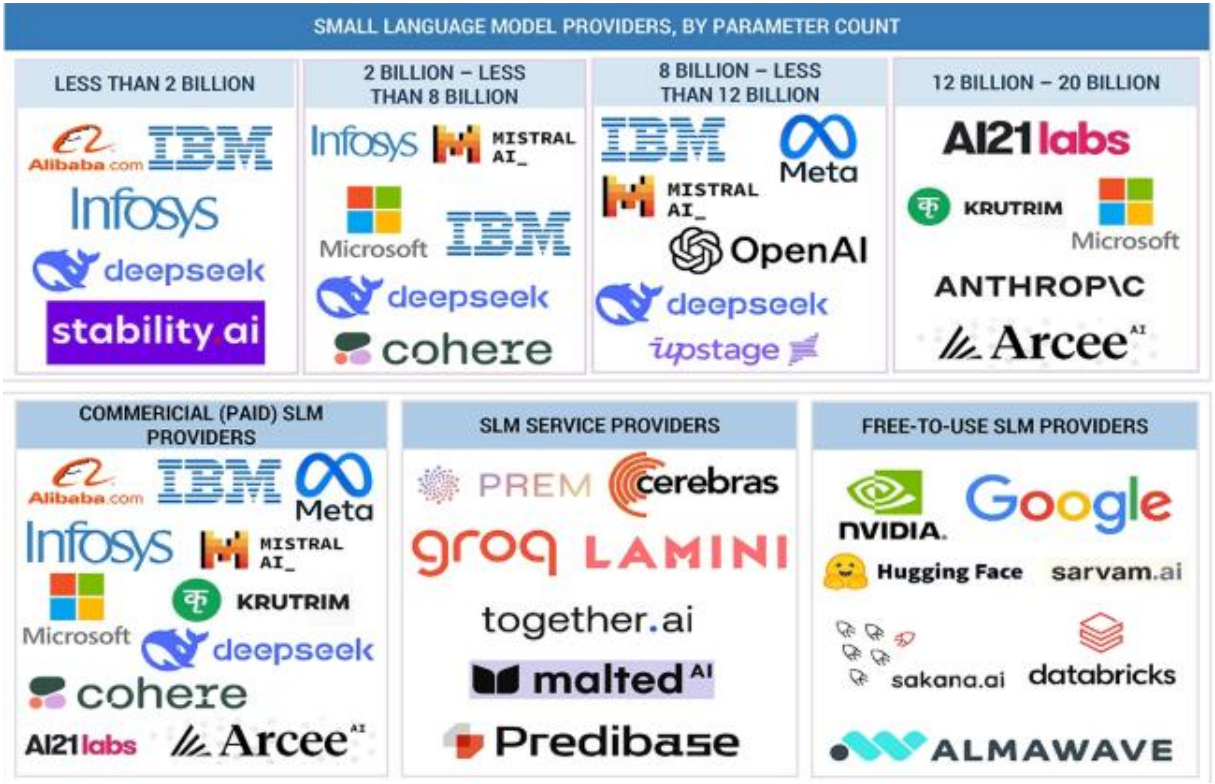


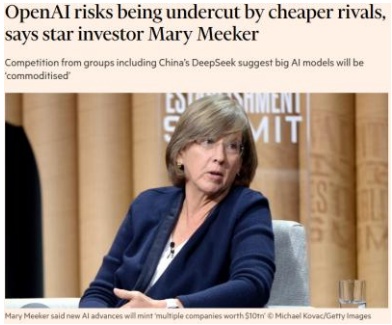
https://www.hankyung.com/article/2025102134571

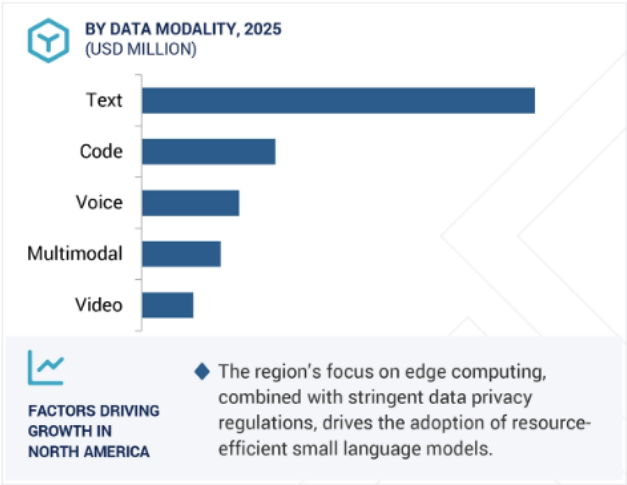
Global Small Language Model (SLM) Market Size

[1]https://market.us/report/large-language-model-powered-tools-market
[2]https://www.databridgemarketresearch.com/reports/global-small-language-model-slm-market

Global Lightweight Language Model Ecosystem







https://www.marketsandmarkets.com/Market-Reports/small-language-model-market-4008452.html

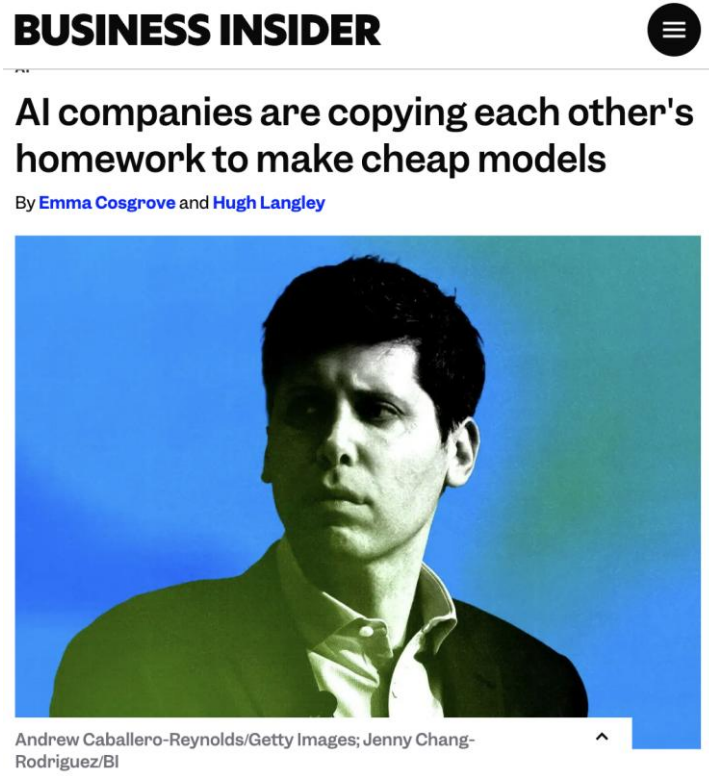
AI companies are copying each other's homework to make cheap models

- AI companies copy LLMs with 'distillation'
- The price of building AI is falling to new lows.

AI companies race to use 'distillation' to produce cheaper models

DeepSeek used technique to create smaller powerful models based on the technology of competitors such as Meta

The technique caught attention after DeepSeek used it to build AI models based on open-source systems released by competitors Meta and Alibaba © FT montage/Getty Images



BUSINESS INSIDER

AI companies are copying each other's homework to make cheap models

By Emma Cosgrove and Hugh Langley

Andrew Caballero-Reynolds/Getty Images; Jenny Chang-Rodriguez/BI

[1]https://www.ft.com/content/c117e853-d2a6-4e7c-aea9-e88c7226c31f

[2]https://www.businessinsider.com/deepseek-openai-distillation-big-tech-trouble-cheap-commodity-ai-2025-3

Cost and resource constraints are new battlefield

- As LLM adoption becomes widespread, **cost and resource constraints are becoming major bottlenecks** for enterprises.

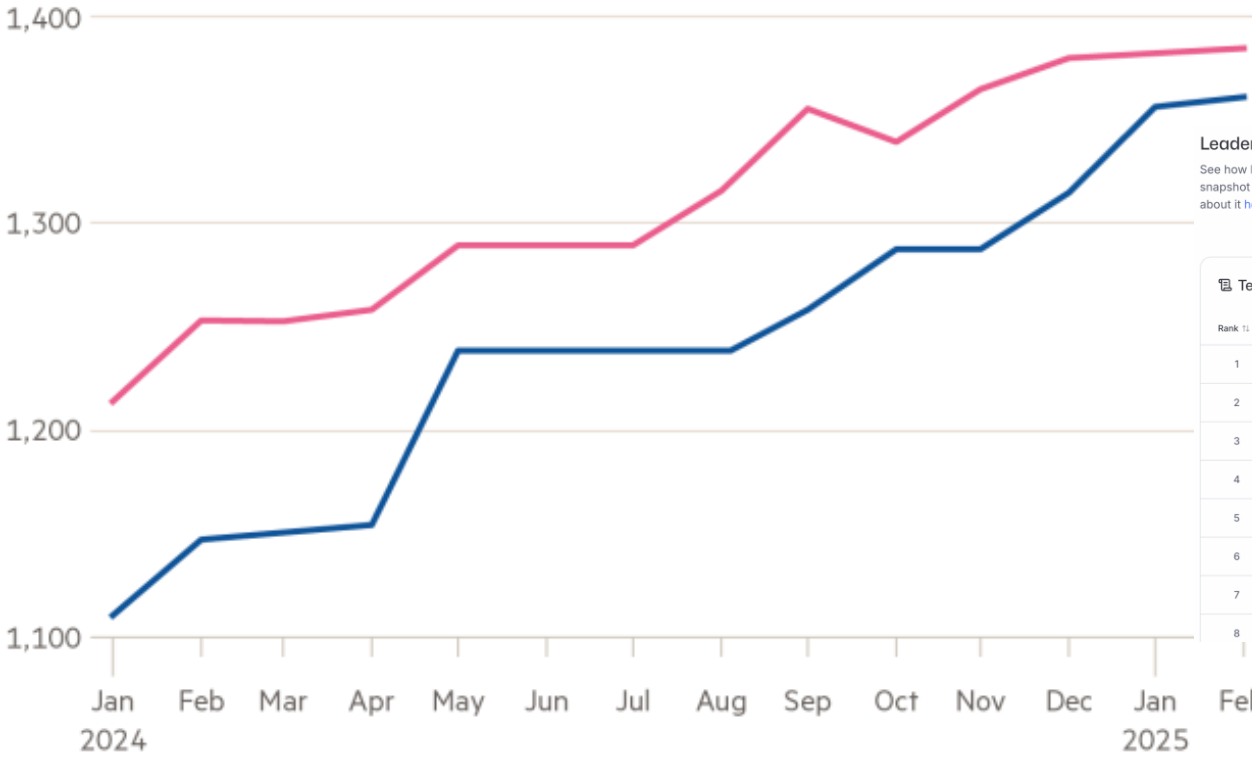
- Companies are aggressively **slashing LLM prices**, triggering a **supply-side cost war** in the AI model market.

Chinese tech giants slash prices of languag models used to power AI chatbots

By **Liam Mo** and **Eduardo Baptista**

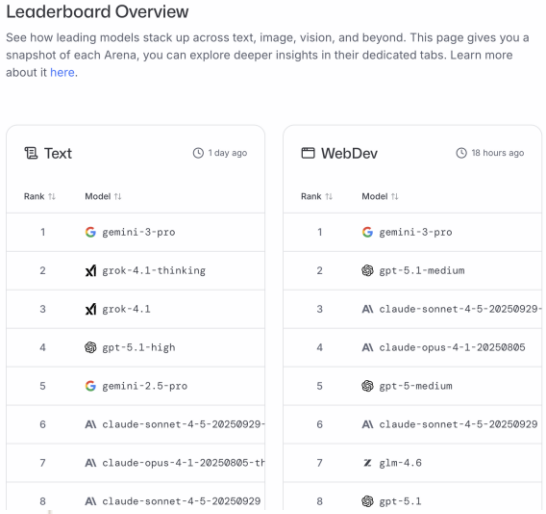
May 21, 2024 7:44 PM GMT+9 · Updated May 21, 2024

LMSYS Chatbot Arena

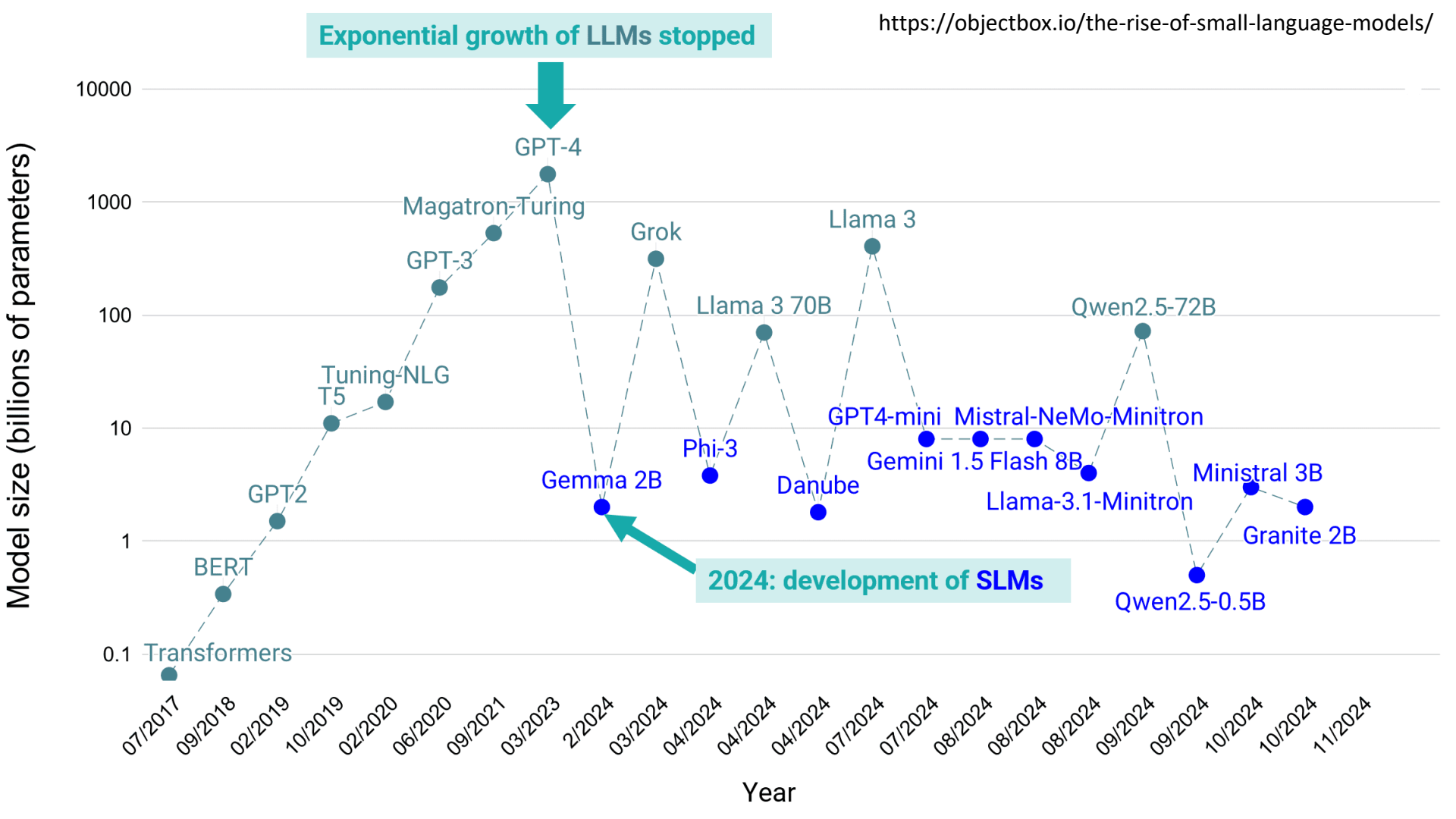


Sources: Bond; LMSYS Chatbot Arena; Stanford HAI

© FT

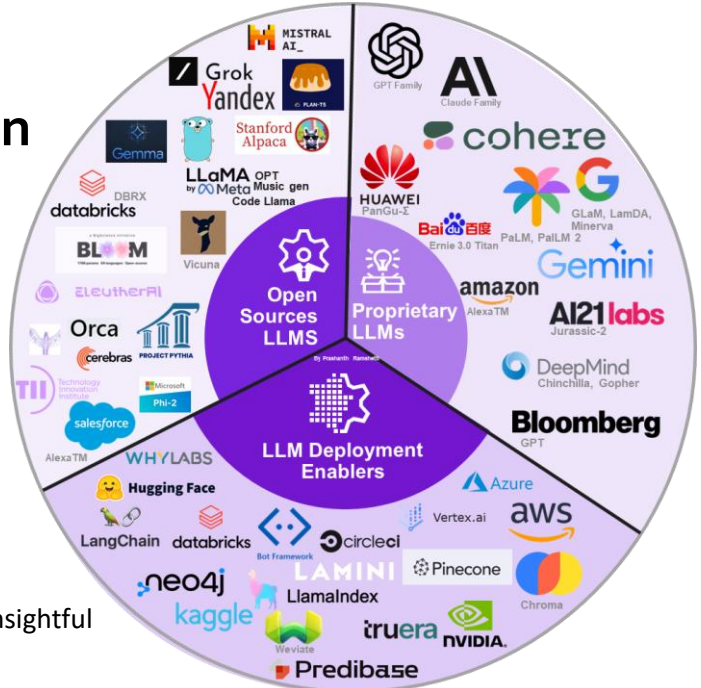


https://lmarena.ai/leaderboard


https://objectbox.io/the-rise-of-small-language-models/
Exponential growth of LLMs stopped
2024: development of SLMs
Model size (billions of parameters)
10000
1000
100
10
1
0.1
Year
Transformers
BERT
GPT2
T5
Tuning-NLG
GPT-3
Magatron-Turing
GPT-4
Gemma 2B
Grok
Phi-3
Llama 3 70B
Danube
Llama 3
GPT4-mini
Gemini 1.5 Flash 8B
Mistral-NeMo-Minitron
Llama-3.1-Minitron
Qwen2.5-72B
Qwen2.5-0.5B
Ministral 3B
Granite 2B
07/2017
09/2018
02/2019
10/2019
02/2020
06/2020
09/2021
03/2023
2/2024
03/2024
04/2024
04/2024
04/2024
07/2024
07/2024
08/2024
08/2024
08/2024
09/2024
09/2024
10/2024
10/2024
11/2024

Adopting Small & Open LLMs

- **Open-source LLMs are emerging as a <u>cost-efficient option</u>** through business-specific deployment models, including **on-premises installations and private-cloud hosting**.
- **Model compression, serving, optimization techniques** are reducing inference costs, hardware requirements, and energy consumption.



Exploring the Dynamic Landscape of Large Language Models: An Insightful Overview by Prashanth Ramshetti

What Makes Small Language Models So Attractive?

💸 Accessible and Affordable

They can be run (in inference mode) on limited resource regimes

🛠️ Easier to Customize

Small models can typically be fine-tuned on just a single GPU.

🌱 More Energy Efficient

Small language models require fewer computational resources, making them more energy-efficient. It also means reduced energy consumption

🖥️ Cheaper to Develop

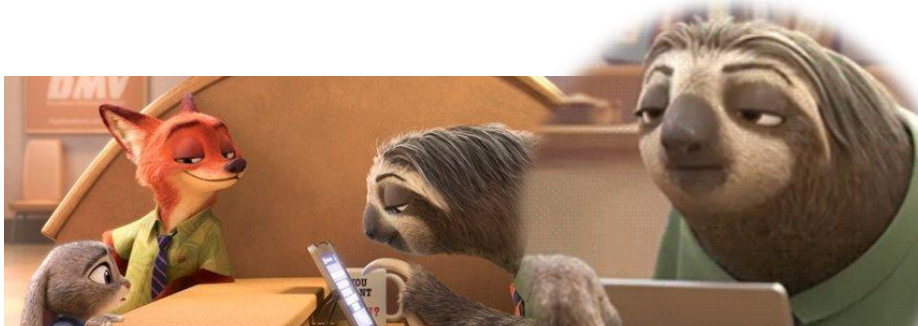
These models only require a relatively small number of GPUs.

🎓 Valuable for Educational Purposes

They are more manageable and thus easier to understand and tweak.

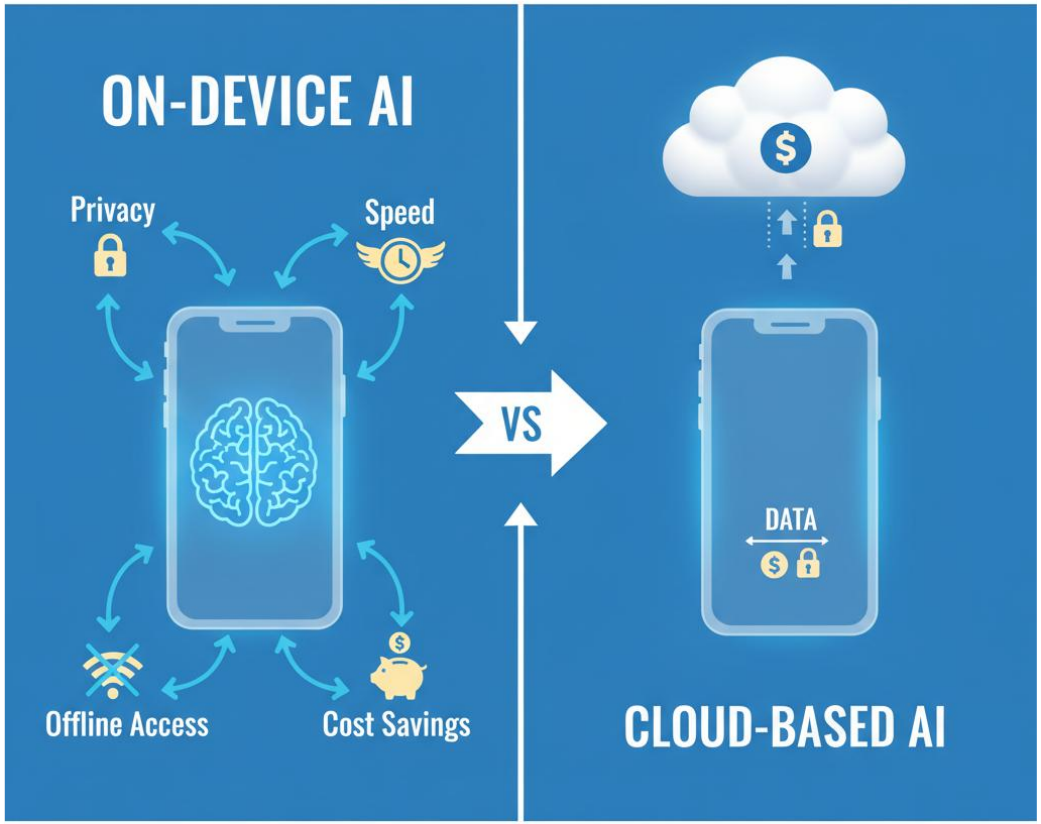
What Makes Small Language Models So Attractive?

- Lightweight language models enable **significantly lower training** and **inference costs** compared to large-scale models.
- On smartphones, robots, cars, and wearables, **latency** is the user experience. large models often feel slow because of server and network bottlenecks.
- Users perceive smartness more from fast, responsive behavior than from sheer model size.



On-device on Mainstream

- Lightweight models can be deployed **locally**, on-device, or in environments with limited connectivity
- This enables **privacy-sensitive use cases** (e.g., healthcare, enterprise internal systems)



https://drlee.io/the-on-device-ai-revolution-why-your-next-mobile-app-needs-a-small-multimodal-language-model-876fad370c1c

On-device on Mainstream

- Talkback - Gemini Nano's multimodal capabilities to improve image descriptions for blind and low vision users.
- Pixel Recorder - Gemini Nano with Multimodality model enables support for longer recordings and higher quality summaries.

On-device on Mainstream

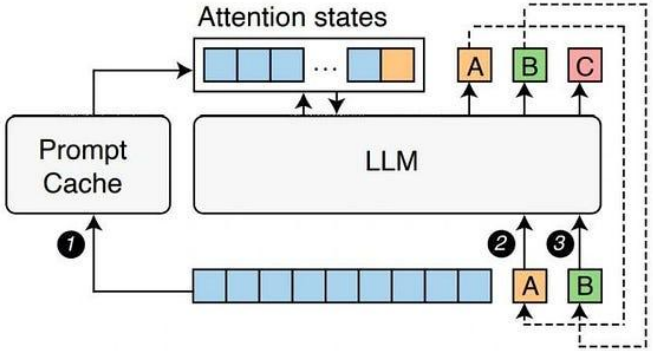
Big tech is betting on **small, efficient models** that live on phones and PCs, not just in giant data centers.

Apple: 2025 tech report introduces a 3B on-device model with KV-cache sharing and 2-bit QAT, plus a PT-MoE server model on Private Cloud Compute.

Microsoft: the Mu model, a 330M encoder–decoder optimized for NPUs on Copilot+ PCs runs Windows Settings entirely on-device.

Chrome: Gemini Nano inference extends to CPUs, so built-in AI works on many more laptops with the same Prompt API.

Headlines That Set the Stage



Gim et al. Prompt Cache: Modular Attention Reuse for Low-Latency Inference

- Providers cut your bill for reused context.
  - Prompt Caching discounts cached tokens.
- Global race for efficient AI
  - low-cost, high-quality models spark a price/perf shake-up.
  - DeepSeek had deliberately "distilled" OpenAI's models into its own.
- More efficient "mini" and reasoning-optimized models for daily use.
  - price/performance pressure that rewards lightweight deployments.

Headlines That Set the Stage

Alibaba Qwen3: open-weight family with hybrid thinking modes (/think vs /no_think) and MoE options, designed for efficient real-world deployment on modest hardware

- Efficiency breakthroughs are coming from **outside the US** too
- They all lean on smaller, smarter, often MoE-based models.

https://qwen.ai/blog?id=qwen3-livetranslate

The Cheapest Tokens Are Cached or Local

OpenAI: prompt caching now gives 90% cheaper cached input tokens, with 24-hour retention for GPT-5.1 and friends.

- OpenAI realtime-mini: text models with 10× cheaper cached tokens
- (e.g., $0.06/1M cached tokens) and lower per-million costs than full-size realtime models.

Anthropic: Claude Haiku 4.5 prices start at $1 / $5 per million (in/out) with up to 90% savings from caching and 50% from batch APIs.

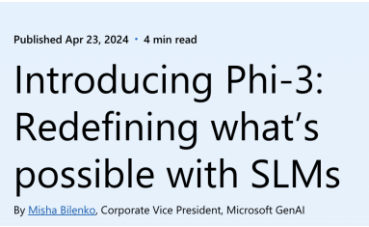
- Which positioned as the "fastest, most cost-efficient" Claude

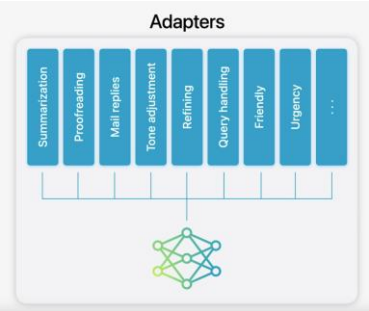
- Now, providers are literally paying you to **reuse context** and **batch work** instead of brute-forcing with bigger models.

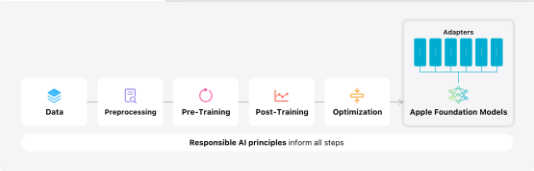
Small Models by Design

Start with small-by-default; go bigger only when SLOs require.

- Microsoft Phi-3: a family of small language models aimed at cost-effective quality.
  - "Outperform same-size and next-size up" across common tasks.
- Apple Intelligence uses small on-device models for privacy & responsiveness.
  - Private Cloud Compute handles heavier tasks with strict privacy guarantees.
  - push simple, frequent tasks on-device; surge to cloud when needed



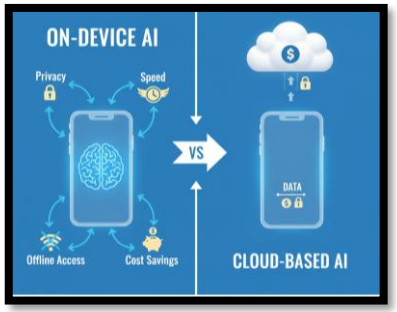




Cloud vs on-device?

It’s small-on-device + efficient cloud, with smart routing and strict SLOs.

- Apple ships custom AI servers for Private Cloud Compute, blending local NPUs with privacy-preserving cloud offload.
- These servers use Apple’s own foundation models plus a paid Gemini backend for some Siri tasks, picking the right model for the right job and budget.



Apple's Houston-built AI servers are now shipping, according to CEO Tim Cook — custom silicon to power Private Cloud Compute

News By Luke James published October 25, 2025

Apple has begun deploying custom silicon servers from a new US facility to power Private Cloud Compute, its privacy-first AI backend.

https://www.tomshardware.com/desktops/servers/apples-houston-built-ai-servers-now-shipping

Pixel Recorder moved On-Device

- The Recorder app on Pixel sees a 24% boost in engagement with Gemini Nano-powered feature
  - Faster, private summaries
  - small models unlock new UX and reduce per-request cost.

> “We were surprised by how **truly capable the model was**... before and after LoRA tuning.”
>
> **Kristi Bradford**
> Product manager for Pixel’s essential apps

https://android-developers.googleblog.com/2024/08/recorder-app-on-pixel-sees-boost-in-engagement-with-gemini-nano.html

Serving Breakthrough, vLLM & PagedAttention

- PagedAttention reduces KV-cache waste and enables continuous batching.
- Academic & industrial benchmarks show large throughput gains at similar latency.
- Business impact: same GPUs, more QPS makes lower $/answer.

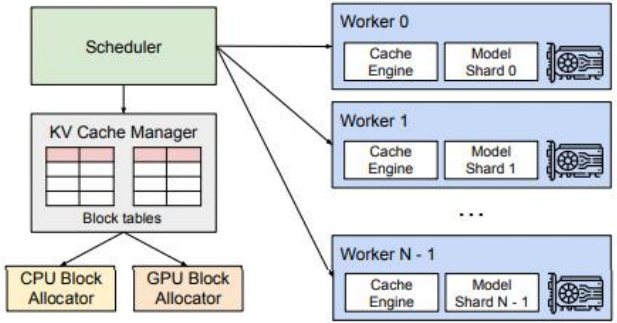


Figure 4. vLLM system overview.

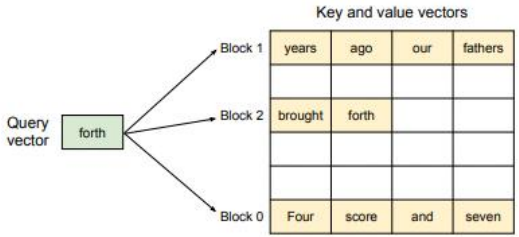
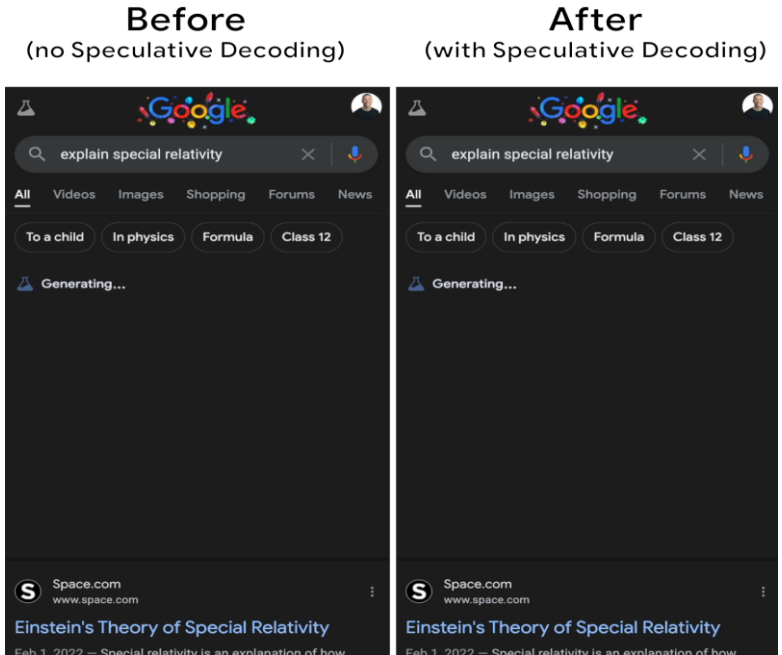


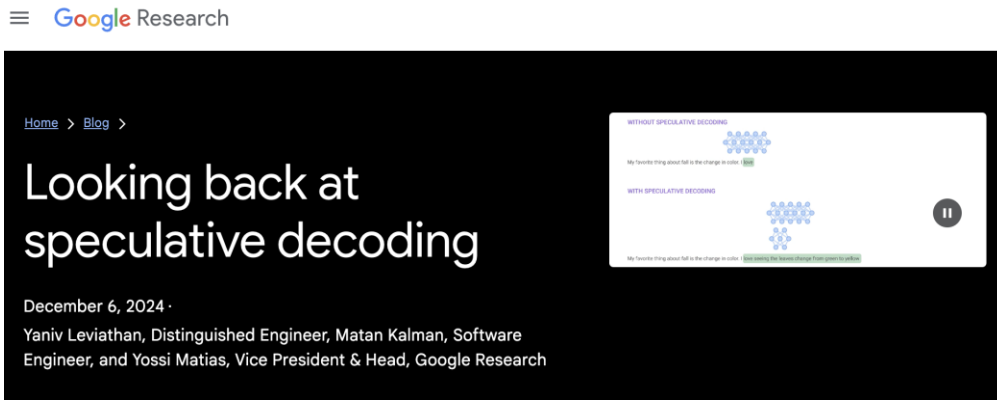
Figure 5. Illustration of the PagedAttention algorithm, where the attention key and values vectors are stored as non-contiguous blocks in the memory.

https://arxiv.org/abs/2309.06180

Decoding Gets Faster : Speculative/Drafting

- Draft-and-verify methods can reduce decoding steps by ~2× in practice.
- Pair with vLLM for multiplicative gains. We'll revisit this later session





Hardware can do	Transformers need
~100s–1000s operations/byte read	**~10** operations/byte read

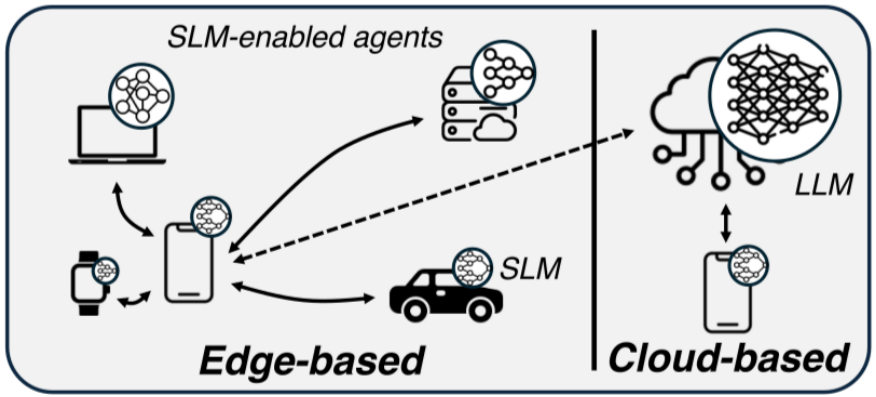
https://research.google/blog/looking-back-at-speculative-decoding/

Putting It All Together:
Why Lightweight LLMs Matter for Us

- Economics: token prices, caching discounts, and small-model SKUs mean your **unit economics can improve by 2~5 x** without inventing new models.
- Tech trend: from Apple, Microsoft, Google, DeepSeek, and Alibaba, the **common pattern** is '**small-by-default, big-when-needed**.'

How Small Models + Big Models Actually Collaborate

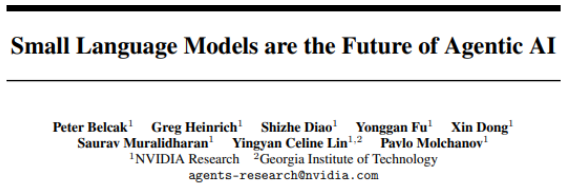
- SLM handles 60–90% of easy queries
- LLM handles only high-complexity reasoning
- Router system: confidence-based escalation
- Example: intent >> extraction >> toolcalling >> reasoning



https://www.themoonlight.io/ko/review/smaller-smarter-closer-the-edge-of-collaborative-generative-ai

Small Language Models are the Future of Agentic AI

- Modern AI agents increasingly perform small, repetitive, specialized tasks rather than open-ended conversation.
- Despite this, most agents still rely on a single large general-purpose LLM for all sub-tasks.
- Small Language Models (SLMs) are sufficiently powerful, more operationally suitable, and significantly more economical
- LLMs should be used **sparingly** for only the tasks requiring broad general reasoning.



Belcak et al. Small Language Models are the Future of Agentic AI

SLMs Are Powerful Enough

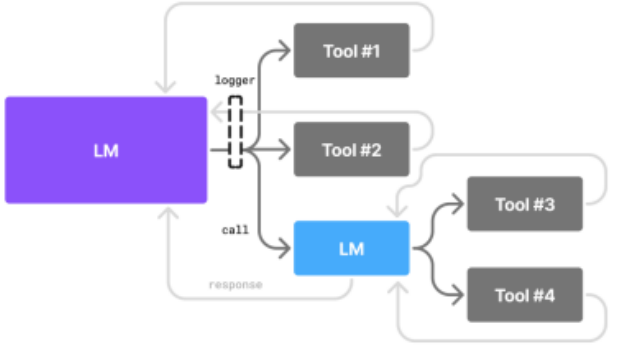
Model size is no longer the limiting factor.
>> Architecture, training data, and inference-time reasoning matter

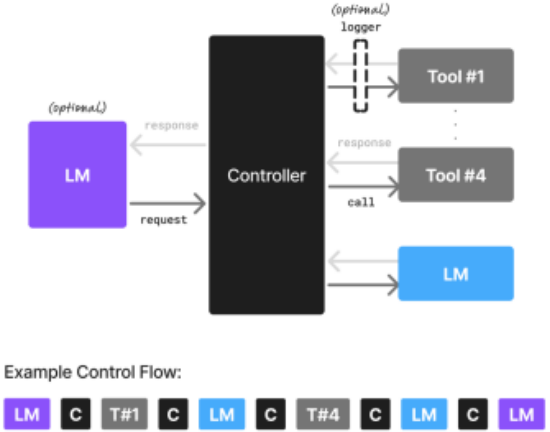
Phi-3 Small (7B)
– Matches or surpasses 70B models in language understanding & code generation.
Nemotron-H (2–9B)
– Instruction-following and coding accuracy comparable to 30B dense LLMs.
SmolLM2 (125M–1.7B)
– Reaches performance of 14B contemporaries; rivals 70B models from two years prior.
DeepSeek-R1-Distill (1.5–8B)
– 7B version outperforms Claude 3.5 Sonnet and GPT-40-0513 in reasoning.
RETRO-7.5B
– GPT-3-level performance using retrieval despite being 25x smaller.
Hymba-1.5B
– Outperforms some 13B models in instruction accuracy; 3.5x faster throughput.

How SLMs + LLMs Work Together

LM Agency (Left)
A single model (LLM or SLM) orchestrates planning and tool calling.
Suitable when a general reasoning root node is needed.

Code Agency (Right)
A controller program orchestrates the flow.
Multiple **specialized SLMs** handle subtasks (formatting, extraction, tool calling).
Only complex reasoning escalates to an LLM





- SLMs for 60~90% of calls
- LLMs only for high-complexity reasoning
- Lower cost, reduced hallucinations, higher predictability.

Figure 1: An illustration of agentic systems with different modes of agency. *Left: Language model agency.* The language model acts both as the HCI and the orchestrator of tool calls to carry out a task. *Right: Code agency.* The language model fills the role of the HCI (optionally) while a dedicated controller code orchestrates all interactions.

SLMs align with how real agent systems actually work

💰 . Cost & Efficiency

- Running a 7B model is **10–30× cheaper** than a 70–175B model (latency, FLOPs, energy).
- Minimal or zero GPU parallelism required. lower infra cost.
- Fine-tuning SLMs takes **hours, not weeks**.

🧩 . Flexibility & Modularity

- Agents naturally break tasks into small subtasks.
- SLMs can be specialized per task (intent parsing, extraction, tool calling, code generation, etc.).
- Faster updates to support new formats, regulations, or behaviors.

🎯 . Narrow Functional Needs

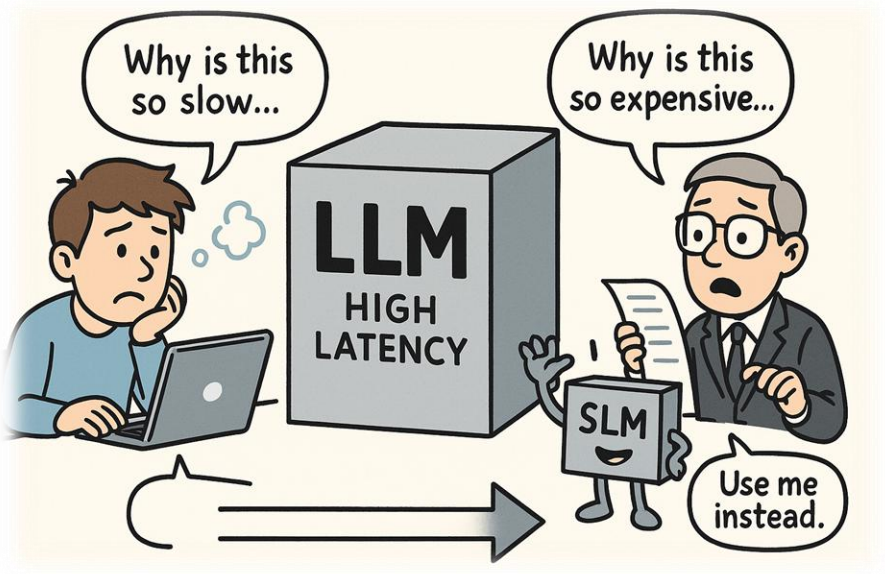
- Most agent interactions use only **very small slices of LM ability**

Agent Workflows Naturally Generate Training Data for SLMs

- Agent interactions produce structured signals: prompts, tool calls, responses, and success or failure traces.
  - These signals form high-quality, task-specific datasets with minimal manual labeling.
- Logs allow clustering of recurring behaviors and identification of SLM-specializable subtasks.
  - Fine-tuned SLMs become increasingly accurate as the agent accumulates more workflow data.
  - This creates a self-improving loop where SLMs gradually replace LLM calls.

Efficiency is the Strategy

- Users feel latency, CFOs see token & GPU bills.
- Lightweight models + smart serving hit both pain points at once.



Efficiency First: The Business Case to Reduce LLM Cost

- Most LLM costs scale with tokens and idle GPU time.
- Small, fast models can meet many SLO (Service Level Objective)s at a fraction of cost when paired with smart serving.
- Cut tokens, choose cheaper models, cache & batch, and instrument cost KPIs.

VS

Where the Money Goes

- API spend: Input/Output tokens × model rates (+ caching discounts).
- Self-host: GPU-hours × $/GPU-hr + power + engineering + utilization effects.
- Hidden factors: context length, decoding strategy, batching efficiency, cache hit rate.

Model Pricing Snapshot

For example,

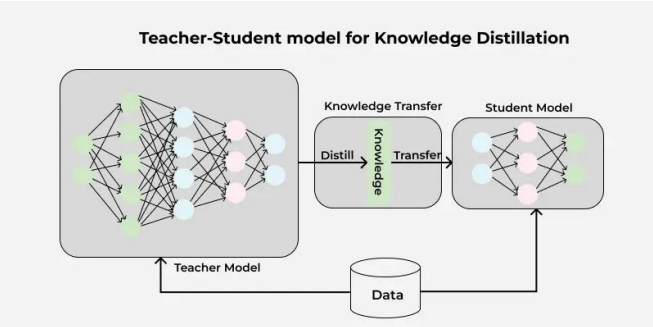
- OpenAI GPT-4.1: Input $3.00 / 1M tokens, Output $12.00 / 1M tokens.
- OpenAI GPT-4.1 mini: Input $0.80 / 1M tokens, Output $3.20 / 1M tokens.
- Anthropic Claude Sonnet (4.x): Input $3.00 / 1M, Output $15.00 / 1M.
- Assume 500 input + 250 output tokens per request.
  - GPT-4.1 ≈ $0.0045 per request , ~$4,500/month at 1M req.
  - GPT-4.1 mini ≈ $0.0012 per request, ~$1,200/month at 1M req.
  - Switching to mini in this workload saves ≈ 73.3%.

Prompt Caching & Token Hygiene

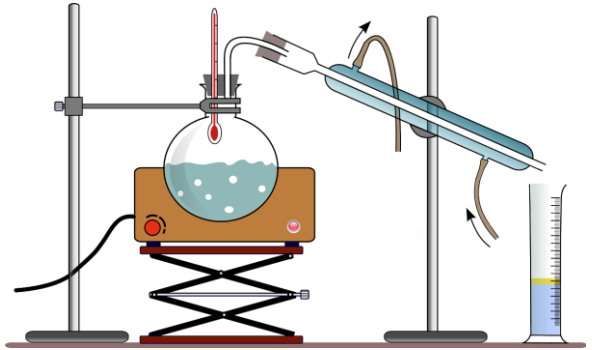
- Provider caching can discount repeated input tokens and reduce latency.
- Design for reuse: shared system prompts, retrieval templates, instruction libraries.
- Token hygiene:
  - shorten context
  - dedupe docs
  - constrain output with JSON schemas.

Choose Smaller Models Early (Distill Later)

- Start with smaller models that satisfy baseline quality.
- Use distillation to close the quality gap if needed
- Keep escape hatch: route hard queries to larger models.



https://www.geeksforgeeks.org/machine-learning/knowledge-distillation/



Serving Engine Matters (Throughput & Money 💲)

- vLLM with PagedAttention reduces KV cache waste & boosts batching.
  - Throughput can be multiples higher vs naive HF serving (same hardware).
- Features to look for:
  - Continuous batching, prefix/prompt caching, chunked prefill.
- Drafting predicts several future tokens per step.
  - Speculative decoding verifies drafts with the base model to cut steps.
- Expect speedups in decode-bound workloads; tune for quality vs speed.

Self-Host Reference Points (GPU-hr)

- GCP T_4 lists around $0.35/GPU-hr
  - newer GPUs (L_4, A100/H100) cost more but run faster.
- Batching + vLLM/TensorRT-LLM can make self-hosting cost-competitive at scale.
- Utilization is the key
  - idle GPUs erase the advantage.

Cost Simulation: Traffic Scales

- 500 input + 250 output tokens per request
- GPT-4.1 vs GPT-4.1 mini vs L4 GPU self-host

Case 1 . API: GPT-4.1 vs GPT-4.1 mini

Monthly Requests	GPT-4.1 API Cost	GPT-4.1 mini API Cost	Savings
50,000	~$225	~$60	**73%↓**
500,000	~$2,250	~$600	**73%↓**
5,000,000	~$22,500	~$6,000	**73%↓**

Case 2. **Self-host (L4 / vLLM)**

- GPU-hour: $1.2/hr (L4 on GCP)
- At 80% utilization, cost ~ **$5.8K/month** for handling ~5M requests
- At 50% utilization, cost jumps to **$9.3K/month**

Cost Simulation: Traffic Scales

- 500 input + 250 output tokens per request
- GPT-4.1 vs GPT-4.1 mini vs L4 GPU self-host

Case 1 . API: GPT-4.1 vs GPT-4.1 mini

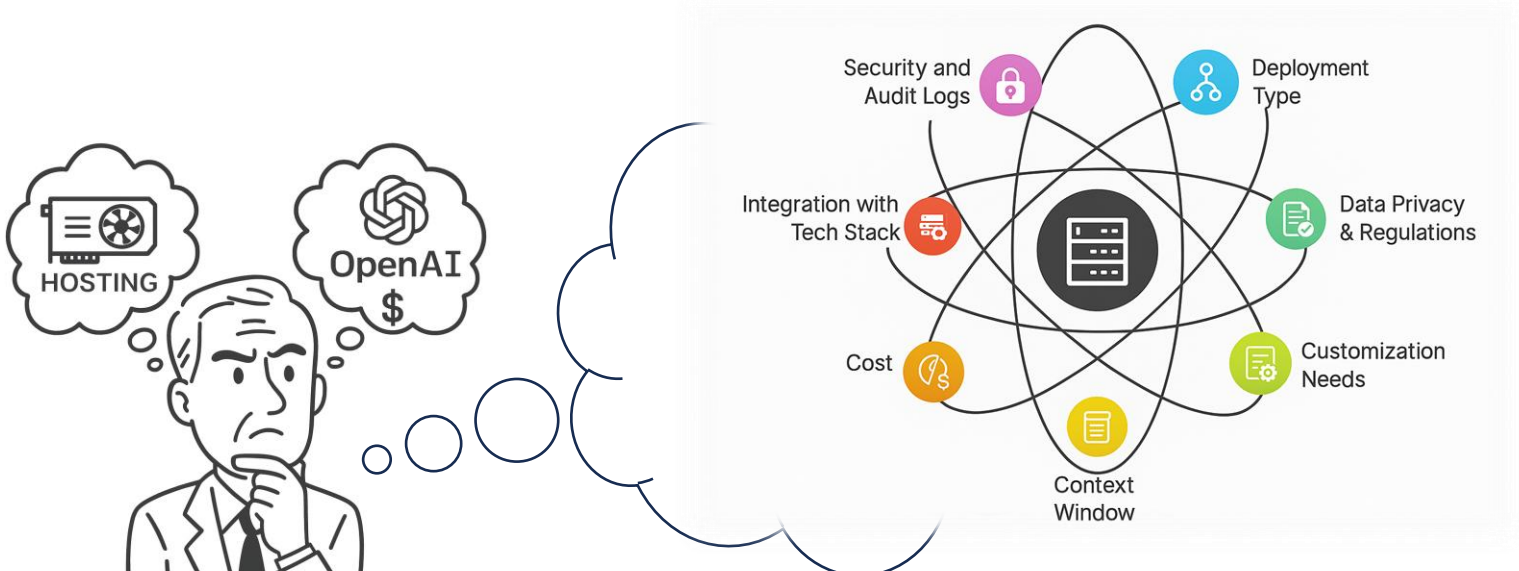
Monthly Requests	GPT-4.1 API Cost	GPT-4.1 mini API Cost	Savings
50,000	~$225	~$60	**73%↓**
500,000	~$2,250	~$600	**73%↓**
5,000,000	~$22,500	~$6,000	**73%↓**

Case 3 . Hybrid Strategy

- API for spikes (10–20% of traffic)
- Self-host for steady baseline (80–90%)
  >> Balanced cost: $6~7K/month
  >> Best latency + predictable cost + elasticity

Make or Buy?

- LOW volume / HIGH variability -> API first (pay per token).
- HIGH volume / STEADY traffic -> consider self-host (optimize $/GPU-hr).
- Hybrid: API for spikes & specialty models; self-host for steady workloads.



Top Cost Levers (Checklist)

- Model choice: small-by-default; route hard cases to large.
- Token control: shorter prompts, better RAG chunking, JSON outputs.
- Caching: provider prompt caching; prefix & KV cache reuse.
- Serving: vLLM/TensorRT-LLM; continuous batching; pinned batch sizes.
- Quantization (4/8-bit) & lightweight fine-tuning (LoRA/QLoRA).
- Speculative/drafting decoding; early stop on confidence.
- Traffic shaping: micro-batch windows; SLA-aware queuing.

Design Patterns We Can Follow..

Small-by-default, route-up when needed

- Default to o-mini / Haiku / small custom models
- route rare hard cases to a larger model

On-device for frequent, low-risk tasks

- Mu for Windows Settings, Apple 3B for UI tasks, Gemini Nano for browser helpers.

Exploit caching & KV optimizations

- Design prompts to maximize cache reuse; consider KV compression/pruning techniques.

Use distillation & fine-tuning to close quality gaps

- Start from a small open or proprietary model and distill from a stronger teacher.

Summary

Why Lightweight LLMs Became Essential

- Scaling laws are bending: small, efficient models now match or exceed last-generation LLMs.
- Lightweight models improve cost, latency, energy efficiency, and privacy by default.
- Big tech shift: “small-on-device + efficient-cloud” with smart routing and strict SLOs.

Summary

The New Efficiency Stack: Models × Serving × Tokens

- Efficiency = (Model choice) × (Serving engine) × (Token strategy).
- vLLM + PagedAttention = 2–8× throughput gains at same latency.
- Speculative/MEDUSA decoding = 2–3× decoding efficiency.
- Prompt caching & token hygiene reduce cost at scale.

Summary

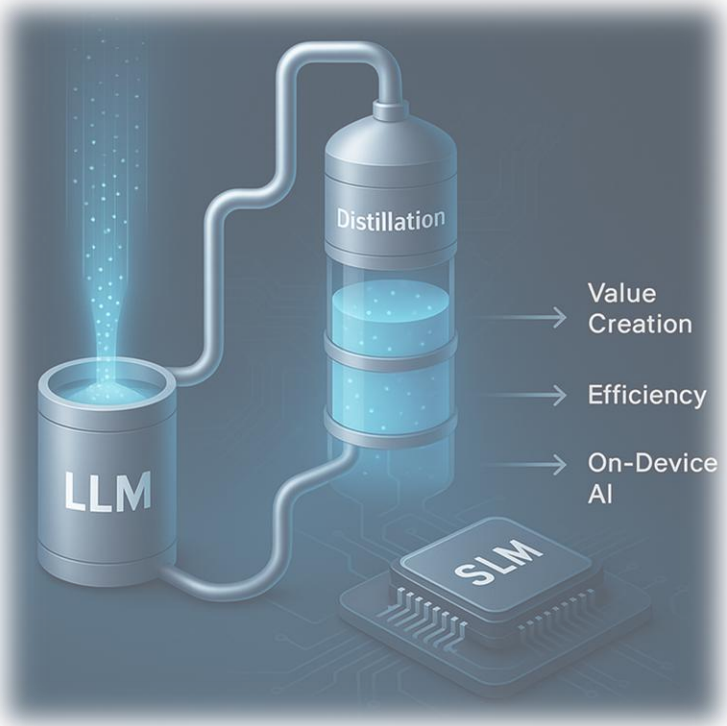
What this mean for us? Our business approach?

- Start small-by-default; distill or route up only when SLOs demand.
- Adopt on-device + efficient-cloud hybrid deployment for privacy & scale.
- Instrument cost KPIs and iterate
  - caching, batching, quantization, distillation.
- Lightweight LLMs are not just cheaper
  - They unlock new UX and new product classes.

Session 2 : U.S. Research Trends in Lightweight LLM

How Global big techs are making models cheaper, smaller, and more efficient

- Focus on recent (2025) research & product launches from U.S. companies
- See how OpenAI, Anthropic, Microsoft, Apple, Google push efficiency
- Extract design patterns we can reuse in our own lightweight LLM



Bird eye view : Efficiency Megatrends

- **Small-by-default models**: o3-mini & o4-mini (OpenAI), Claude Haiku 4.5 (Anthropic), Mu (Microsoft), Apple's 3B on-device model.
- **On-device & hybrid:** Apple Intelligence, Windows on-device agent, Gemini Nano on Android/Chrome with Prompt API and CPU inference.
- **Infra & caching:** 90%-off prompt/context caching (OpenAI GPT-5.1, Claude), KV-cache optimization and MoE server models.
    - Architectures evolved for fast inference: Princeton's hardware-efficient attention (GLA/GTA), new decoding kernels, and KV-cache research matured.

Open AI : Cost-efficient reasoning model

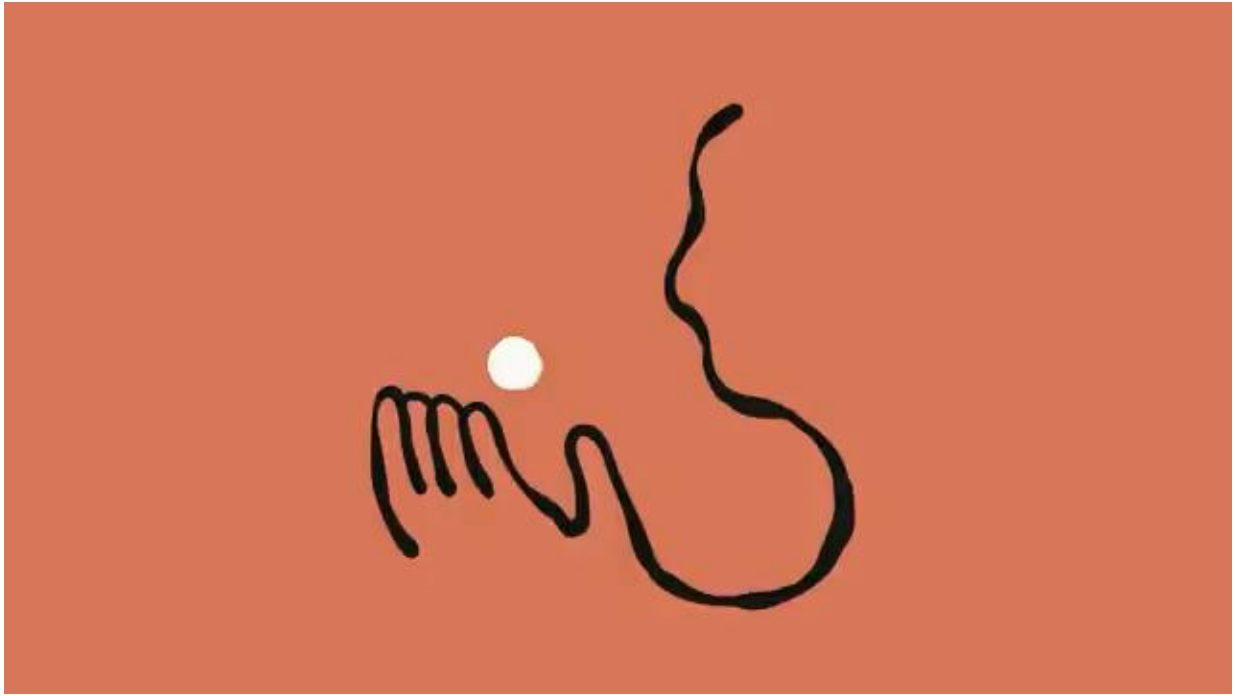
- o3-mini (Jan 2025): strong STEM/coding with low cost & latency
- o4-mini (Apr 2025): smaller model optimized for fast, cost-efficient reasoning, top performance on AIME 2024/2025 benchmarks.
- GPT-5.1 prompt caching (2025): cached input tokens 90% cheaper (e.g. $0.125/M vs $1.25/M), extended 24h retention. Pattern: small reasoning models + caching fundamentally change $/query economics.

OpenAI frames these as reasoning-oriented small models, studied under its preparedness and safety frameworks.

https://www.anthropic.com/news/claude-haiku-4-5

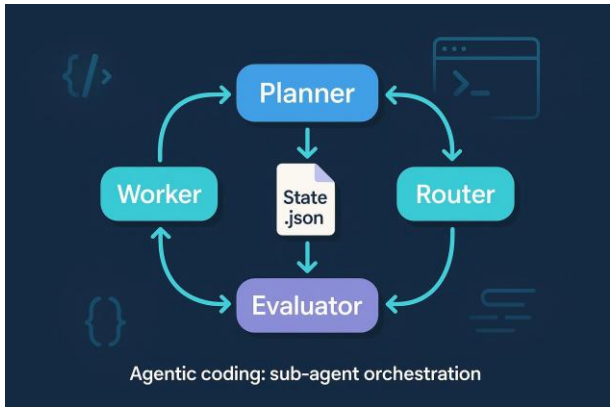
Anthropic

- Claude Haiku 4.5 (Oct 2025): Anthropic's fastest, most cost-efficient model, matching or surpassing Sonnet 4



Anthropic AI: Lightweight LLMs & Multi-agent orchestration

- Claude Haiku 4.5 (Oct 2025): Anthropic's fastest, most cost-efficient model, matching or surpassing Sonnet 4
- Cost is about 1/3 the price of Sonnet 4 and 1/15 of Opus, targeted at high-volume enterprise workloads.
- Multi-agent orchestration : Sonnet 4.5 can break a task into subtasks and coordinate a "team of Haikus"



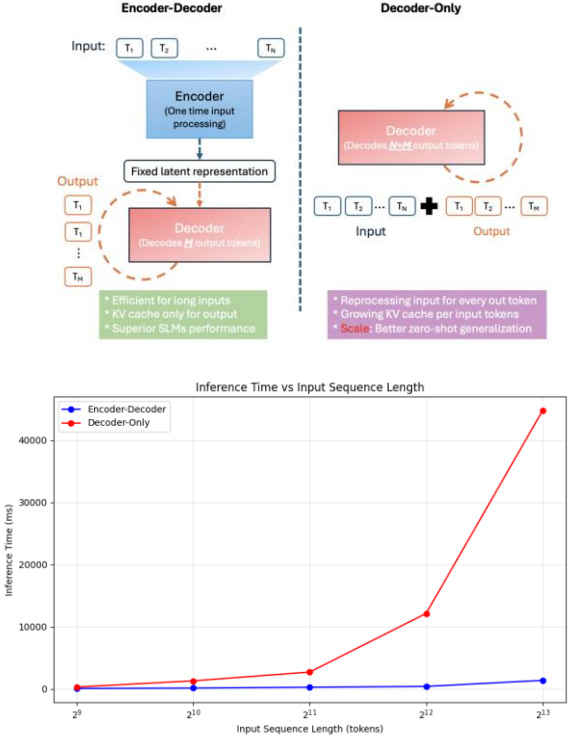
https://skywork.ai/blog/agentic-coding-claude-haiku-4-5-beginners-guide-sub-agent-orchestration/

https://blogs.windows.com/windowsexperience/2025/06/23/introducing-mu-language-model-and-how-it-enabled-the-agent-in-windows-settings

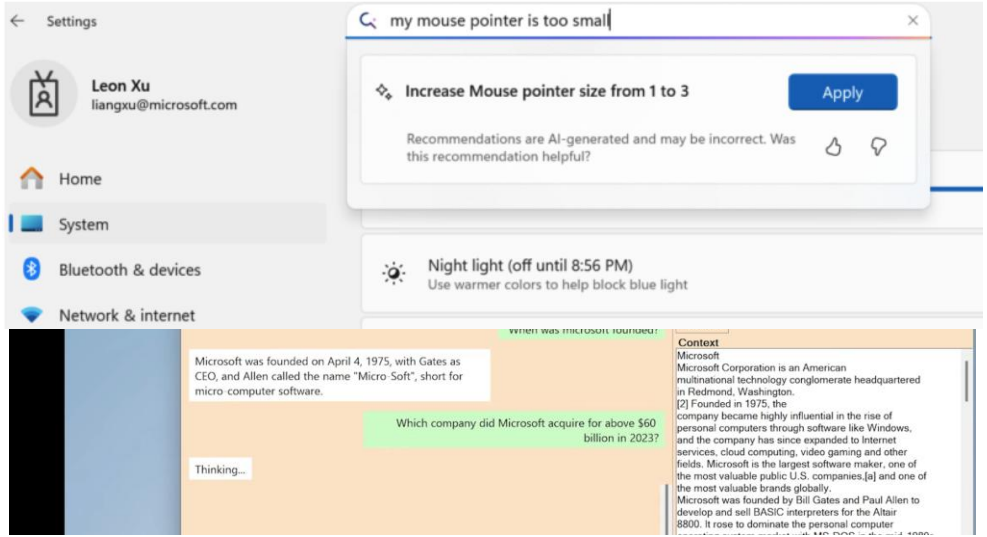
Microsoft : Mu & On-Device NPU Research

- Mu language model (Jun 2025): 330M encoder–decoder LM optimized for NPUs on Copilot+ PCs, enabling the Windows Settings agent to run fully on-device.
- Training recipe: hybrid dataset (~3.6M examples) + noise injection, instruction tuning, and LoRA for robustness & efficiency.
- Broader research: on-device NPU inference systems (NPU offloading to cut prefill latency) influence this design space.

By pairing state-of-the-art quantization techniques with hardware-specific optimizations



Encoder-Decoder Architecture (Mu) compared to Decoder-only Architecture (GPT)



"It has the fast token throughputs and ultra-fast time to first token responses despite the large amount of input context provided to the model."

https://blogs.windows.com/windowsexperience/2025/06/23/introducing-mu-language-model-and-how-it-enabled-the-agent-in-windows-settings

Apple : On-device, efficiency and privacy

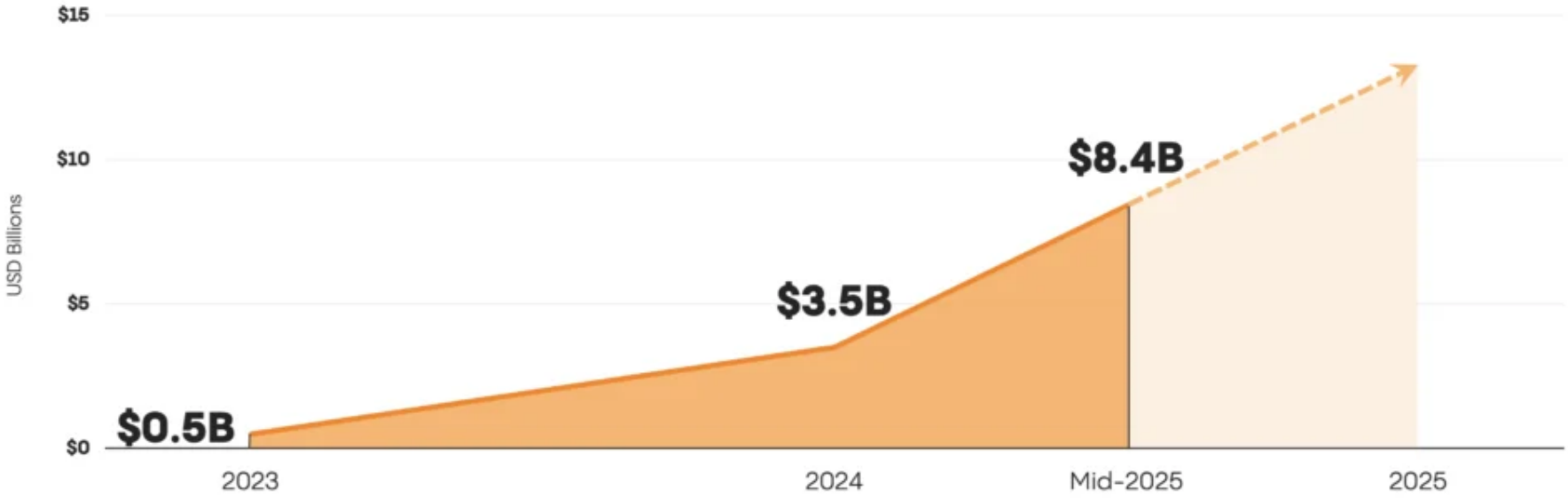
- Apple Intelligence Foundation Language Models: Tech Report 2025 introduces two core models
  - An ~3B-parameter on-device multilingual, multimodal model with KV-cache sharing and 2-bit quantization-aware training (QAT), optimized for Apple silicon
  - A scalable Parallel-Track Mixture-of-Experts (PT-MoE) server model with interleaved global-local attention for Apple's Private Cloud Compute.
- Research contributions: architectural tweaks, data curation, and inference optimizations explicitly framed around efficiency and privacy.

Google : Gemini Nano as a Local AI Runtime

- Android AICore & Gemini Nano: system-level service running on-device LLMs for features like summaries, replies, and translations, optimized for low-latency.
- Chrome Prompt API (2025): lets web apps call Gemini Nano directly in the browser; recent updates add CPU inference so more laptops can run local AI.
- Research direction: treat Nano as a standard local inference layer, while heavier Gemini models live in the cloud for complex tasks.

Enterprise* Spend on Foundation Model APIs Continues to Grow

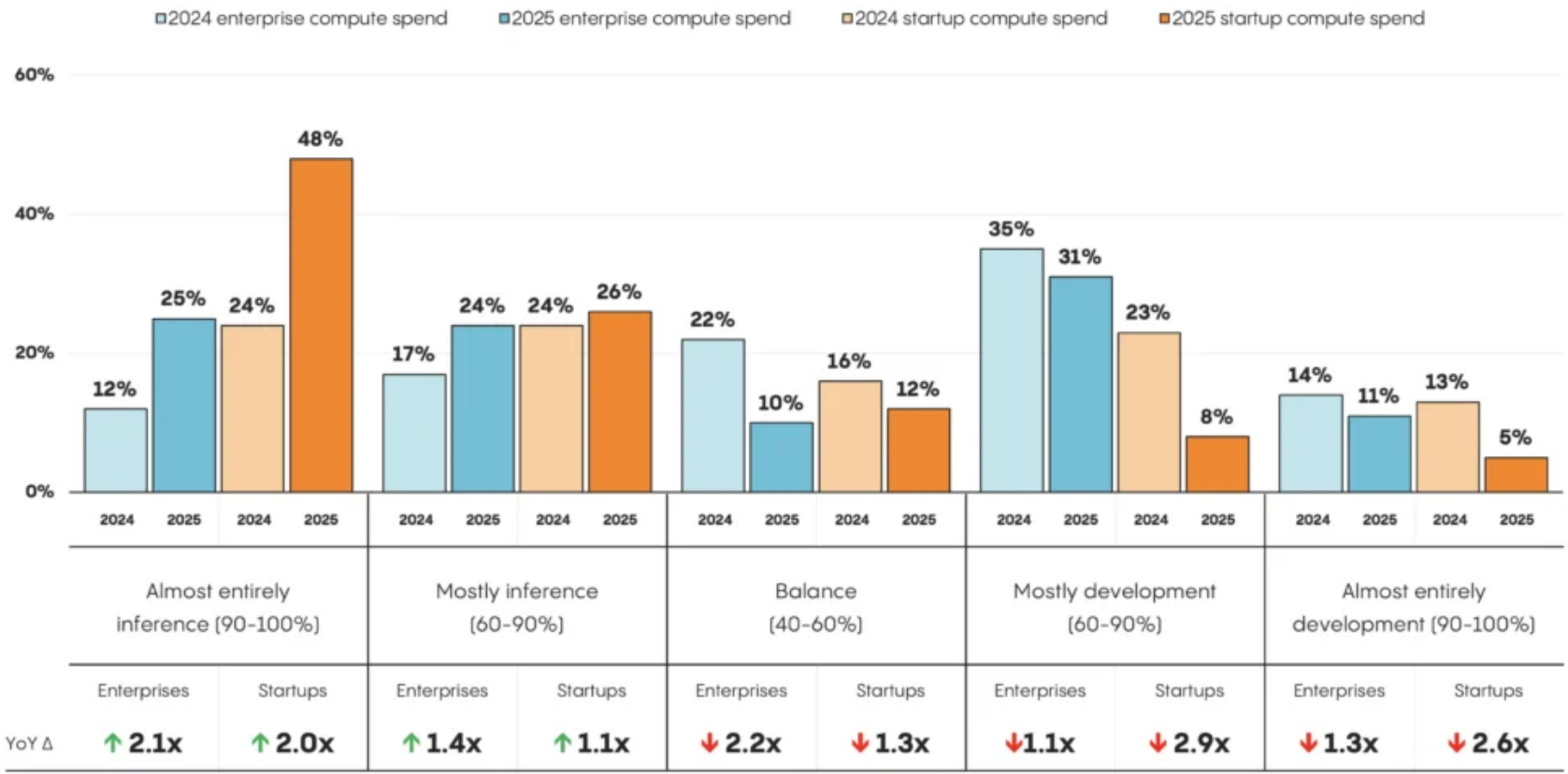
Enterprise spend in the first six months of 2025 has already more than doubled all of 2024



*Non-consumer companies, including both enterprises and startups

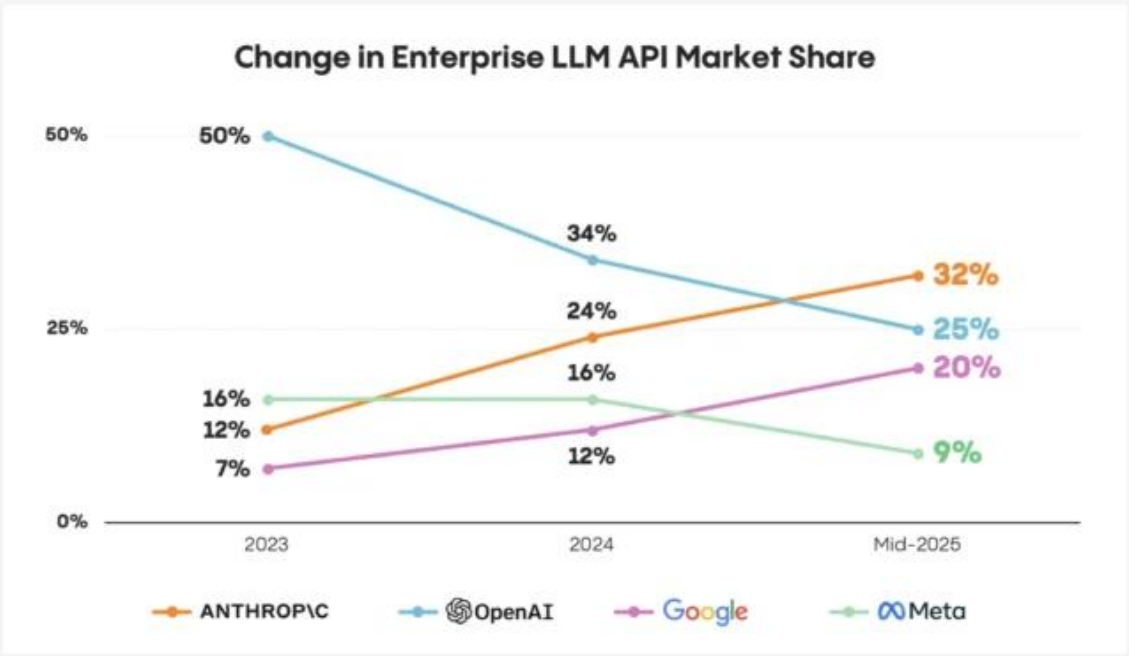
https://menlovc.com/perspective/2025-mid-year-llm-market-update/

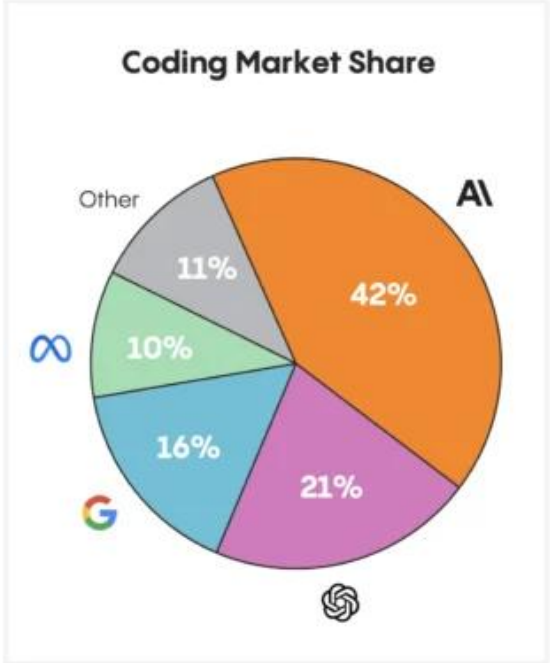
Shift in Compute Spend: Inference vs. Development



https://menlovc.com/perspective/2025-mid-year-llm-market-update/

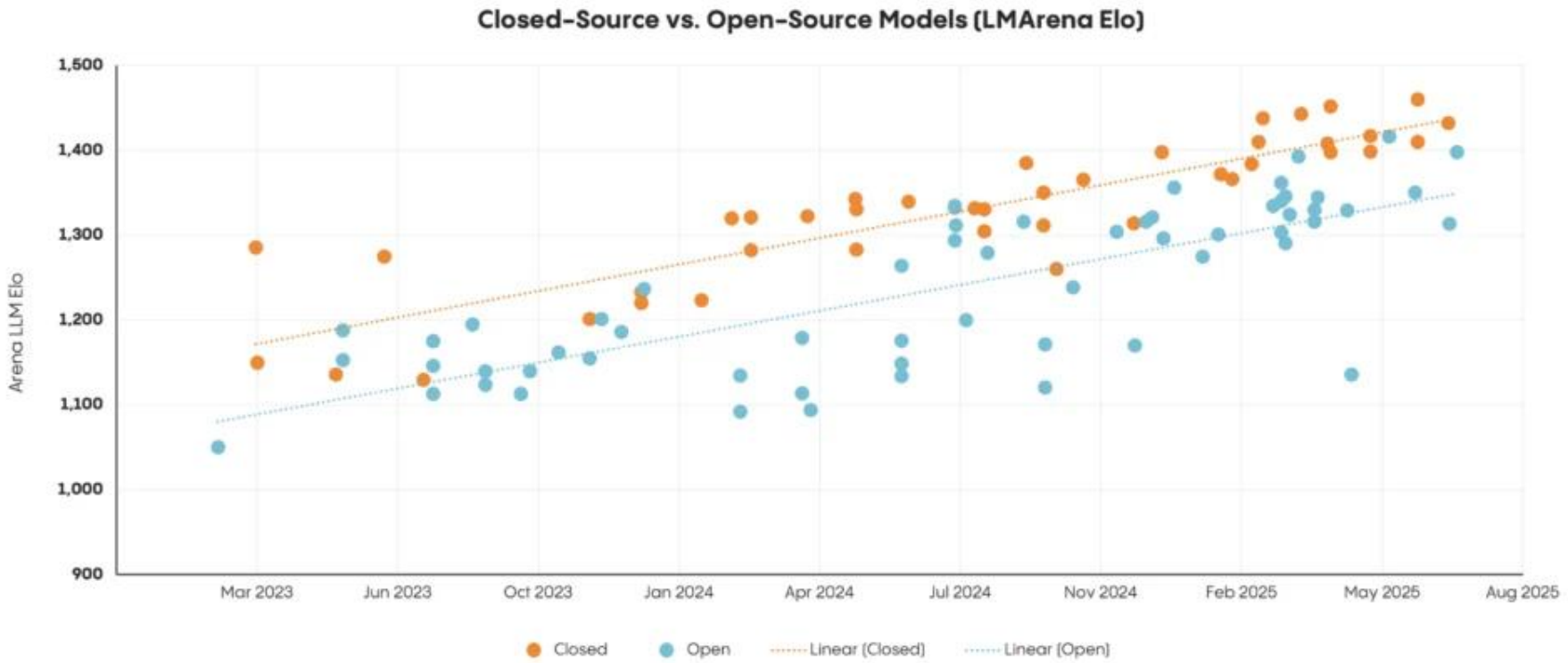
Enterprise LLM API Market Share by Usage





https://menlovc.com/perspective/2025-mid-year-llm-market-update/

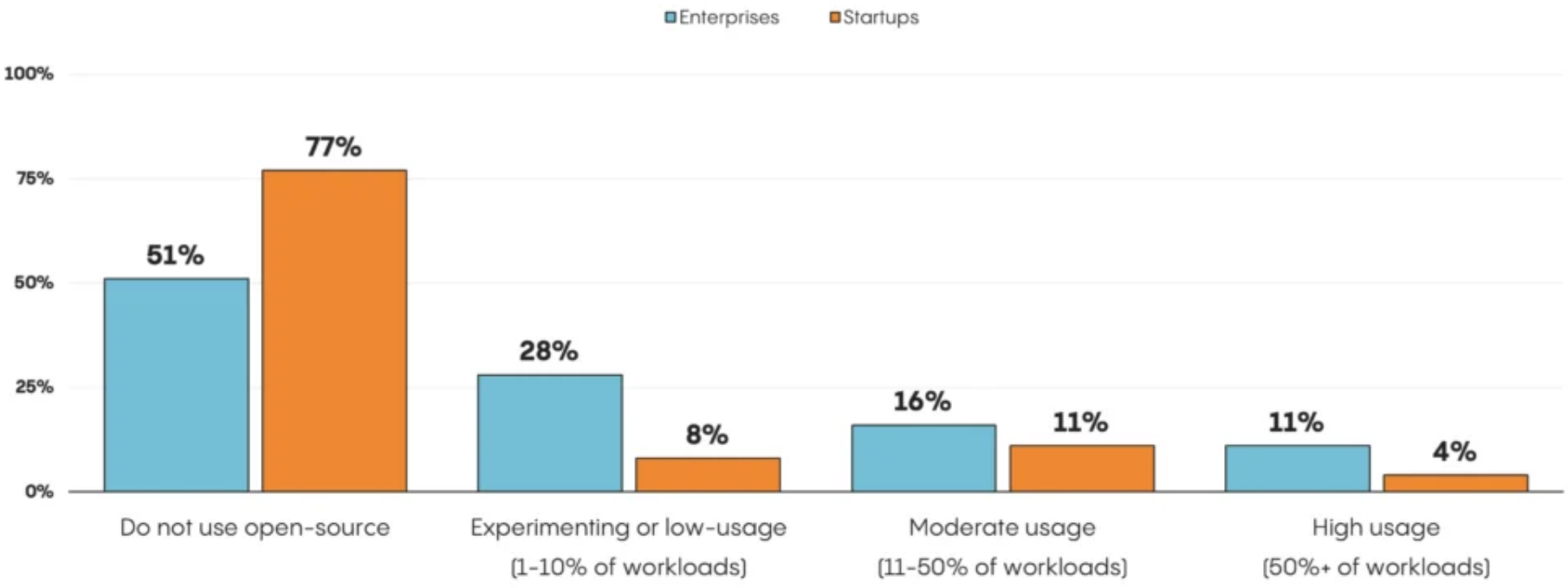
Closed-Source vs. Open-Source Models



Source: LMArena

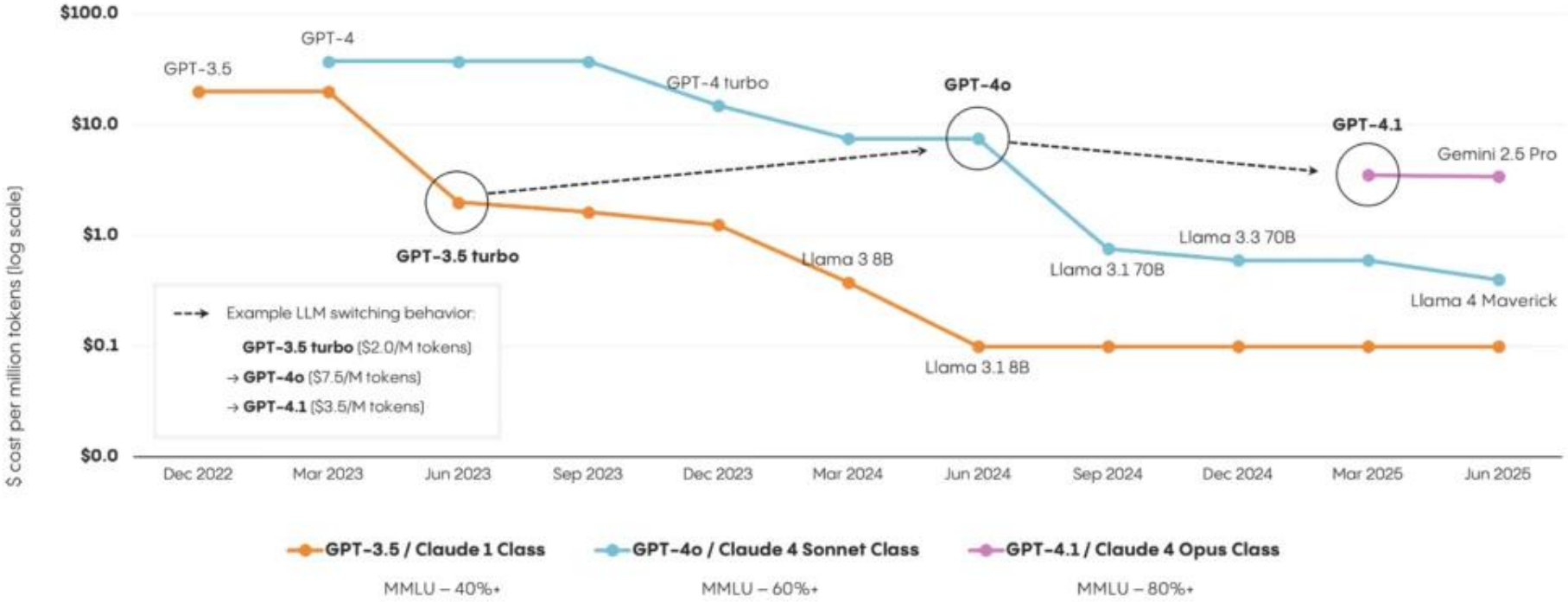
https://menlovc.com/perspective/2025-mid-year-llm-market-update/

Companies Choose Closed-Source



https://menlovc.com/perspective/2025-mid-year-llm-market-update/

Builders Pay to Stay on the Frontier Despite 10x Annual Cost Decreases



Source: Artificial Analysis

https://menlovc.com/perspective/2025-mid-year-llm-market-update/

Serving Breakthrough vLLM : vLLM V1

- Zero-overhead prefix caching in V1 (near-zero CPU cost even at 0% hit-rate)
- On by default in V1 engine; simple migration from V0
- TPU backend: unified JAX + PyTorch path, broader coverage
- Lesson: engine-first wins (TPS/QPS gains without model changes)

Academic Focus : KV Cache is the Bottleneck

- KV cache = the model’s short-term memory of past tokens
- It avoids re-computing attention for the whole history
- But it grows with context → big memory & latency cost
- KV memory dominates long-context latency/cost
- Tackle with quantization/compression + smart policies

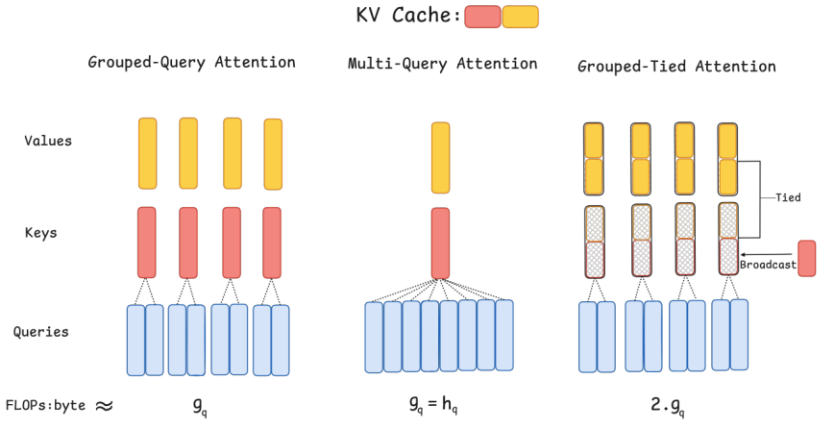
Academic Focus : Algorithms for Faster Decode

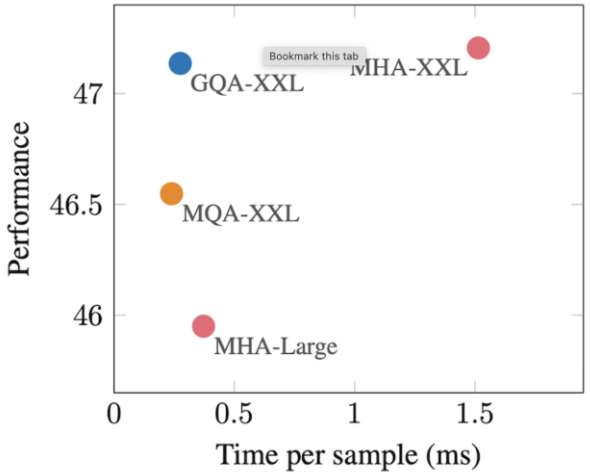
- A small **draft model** guesses several tokens
- The **main model** quickly **verifies or fixes** them
- Fewer total steps -> **faster decoding**

- ReDrafter (ICLR'25): state-of-the-art speculative drafting
- Diffusion-based drafting (NAACL'25): parallelize draft + verify
- Hierarchical/training-free variants emerge
- Engine hooks: vLLM / TensorRT-LLM speculative decoding

Academic Focus : Hardware-Efficient Attention

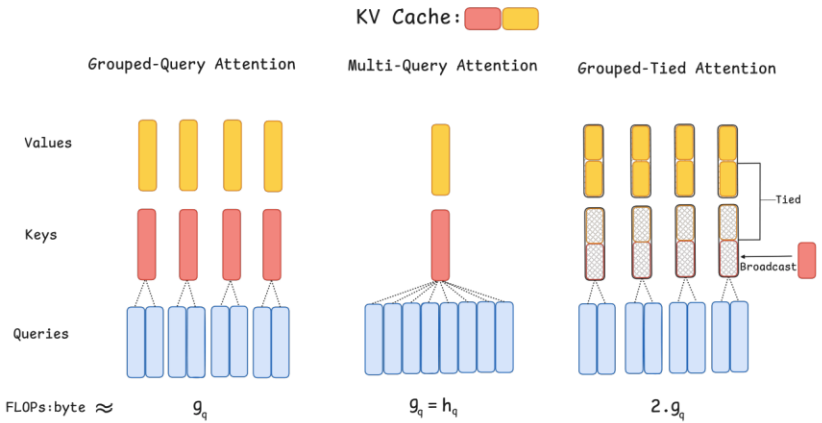
- GQA (Grouped-Query Attention): many Q heads, but fewer K/V heads shared -> smaller KV cache
- KV memory shrinks roughly in proportion to K/V heads
- Use when context is long or VRAM is tight





Academic Focus : Hardware-Efficient Attention

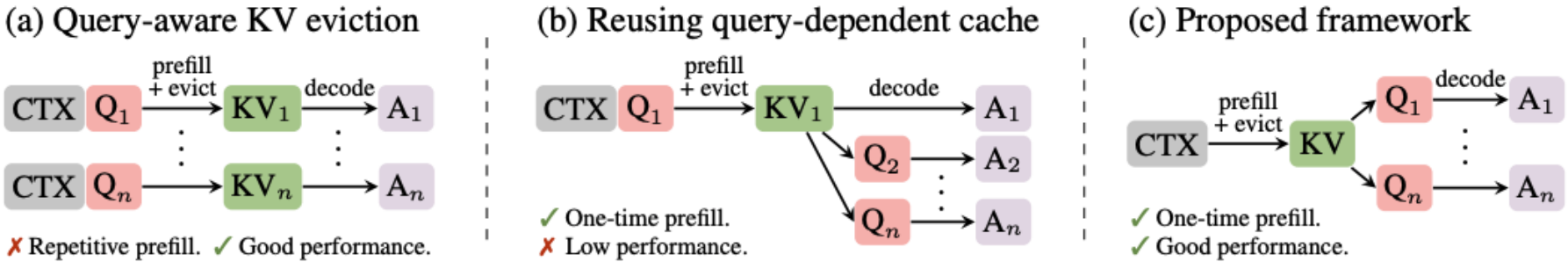
- GTA: matches GQA quality with ~1/2 KV footprint
- GLA: parallel-friendly latent attention; up to ~2× decoding-kernel speedup vs FlashMLA
- Use when context is long & KV dominates



Overview of Grouped-Tied Attention (GTA)

KVzip: Compressing KV Without Retraining

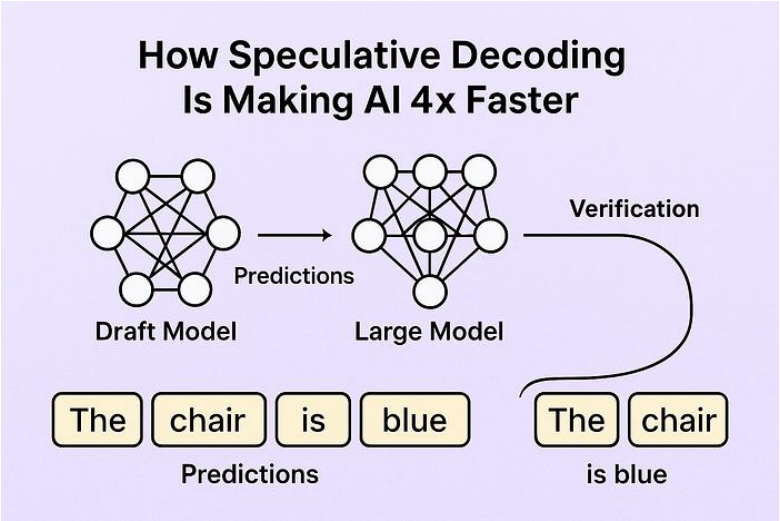
- 3~4× smaller KV around 2× faster decoding
- Query-agnostic: one compressed KV works across prompts
- Drop-in for long-chat products

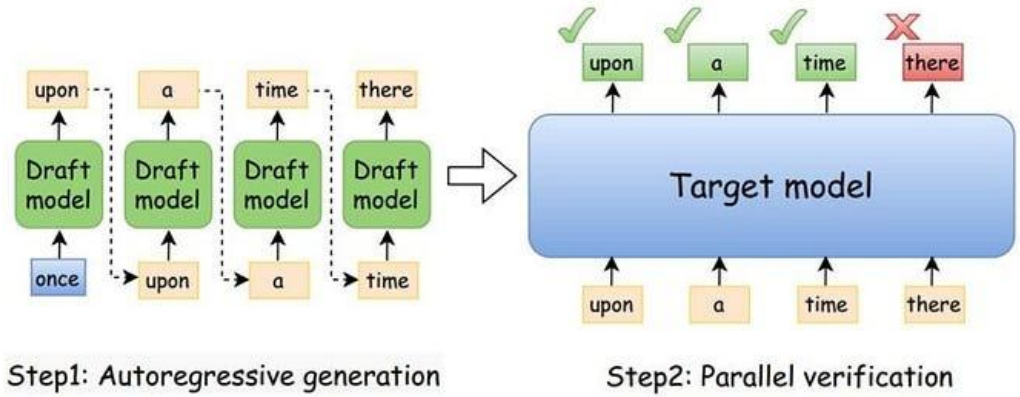


https://www.arxiv.org/abs/2505.23416

Decoding Gets Faster : Speculative/Drafting

- Draft-and-verify methods can reduce decoding steps by ~2× in practice.
- Great for decode-bound workloads, with minor quality tuning.





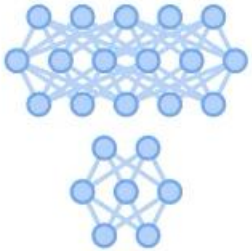
https://medium.com/@genai.works/speed-up-llm-inference-with-speculative-decoding-1fc79701e9d6

Decoding Gets Faster : Speculative/Drafting

WITHOUT SPECULATIVE DECODING

My favorite thing about fall

WITH SPECULATIVE DECODING



My favorite thing about fall

https://research.google/blog/looking-back-at-speculative-decoding/

There are keywords along with SLMs

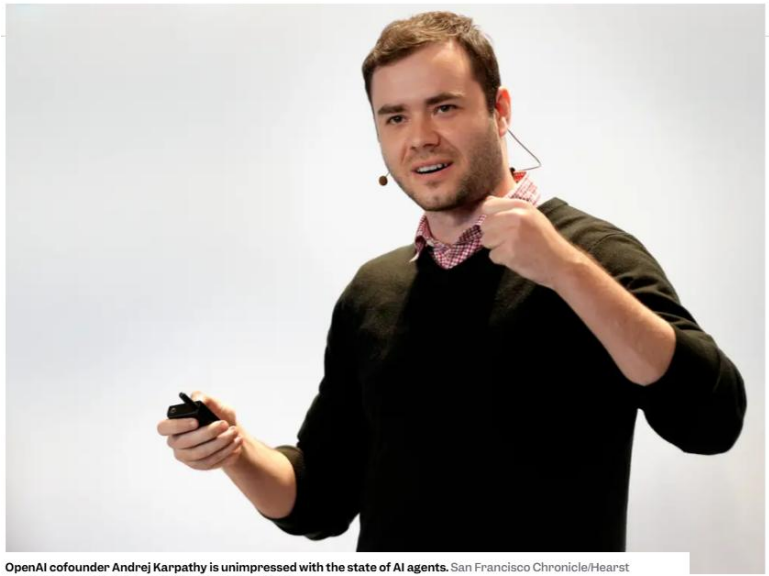
Most potential for scaling LLMs

- 🤖 Agent
- ⏱️ Test-Time Scaling
- 🧪 Synthetic Data
- 🛠️ Post-training
- ⚡ Inference-time reduction

Agent! we're at the start of the **decade of agents**

OpenAI cofounder Andrej Karpathy says it will decade before AI agents actually work

By Lakshmi Varanasi + Follow



OpenAI cofounder Andrej Karpathy is unimpressed with the state of AI agents. San Francisco Chronicle/Hearst Newspapers via Getty Images/Contributor/Getty Images

https://www.businessinsider.com/andrej-karpathy-ai-agents-timelines-openai-2025-10

High-level Architecture of Advanced Research

Claude.ai chat

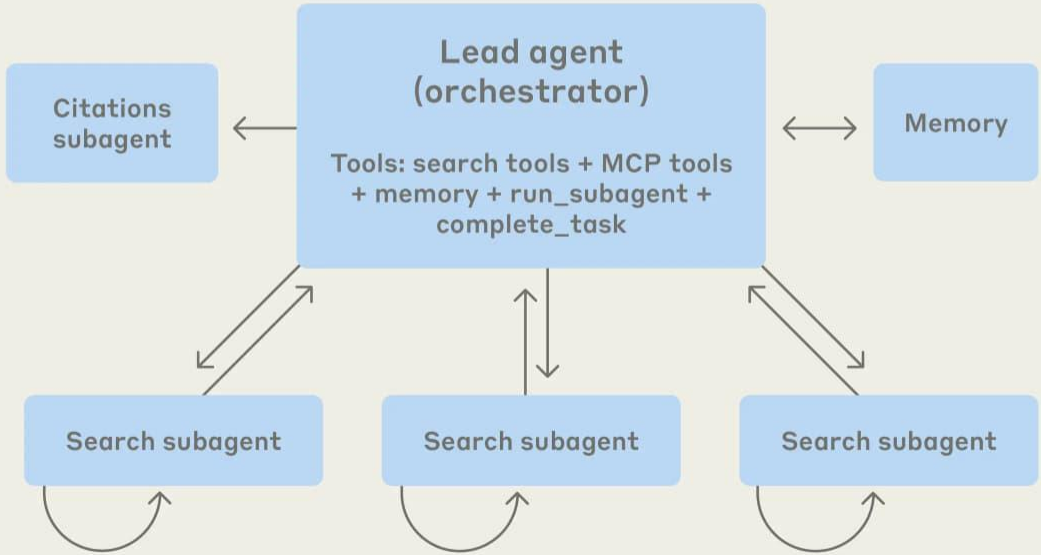
What's on your mind tonight?

what are all the companies in the united states working on AI agents in 2025? make a list of at least 100. for each company, include the name, website, product, description of what they do, type of agents they build, and their vertical/industry.

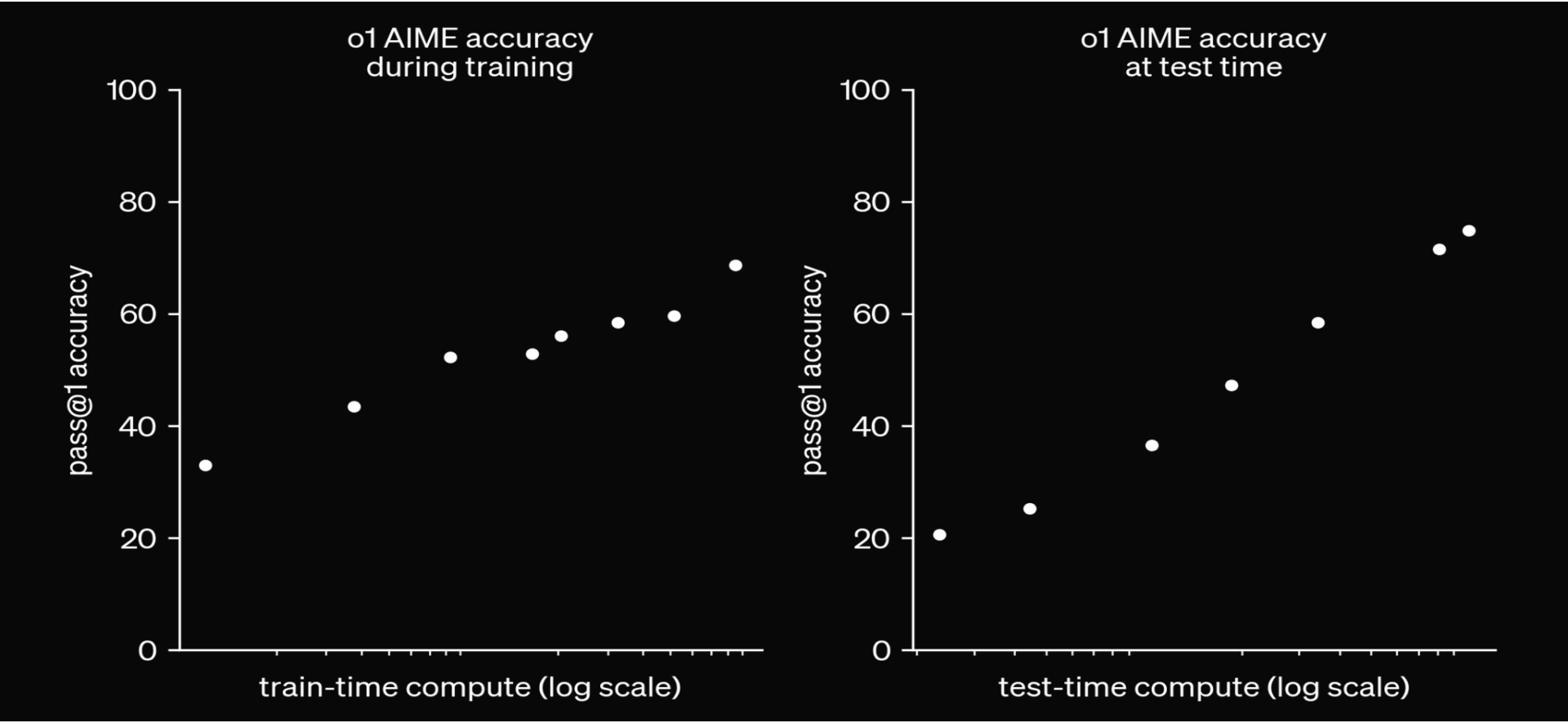
User request

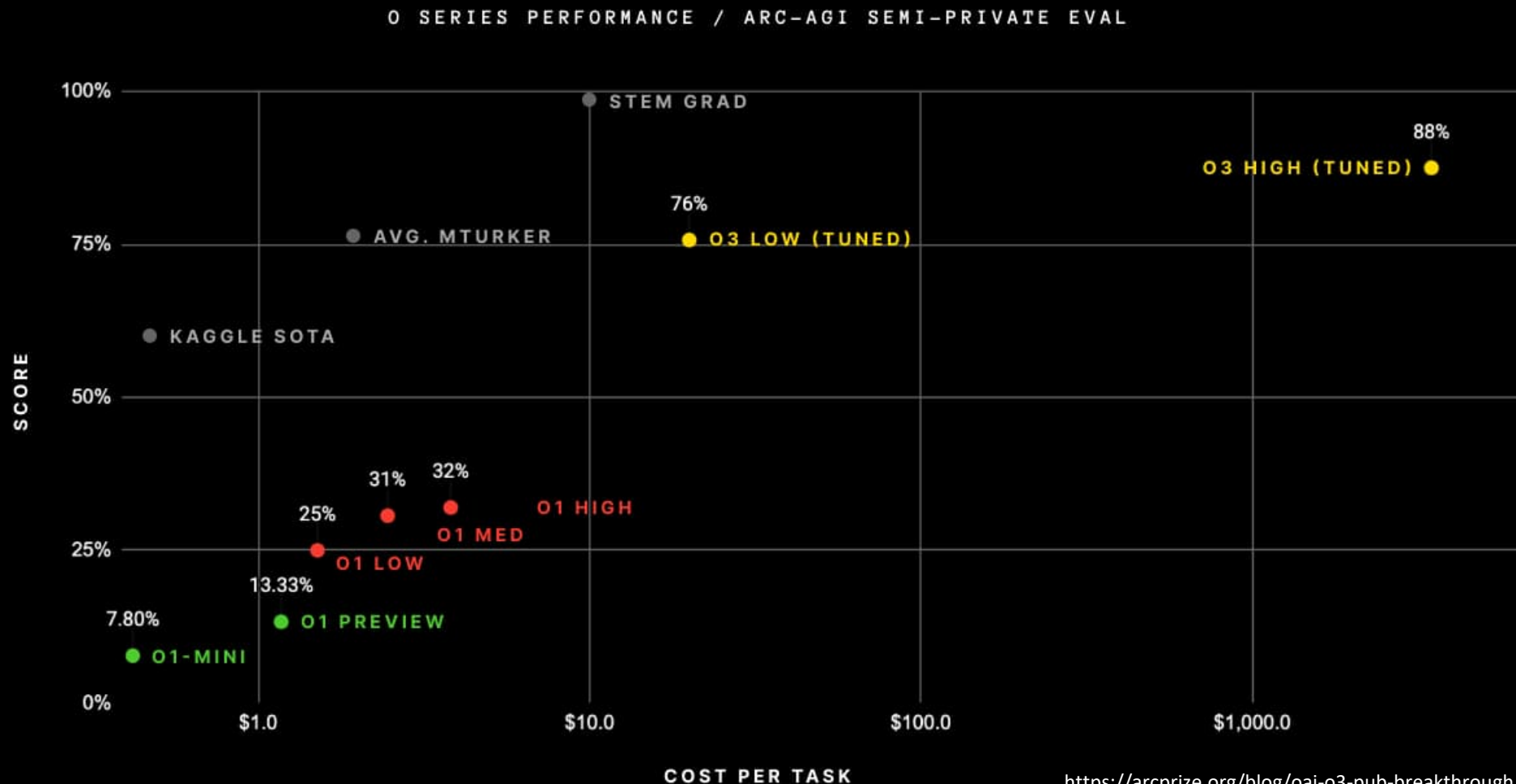
Final report

Multi-agent research system

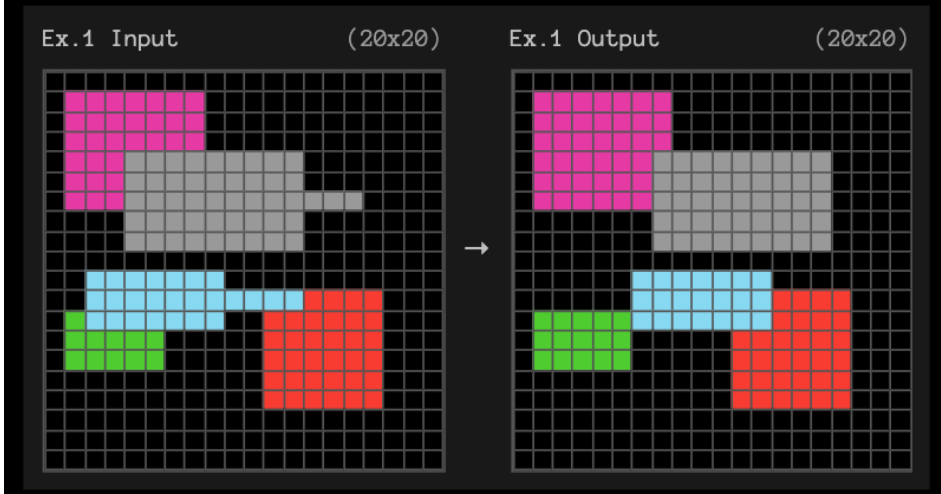


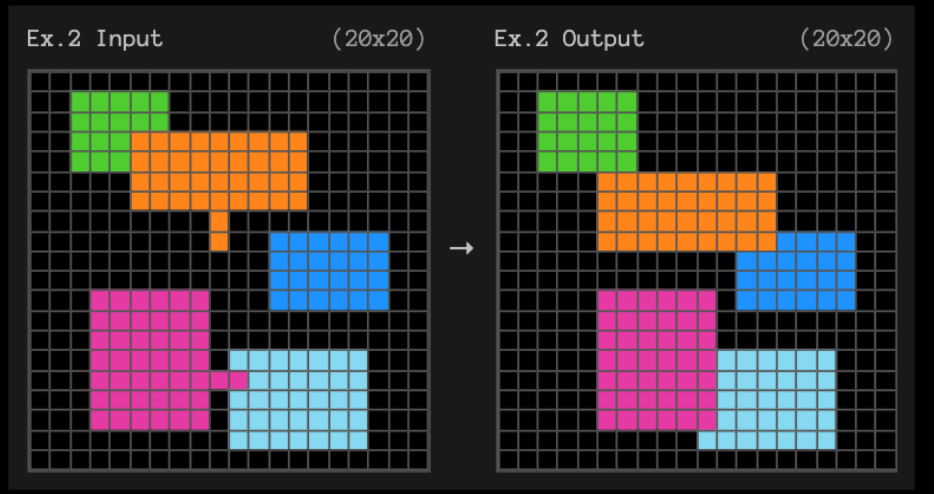
Test-Time Compute

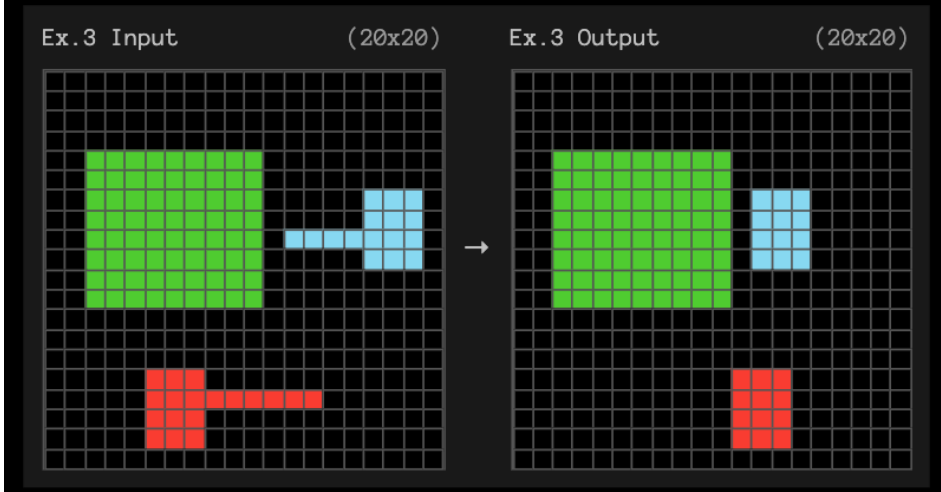


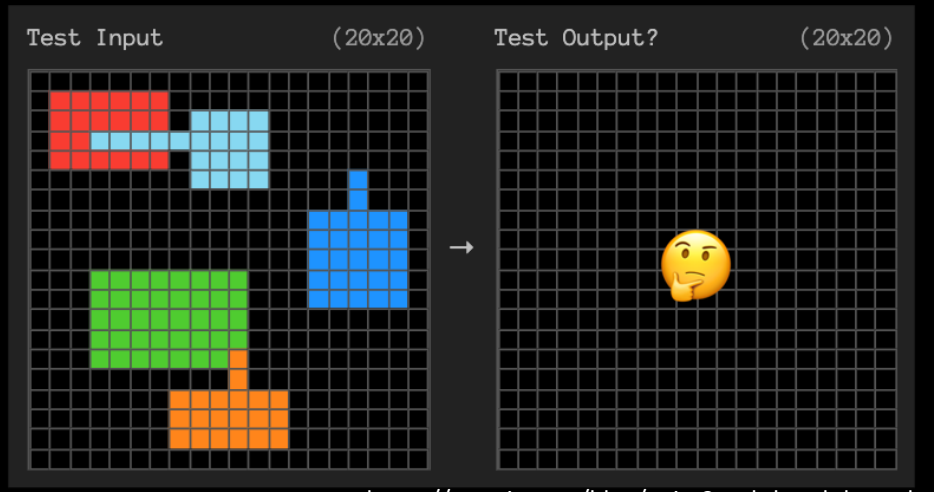


https://arcprize.org/blog/oai-o3-pub-breakthrough



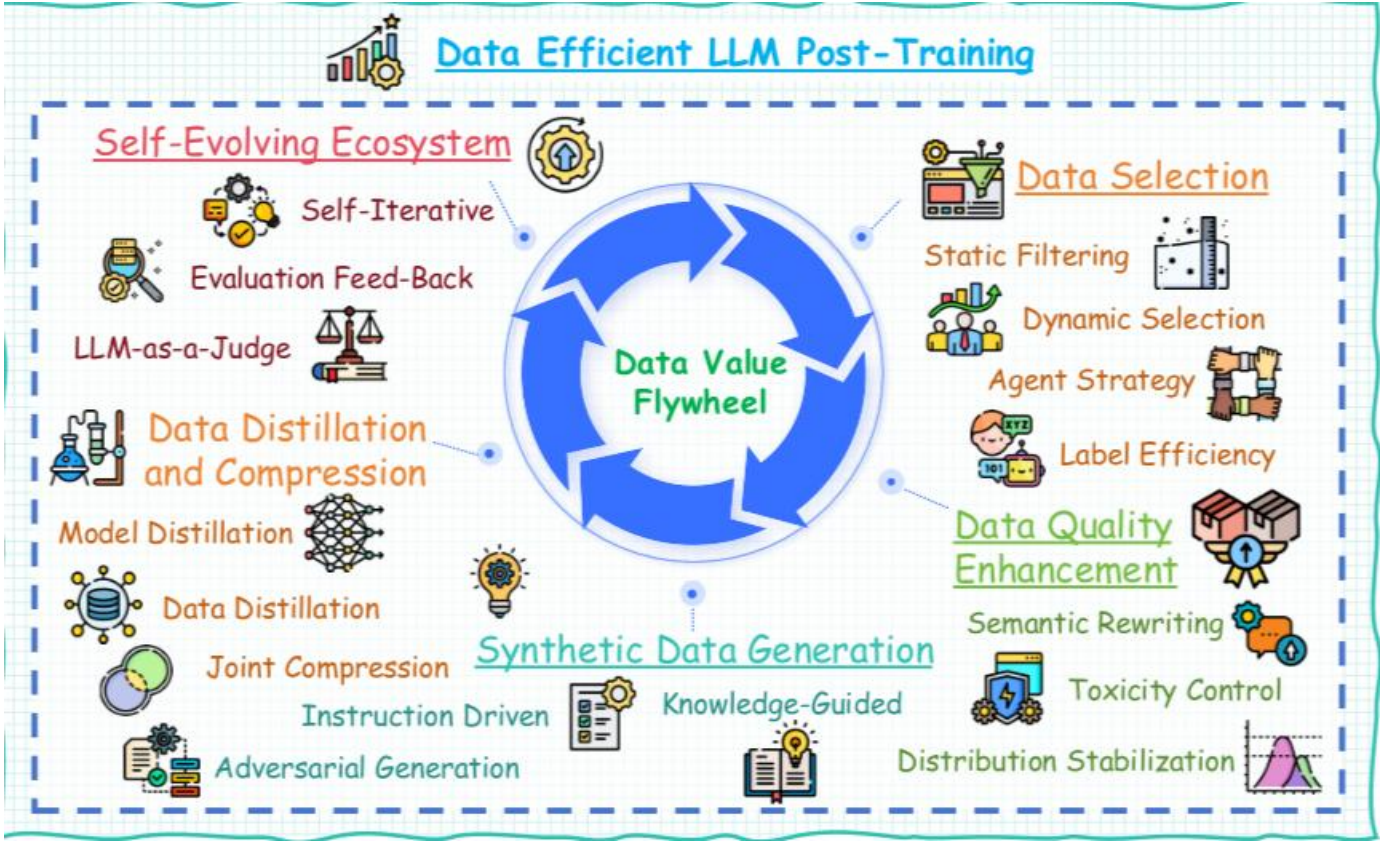






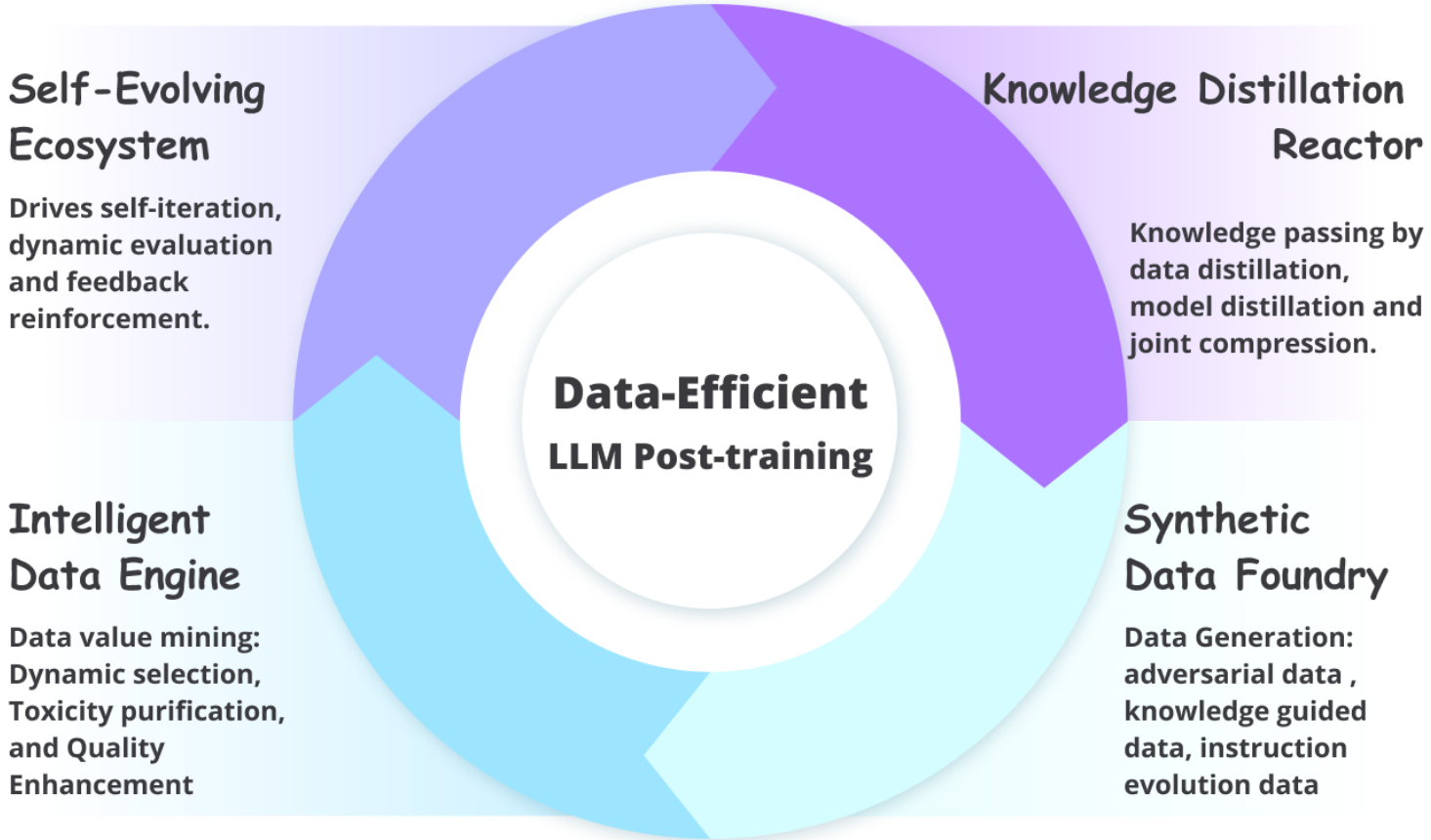
https://arcprize.org/blog/oai-o3-pub-breakthrough

Post-Training



Luo et al. A Survey on Efficient Large Language Model Training: From Data-centric Perspectives

Post-Training



https://github.com/luo-junyu/Awesome-Data-Efficient-LLM

Synthetic Data :Llama

Meta

The Llama 3 Herd of Models

Llama Team, AI @ Meta[1]
[1]A detailed contributor list can be found in the appendix of this paper.

Modern artificial intelligence (AI) systems are powered by foundation models. This paper presents a new set of foundation models, called Llama 3. It is a herd of language models that natively support multilinguality, coding, reasoning, and tool usage. Our largest model is a dense Transformer with 405B parameters and a context window of up to 128K tokens. This paper presents an extensive empirical evaluation of Llama 3. We find that Llama 3 delivers comparable quality to leading language models such as GPT-4 on a plethora of tasks. We publicly release Llama 3, including pre-trained and post-trained versions of the 405B parameter language model and our Llama Guard 3 model for input and output safety. The paper also presents the results of experiments in which we integrate image, video, and speech capabilities into Llama 3 via a compositional approach. We observe this approach performs competitively with the state-of-the-art on image, video, and speech recognition tasks. The resulting models are not yet being broadly released as they are still under development.

Date: July 23, 2024
Website: https://llama.meta.com/

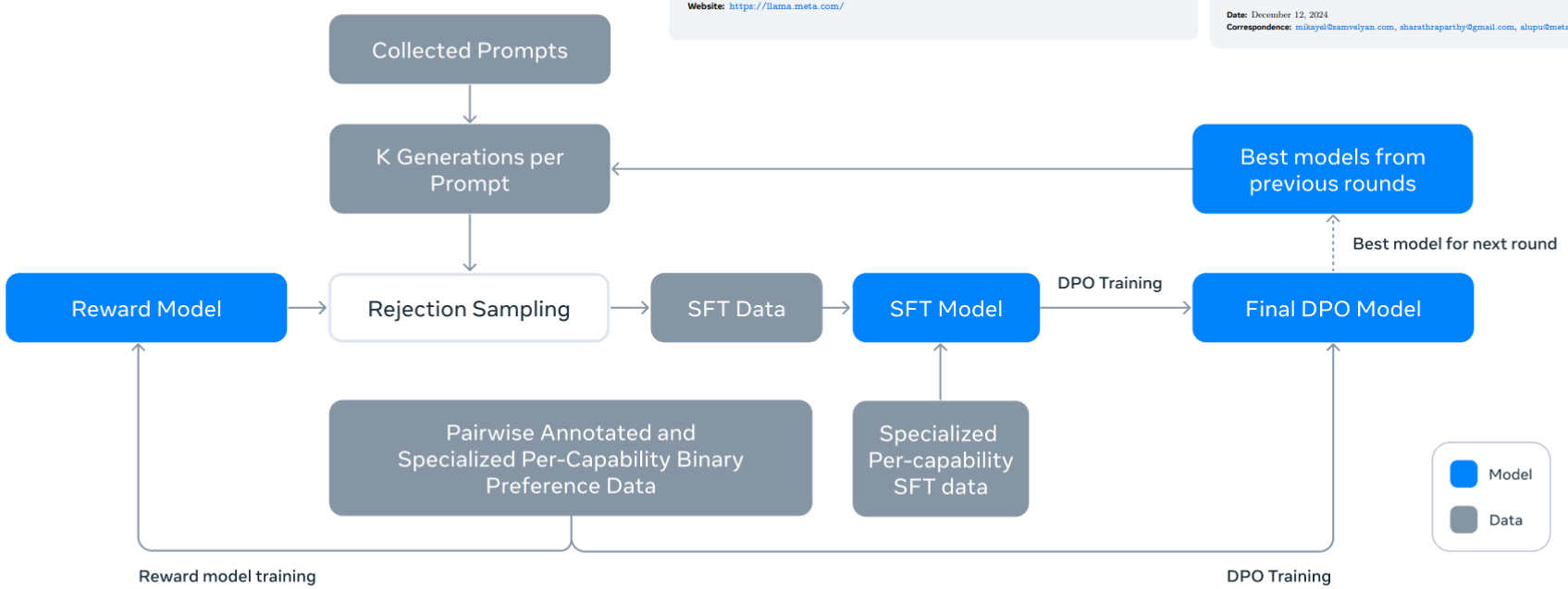
Rainbow Teaming:
Open-Ended Generation of Diverse Adversarial Prompts

Mikayel Samvelyan[*,1,2], **Sharath Chandra Raparthy**[*,1], **Andrei Lupu**[*,1,3], **Eric Hambro**[1], **Aram H. Markosyan**[1], **Manish Bhatt**[1], **Yuning Mao**[1], **Minqi Jiang**[1], **Jack Parker-Holder**[2], **Jakob Foerster**[3], **Tim Rocktäschel**[2], **Roberta Raileanu**[1,2]

[1]Meta, [2]University College London, [3]University of Oxford
[*]Equal contributions.

As large language models (LLMs) become increasingly prevalent across many real-world applications, understanding and enhancing their robustness to adversarial attacks is of paramount importance. Existing methods for identifying adversarial prompts tend to focus on specific domains, lack diversity, or require extensive human annotations. To address these limitations, we present RAINBOW TEAMING, a novel black-box approach for producing a diverse collection of adversarial prompts. RAINBOW TEAMING casts adversarial prompt generation as a quality-diversity problem, and uses open-ended search to generate prompts that are both effective and diverse. Focusing on the safety domain, we use RAINBOW TEAMING to target various state-of-the-art LLMs, including the Llama 2 and Llama 3 models. Our approach reveals hundreds of effective adversarial prompts, with an attack success rate exceeding 90% across all tested models. Furthermore, we demonstrate that prompts generated by RAINBOW TEAMING are highly transferable and that fine-tuning models with synthetic data generated by our method significantly enhances their safety without sacrificing general performance or helpfulness. We additionally explore the versatility of RAINBOW TEAMING by applying it to question answering and cybersecurity, showcasing its potential to drive robust open-ended self-improvement in a wide range of applications.

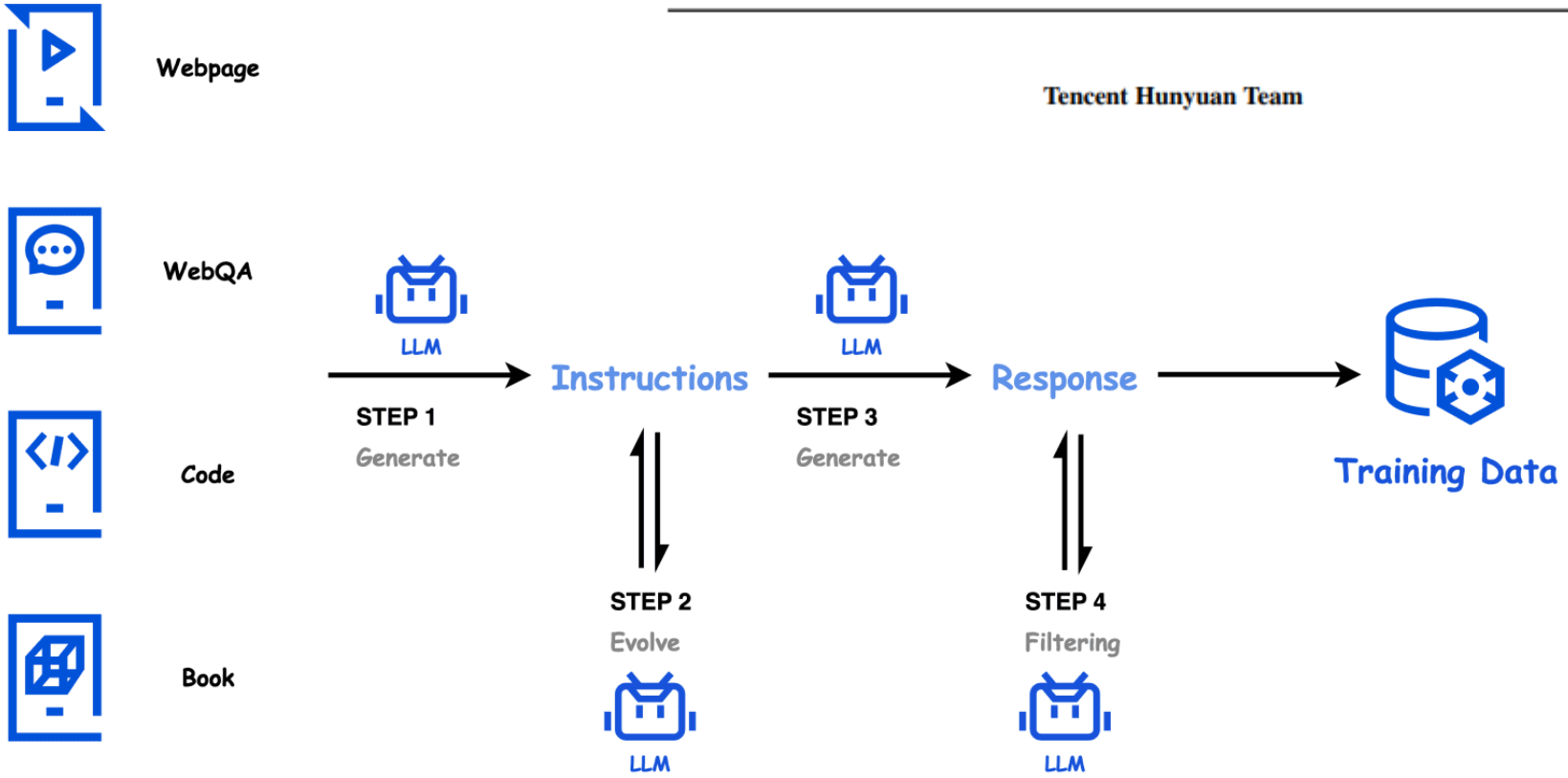
Date: December 12, 2024
Correspondence: mikayel@samvelyan.com, sharathraparthy@gmail.com, alupu@meta.com



Synthetic Data

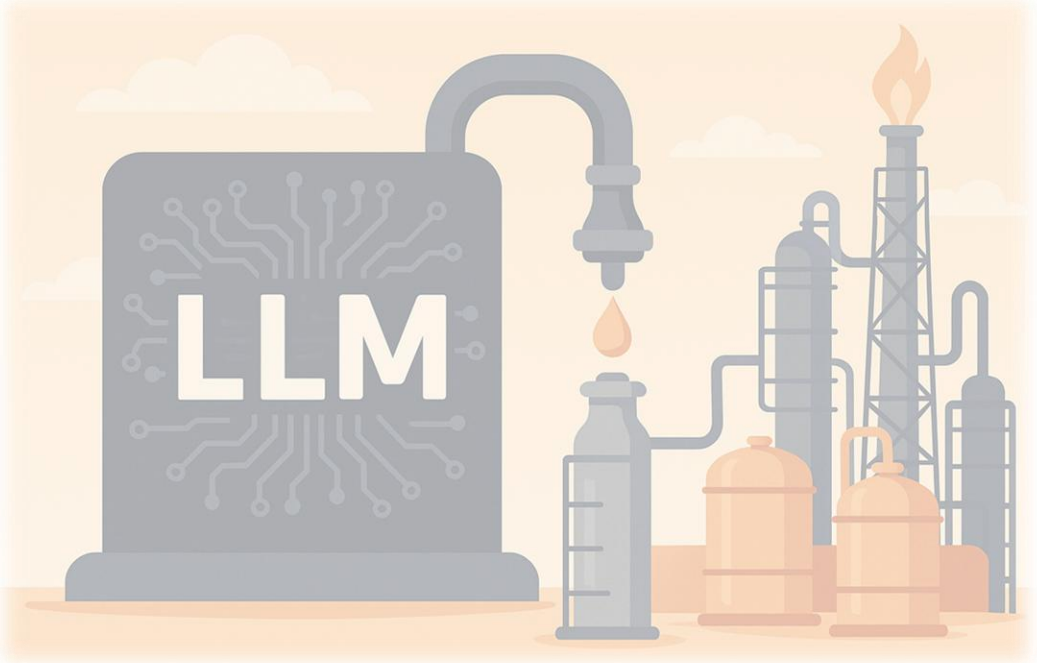
Hunyuan-Large: An Open-Source MoE Model with 52 Billion Activated Parameters by Tencent

Tencent Hunyuan Team

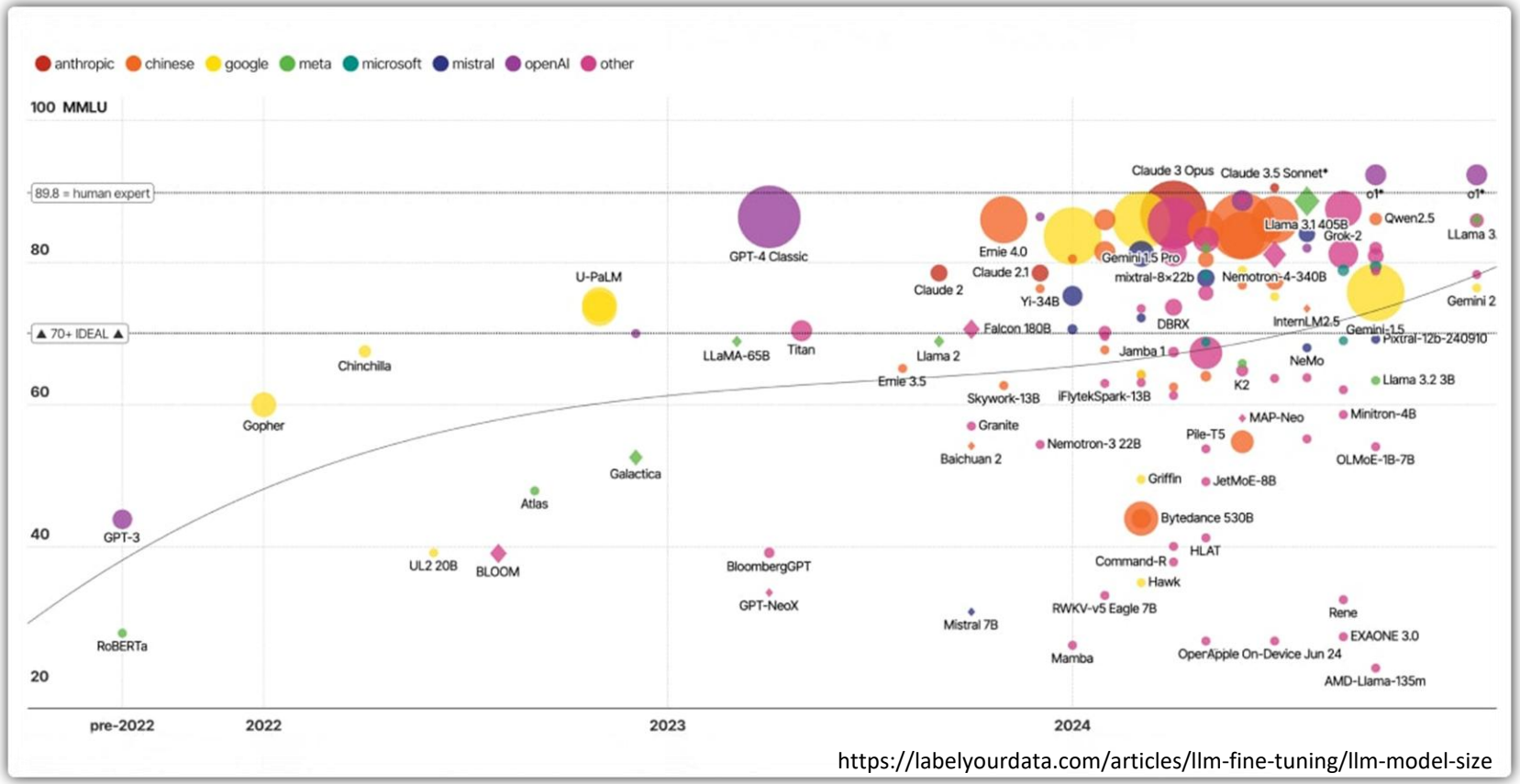


Session 3 : Breaking Scaling Law – Distillation from Large-Scale Intelligence to Lightweight Deployment

- Scaling Law
- Why distillation now
- Supervised KD (Forward KL)
- Synthetic-data distillation
- On-policy / Generalized KD
- Multimodal Distillation
- Takeaways



Arms race on **LLM Size vs Performance**



https://labelyourdata.com/articles/llm-fine-tuning/llm-model-size

Arms race on **LLM Size vs Performance**

- Increasing parameter count improves reasoning, comprehension, and generalization
- Models between 7B and 13B parameters deliver a strong balance of speed, accuracy, and cost efficiency

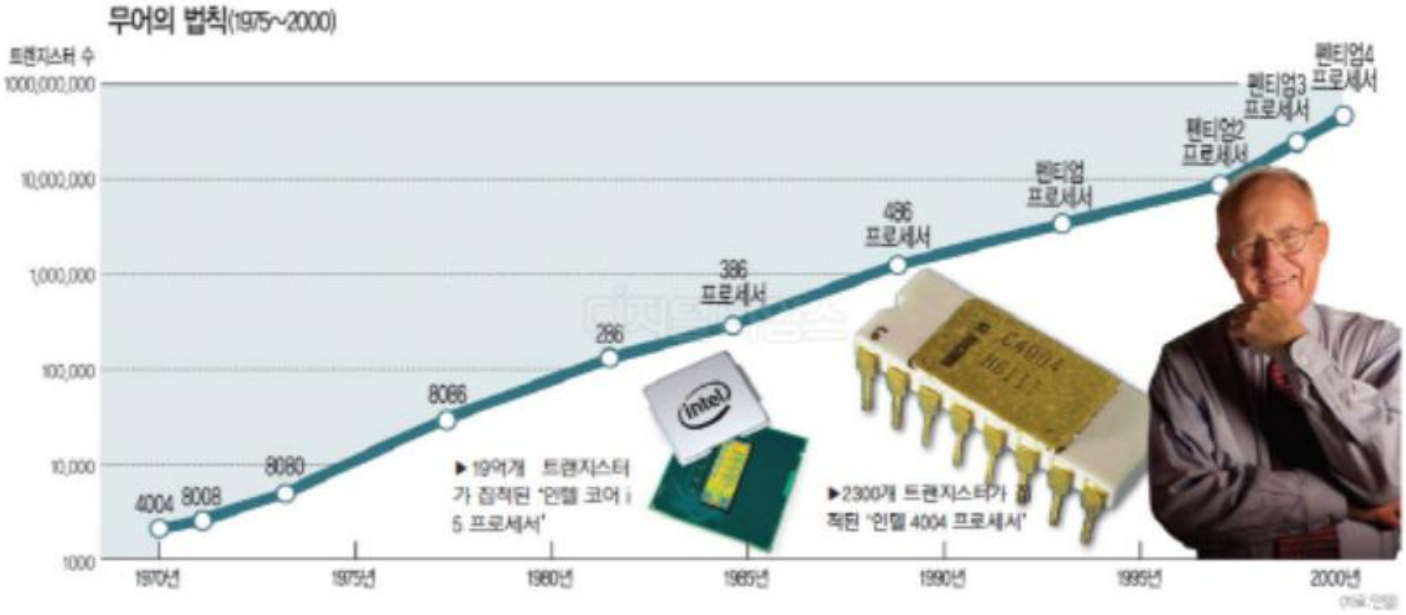
Model Size Range	Typical Tasks	Performance	Trade-Offs
1–3B	Simple NLP, embeddings, mobile inference	Fast, limited reasoning	Shallow context understanding
7–13B	General chat, summarization, QA	Strong balance	Moderate compute cost
30–70B	Advanced reasoning, multilingual, code generation	High accuracy	Requires enterprise GPUs
100B+	Multimodal, research-scale models	Peak performance	Very high cost and latency

Background of Arms race

Scaling Laws for Language model

Scaling Laws for Autoregressive Generative Modeling

🤔 Why the scaling law matters?



Background of big multimodal AI

Scaling Laws for Language model

Scaling Laws for Autoregressive Generative Modeling

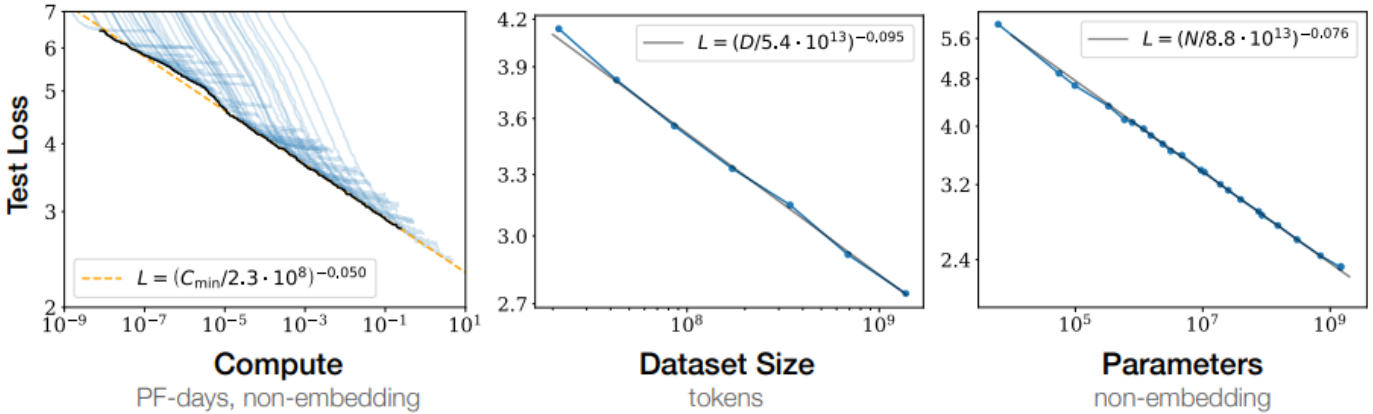
🤔 Why the scaling law matters?

- What matters, what doesn't matter for ML performance
- What problems should we work on?
- What should we expect in the future?

😮 Maybe, we can make progress with building a better engine rather than improving the main hypothesis

Scaling Laws for Language Models

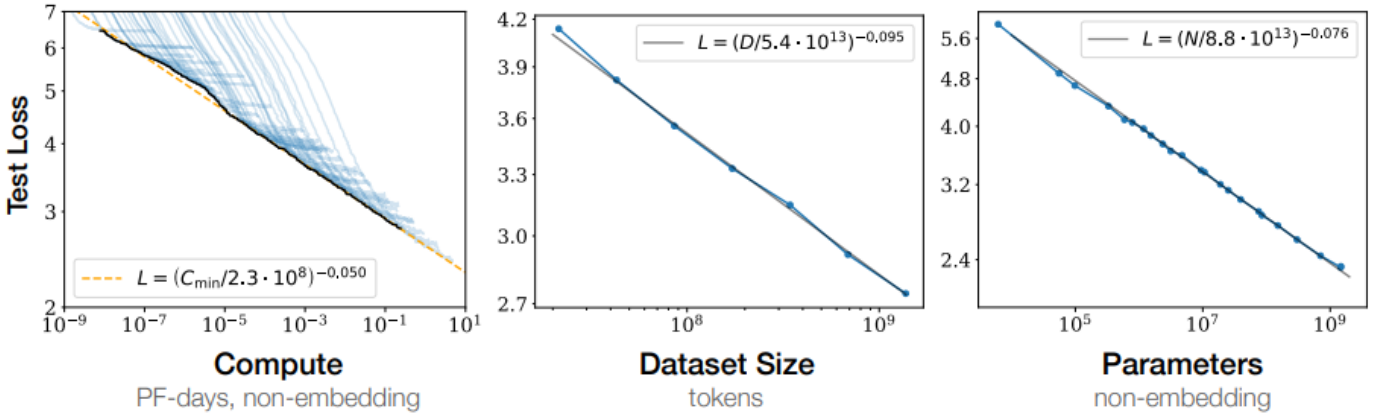
- **There are precise scaling laws for performance of ML models**
- **As a function of**
  - Parallel pathways Model parameters, *N*
  - Dataset size, *D*
  - Total compute used for training, *C*



Kaplan et al. Scaling Laws for Neural Language Models

Scaling Laws for Language Models

- Power-law relationships with each individual factor when not bottlenecked by the other two.
- In each case, all of the other quantities are much larger.
  Ex) for model size *N* scaling, we must have very large dataset *D*



Kaplan et al. Scaling Laws for Neural Language Models

Achieve scaling, avoid bottleneck

- Performance mostly about avoiding bottlenecks
  - Not enough data
  - Not enough parameter
  - Not enough compute
  - Bad information propagation through the network design (Resnet, Transformer, Batchnorm solve this bottleneck)

- If we already have a good scalable architecture (i.e. transformer) and optimization method, many other details don't matter much.

  😐 just change of constant prefactors

New Motivation – Other modalities

Previously,
We observed scale-law relationships from LMs [1]

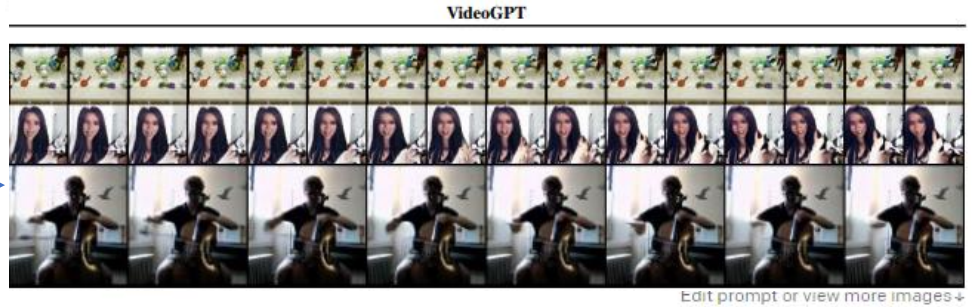
- **Do they apply to all data modalities?**
  - Image, Video, Math …

- **How do improvements on the loss translate to**
  - Improvements in representation quality
  - Performance on downstream tasks?

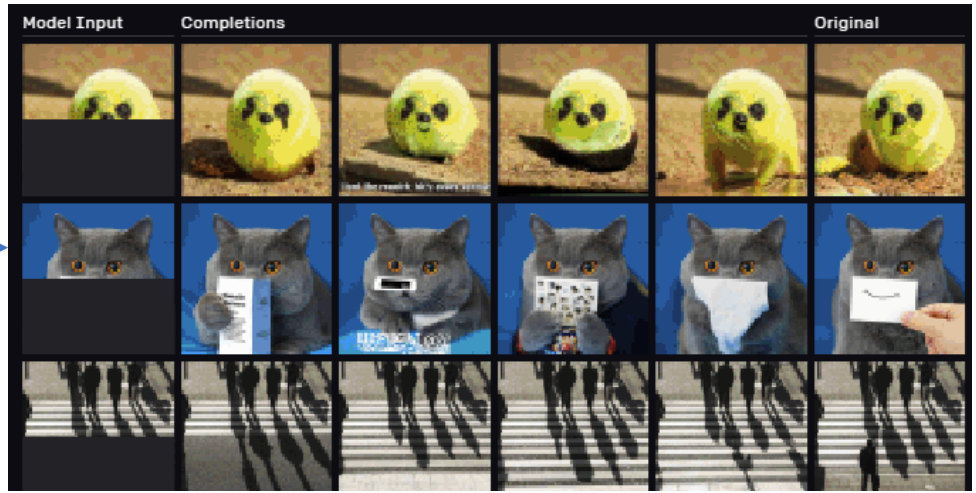
🤔 **From there, What else can we learn from them?**

[1] Kaplan et al. Scaling Laws for Neural Language Models

Multimodal



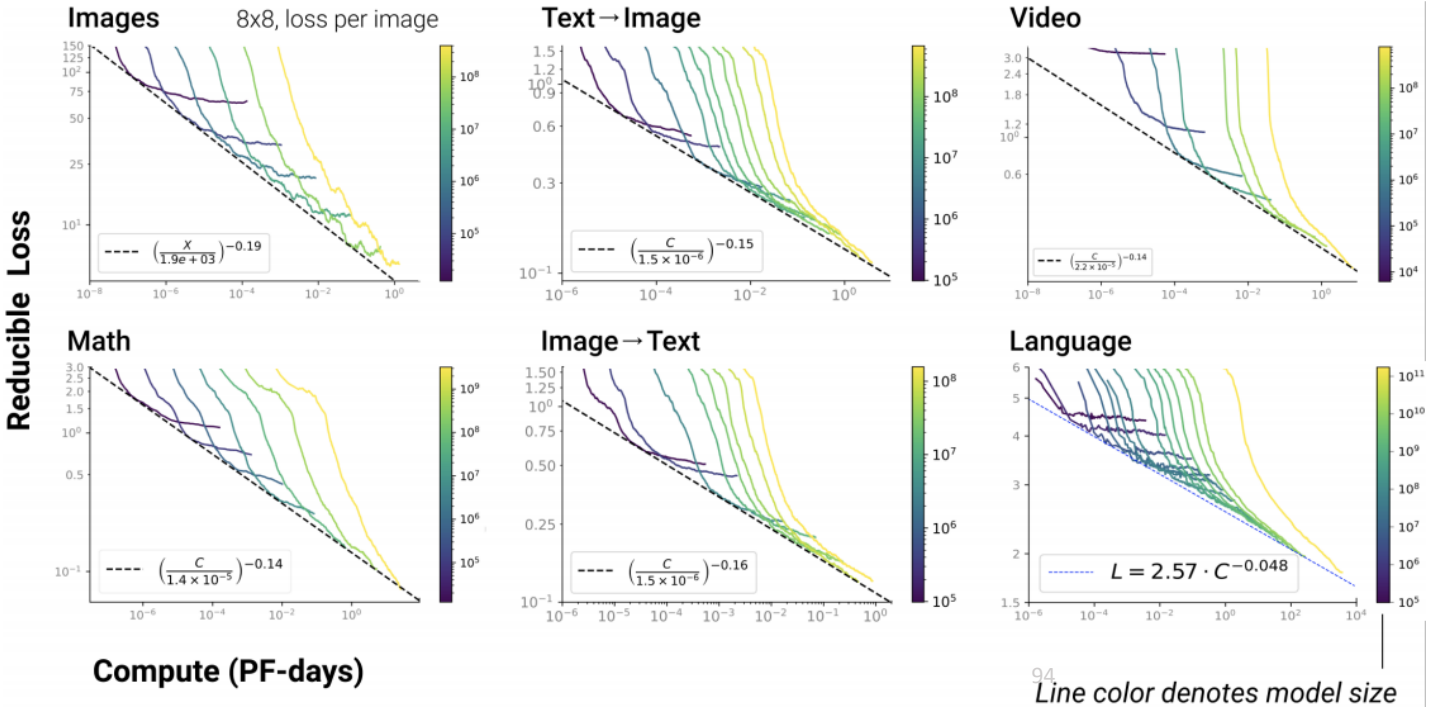
Transformer



Transformer
applies to all data modalities

Smooth scaling of reducible loss across domains

- Transformer with an autoregressive cross-entropy loss
- Scaling law apply to generative modeling across a wide variety of data modalities
  - Irreducible loss L ∞ is a fitted domain-dependent const



Information theoretic interpretation

- model sizes N, compute budgets C, or dataset sizes D,
  The scaling relation for the loss
  $L(x)=L_\infty+(x_0/x)\^(\alpha_x\ \ )\ \approx$S(True)+ D_KL (True||Model)

- $x=N,\ C,\ D$

- α_x is a modality-dependent exponent

- L_∞ is irreducible loss,
  estimates the entropy of the true data distribution

- $\left(\frac{\mathbf{x_0}}{\mathbf{x}}\right)^{\boldsymbol{\alpha_x}}$ Reducible loss estimate of the KL divergence between the true and model distributions

Information theoretic interpretation

- The scaling relation for the loss

$$L(x) = L_\infty + \left(\frac{x_0}{x}\right)^{\alpha_x} \approx S(\text{True}) + D_{KL}(\text{True}||\text{Model})$$

$$\text{“Irreducible Loss”} \qquad L_\infty \approx S(\text{True})$$

$$\text{“Reducible Loss”} \qquad \left(\frac{x_0}{x}\right)^{\alpha_x} \approx D_{KL}(\text{True}||\text{Model})$$

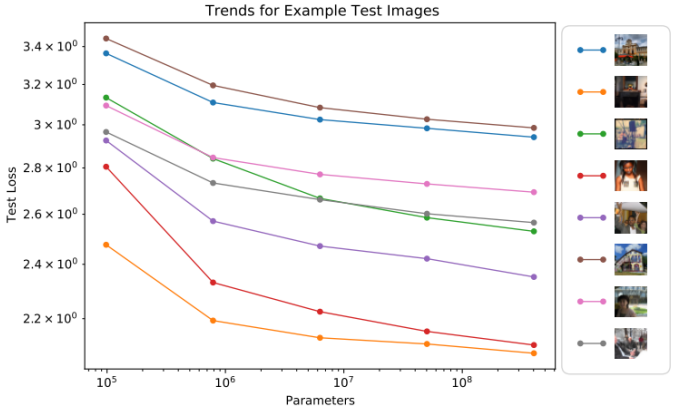


Figure 17 Loss trend for individual images— We show the loss trend for eight randomly chosen images from the test set. These results are fairly typical.

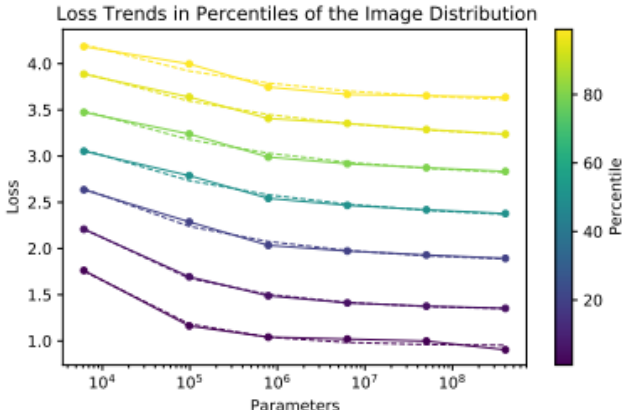


Figure 10 Performance trends for image dataset percentiles-

Then what is the optimal model size?

- The optimal model size for a given compute budget.
- "Opt Model Size vs Compute" relation is very nearly a pure power-law

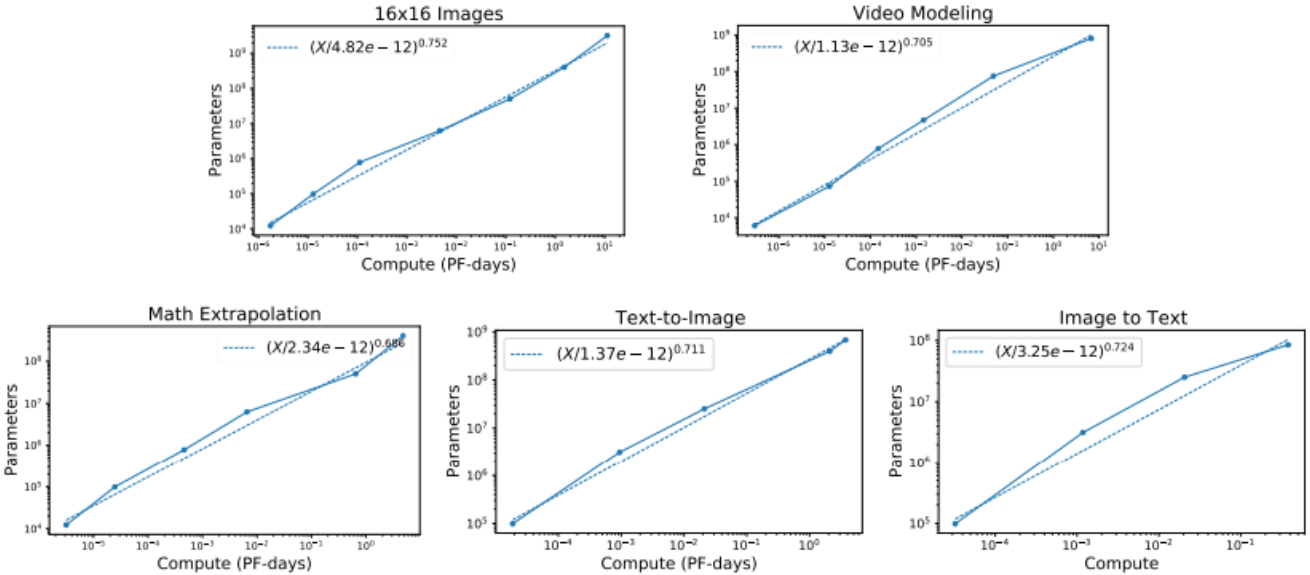


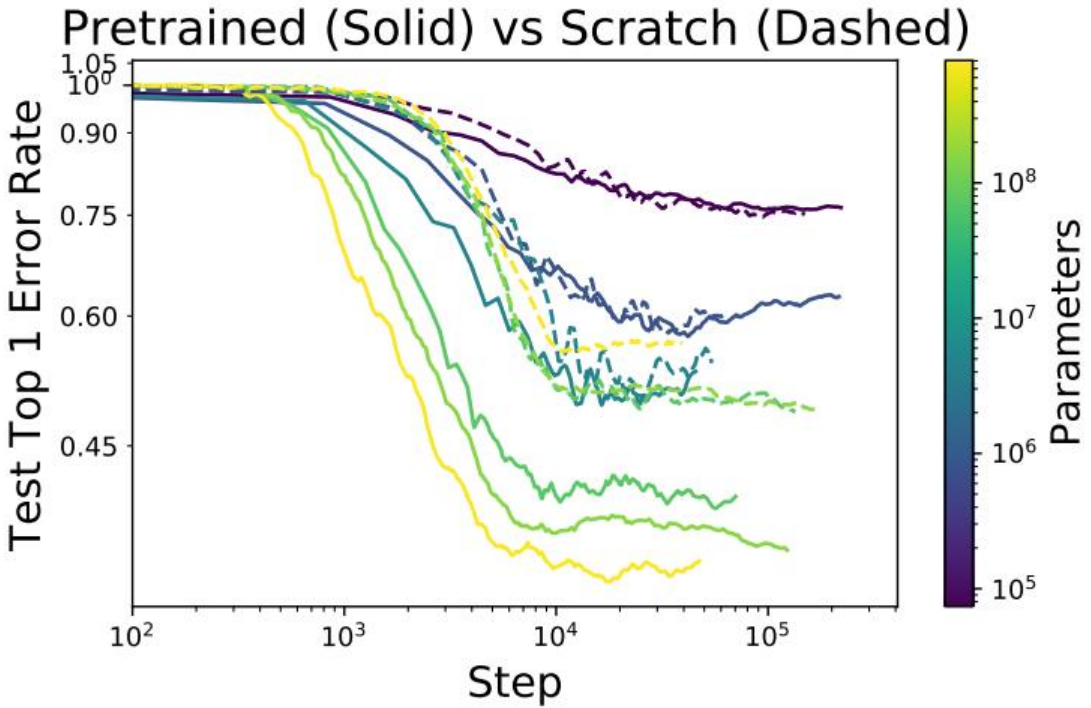
Figure 16 Optimal model size (individual trends)— We show the optimal model size for a given compute budget, along with power-law fits, based on the points at the compute-efficient frontier of figure 5. These trends are combined in figure 2.

Information theoretic interpretation

Domain	$L(N)$ (model size)	$L(C)$ (compute)	$N_{\text{opt}}(C)$
Language	$\left(\frac{N}{1.47\times10^{14}}\right)^{-0.070}$	$\left(\frac{C}{3.47\times10^{8}}\right)^{-0.048}$	$\left(\frac{C}{3.3\times10^{-13}}\right)^{0.73}$
Image 8x8	$3.12+\left(\frac{N}{8.0\times10^{1}}\right)^{-0.24}$	$3.13+\left(\frac{C}{1.8\times10^{-8}}\right)^{-0.19}$	$\left(\frac{C}{5.3\times10^{-14}}\right)^{0.64}$
Image 16x16	$2.64+\left(\frac{N}{2.8\times10^{2}}\right)^{-0.22}$	$2.64+\left(\frac{C}{1.6\times10^{-8}}\right)^{-0.16}$	$\left(\frac{C}{4.8\times10^{-12}}\right)^{0.75}$
Image 32x32	$2.20+\left(\frac{N}{6.3\times10^{1}}\right)^{-0.13}$	$2.21+\left(\frac{C}{3.6\times10^{-9}}\right)^{-0.1}$	$\left(\frac{C}{1.6\times10^{-13}}\right)^{0.65}$
Image VQ 16x16	$3.99+\left(\frac{N}{2.7\times10^{4}}\right)^{-0.13}$	$4.09+\left(\frac{C}{6.1\times10^{-7}}\right)^{-0.11}$	$\left(\frac{C}{6.2\times10^{-14}}\right)^{0.64}$
Image VQ 32x32	$3.07+\left(\frac{N}{1.9\times10^{4}}\right)^{-0.14}$	$3.17+\left(\frac{C}{2.6\times10^{-6}}\right)^{-0.12}$	$\left(\frac{C}{9.4\times10^{-13}}\right)^{0.7}$
Text-to-Im (Text)	$\left(\frac{N}{5.6\times10^{8}}\right)^{-0.037}$	(combined text/image loss)	
Text-to-Im (Image)	$2.0+\left(\frac{N}{5.1\times10^{3}}\right)^{-0.16}$	$1.93+\left(\frac{C}{1.5\times10^{-6}}\right)^{-0.15}$	$\left(\frac{C}{9.4\times10^{-13}}\right)^{0.7}$
Im-to-Text (Text)	$\left(\frac{N}{7.0\times10^{8}}\right)^{-0.039}$	(combined text/image loss)	
Im-to-Text (Image)	$2.0+\left(\frac{N}{5.5\times10^{3}}\right)^{-0.15}$	$1.97+\left(\frac{C}{1.5\times10^{-6}}\right)^{-0.16}$	$\left(\frac{C}{3.3\times10^{-12}}\right)^{0.72}$
Video VQ 16x16x16	$1.01+\left(\frac{N}{3.7\times10^{4}}\right)^{-0.24}$	$0.95+\left(\frac{C}{2.2\times10^{-5}}\right)^{-0.14}$	$\left(\frac{C}{1.13\times10^{-12}}\right)^{0.71}$
Math (Extrapolate)	$0.28+\left(\frac{N}{1.1\times10^{4}}\right)^{-0.16}$	$0.14+\left(\frac{C}{1.4\times10^{-5}}\right)^{-0.17}$	$\left(\frac{C}{2.3\times10^{-12}}\right)^{0.69}$

Finetuning Image GMs to ImageNet Classification

- The smooth trends for finetuned performance on image classification
  - downstream performance also improves with model size and compute



Finetuning Image GMs to ImageNet Classification

- Model parameters, N ↑
- Dataset size, D ↑
- Total compute for training, C ↑

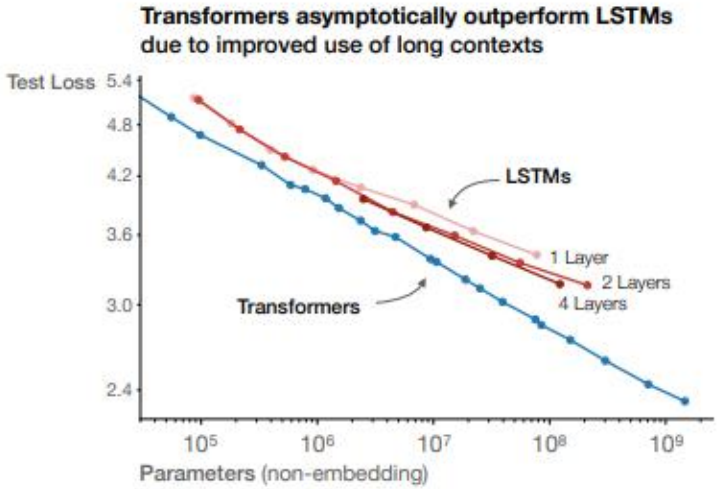
체급이 깡패다

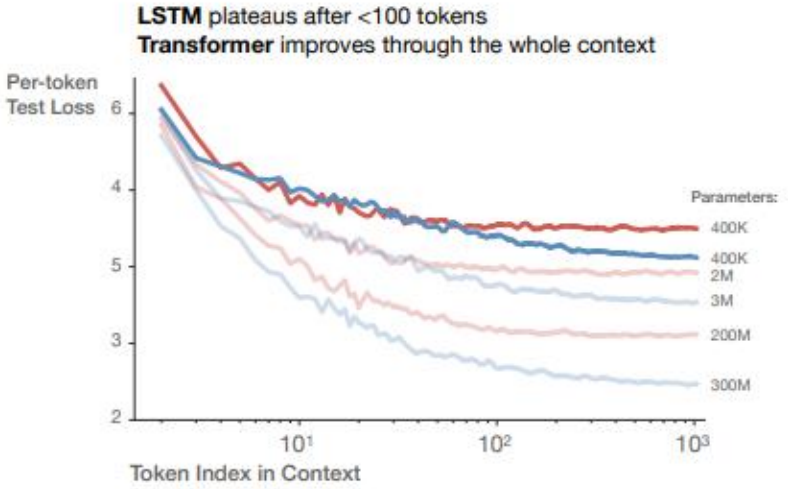
Upcoming trends : Architecture is less important

- Except when the architecture itself creates bottleneck.

Past : Solve a bad information propagation problem, (i.e. architecture, solver)

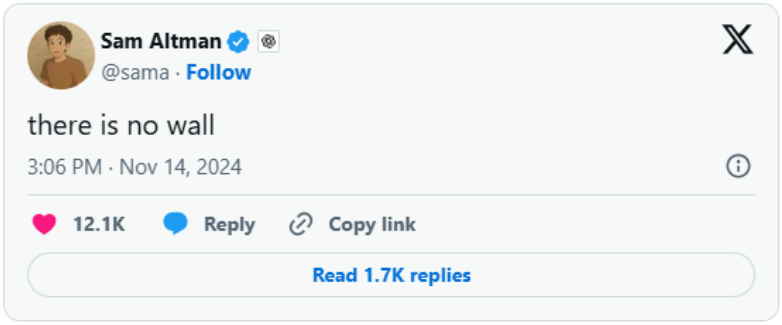
Next : Now let's scale up





Kaplan et al. Scaling Laws for Neural Language Models

Have we hit a wall?



Language training dataset size — Likely

3.7 x/year

The training dataset size for language models has grown by 3.7x per year since 2010.

90% confidence interval: 3.2x to 4.3x.

When will the largest training runs use all public human-generated text? — Plausible

2028

The median projected year in which most of the effective stock of publicly available human-generated text will be used in a training run is 2028.

90% confidence interval: 2026 to 2033.

Largest training dataset used to train an LLM — Uncertain

> 30 trillion tokens

Llama 4 models were trained using a collection of over 30 trillion tokens from text, image, and video datasets. This makes them the AI models with the largest publicly confirmed datasets.

Stock of data on the internet — Plausible

510 trillion tokens

The amount of tokens in the indexed web, the portion of the web that is publicly accessible from search engines, is estimated at 510 trillion tokens.

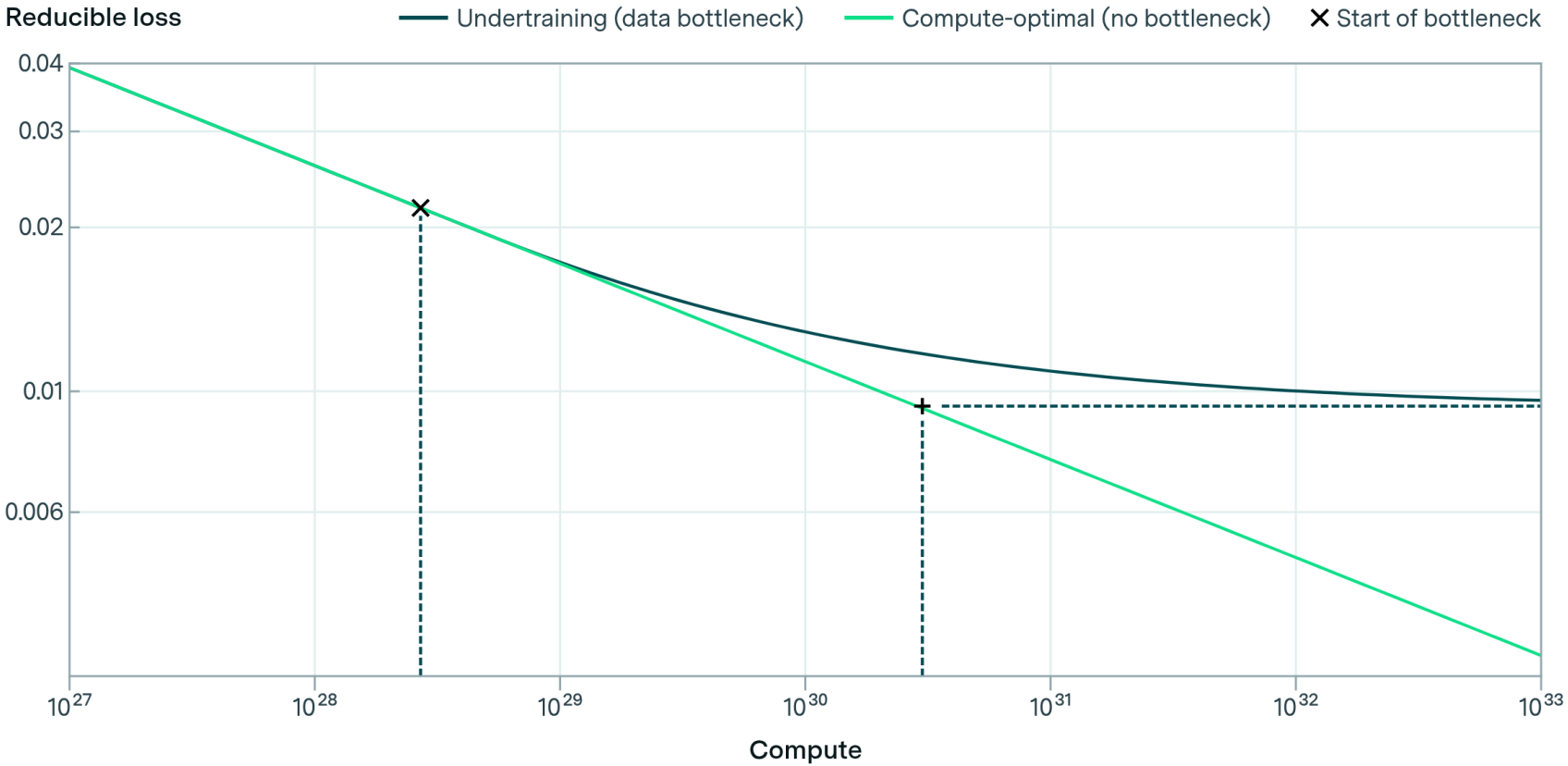
95% confidence interval: 130 trillion tokens to 2100 trillion tokens.


Pre-training as we know it will end
Compute is growing:
- Better hardware
- Better algorithms
- Larger clusters
Data is not growing:
- We have but one internet
- The fossil fuel of AI
Ilya Sutskever at NeurIPS 2024
Internet. We have, but one Internet. You could even say you can even go as far as to say.
That data is the fossil fuel of AI. It was like, created somehow. And now we use it.

Projected improvements from scaling under data scarcity

Will we run out of data? Limits of LLM scaling based on human-generated data

Pablo Villalobos [1] Anson Ho [1] Jaime Sevilla [1 2] Tamay Besiroglu [1 3] Lennart Heim [1 4] Marius Hobbhahn [1 5]



epoch.ai

Clean Data : The fossil fuel of AI?

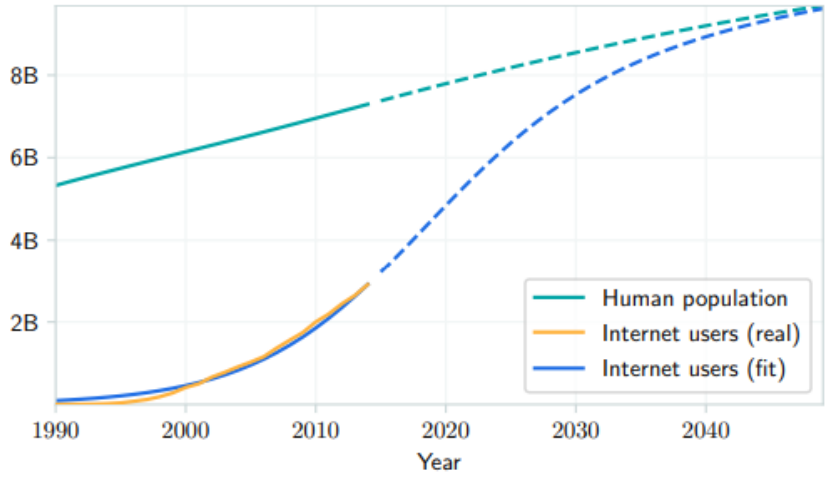


Figure 2. Historical and projected evolution of internet users. Historical data is from Ritchie & Roser (2020).

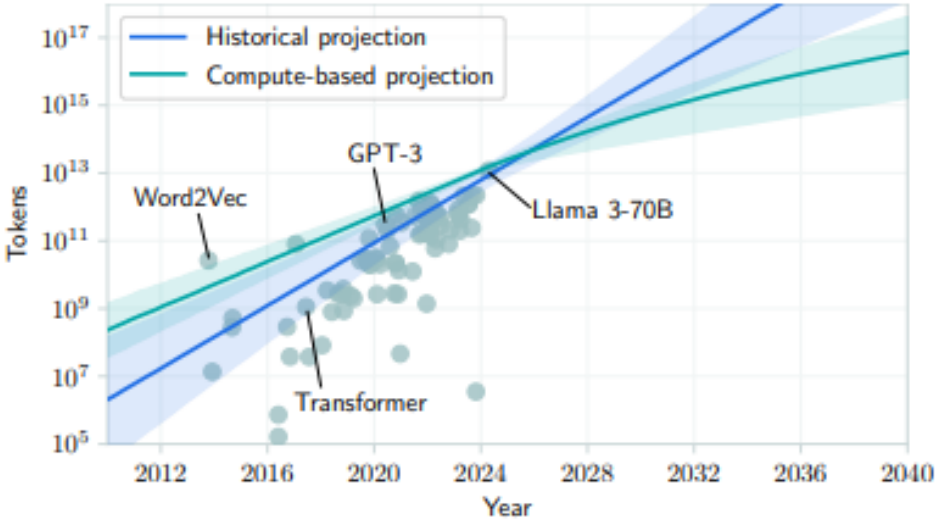
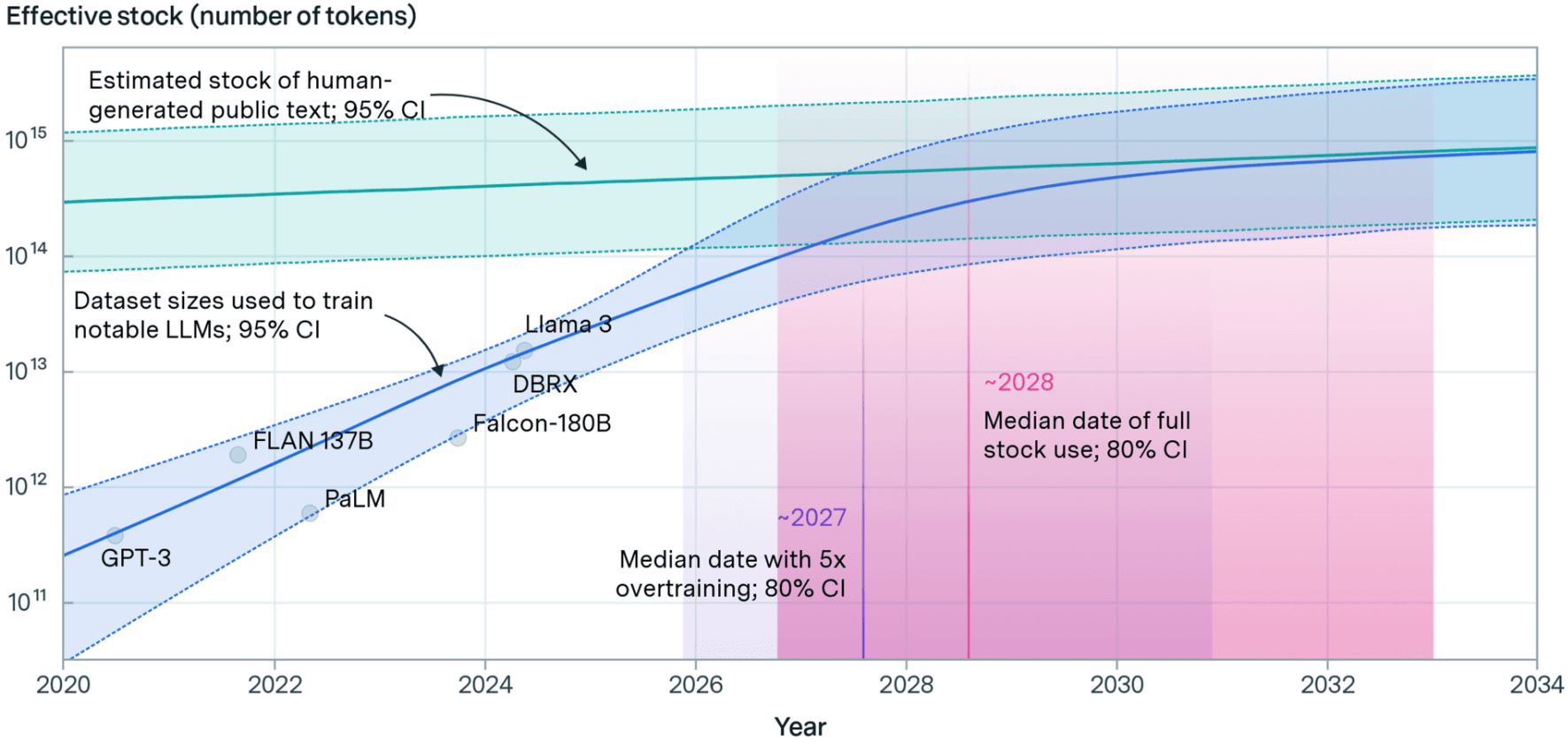


Figure 4. Projections of data usage. Two extrapolations of data usage, one from past trends and one from compute availability estimations plus scaling laws. The shaded areas denote a 90% CI for the extrapolated median. The dots are individual training runs.

Projections of the stock of public text and data usage

EPOCH AI



https://epoch.ai/trends#data

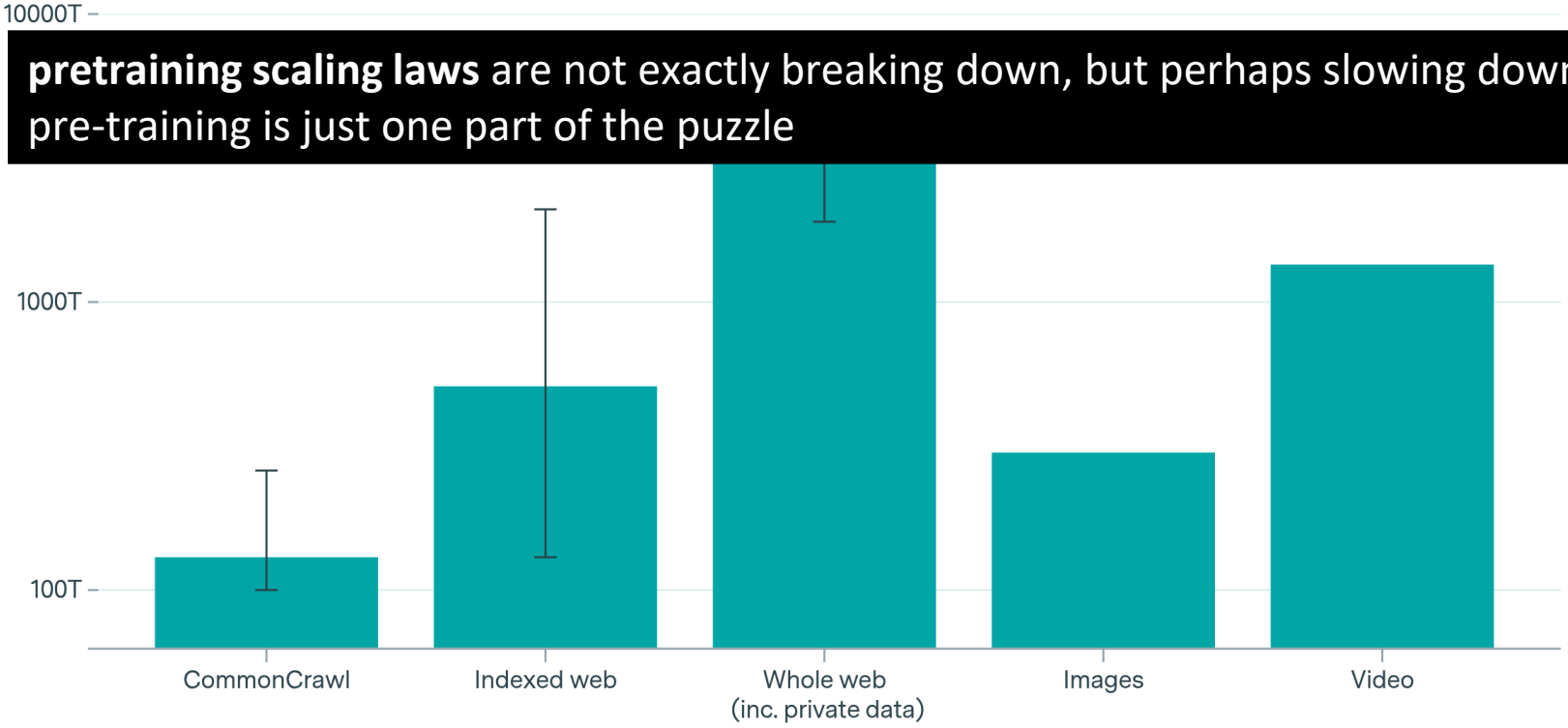
Estimates of different stocks of data

Will we run out of data? Limits of LLM scaling based on human-generated data

Pablo Villalobos [1] Anson Ho [1] Jaime Sevilla [1 2] Tamay Besiroglu [1 3] Lennart Heim [1 4] Marius Hobbhahn [1 5]

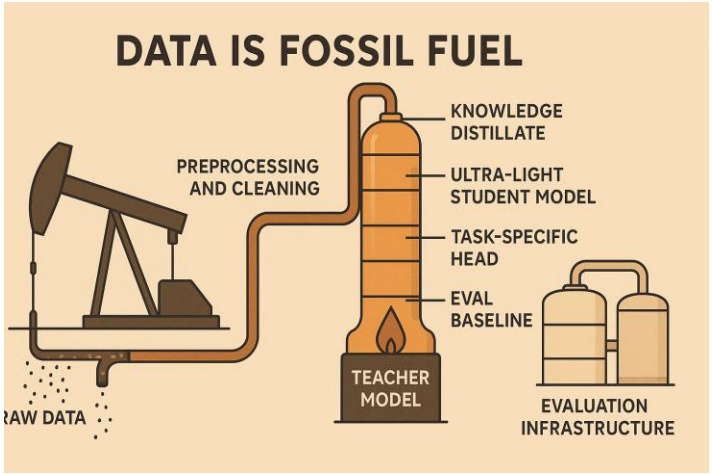
pretraining scaling laws are not exactly breaking down, but perhaps slowing down

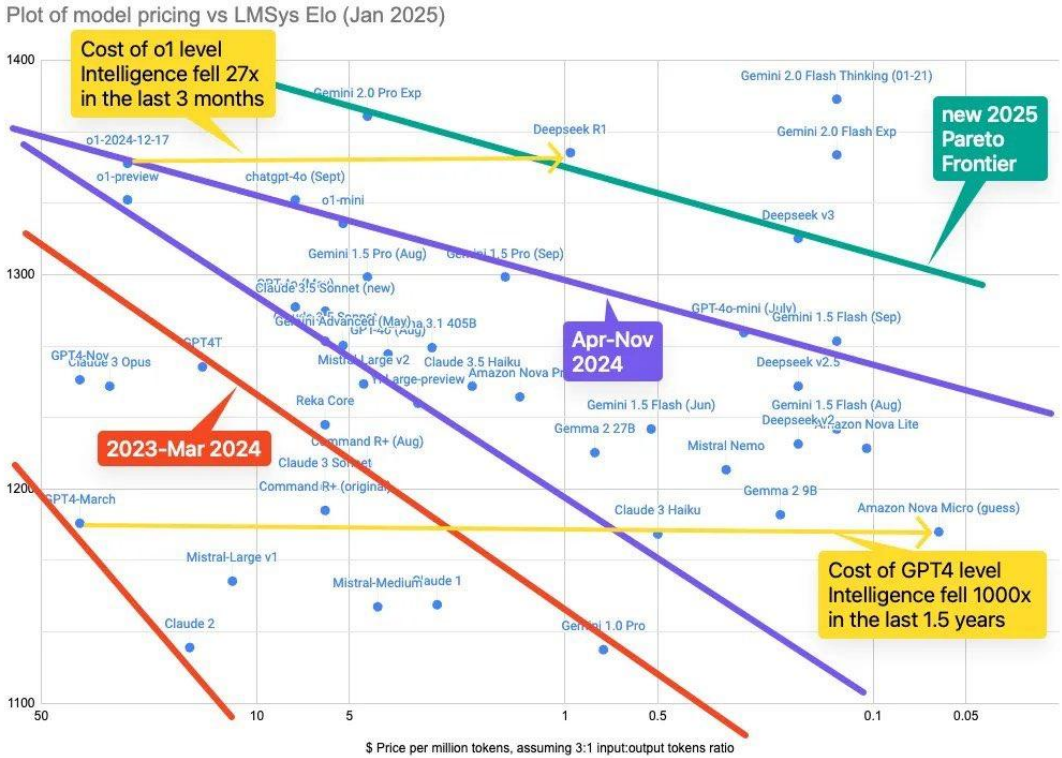
pre-training is just one part of the puzzle



epoch.ai

Can we bend the Scaling Law?

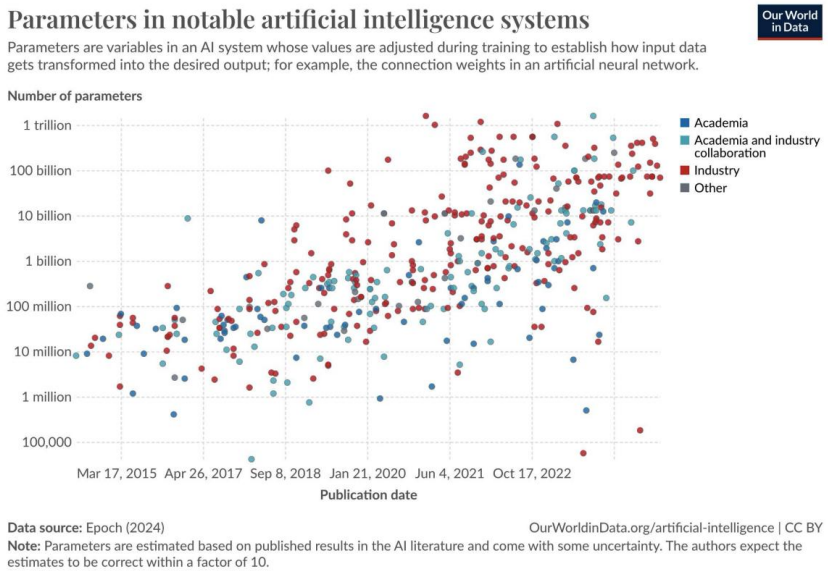


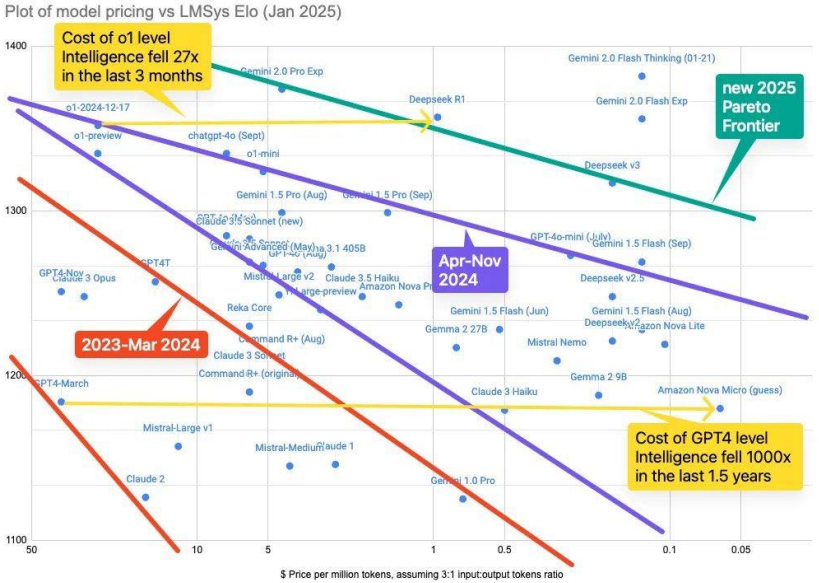


https://x.com/swyx/status/1830866865884991999/photo/1

The Crack in 'Bigger is Always Better'

- Cost–performance frontier shows smaller/cheaper models catching up.
- Distillation helps shift the frontier by transferring knowledge to compact students.



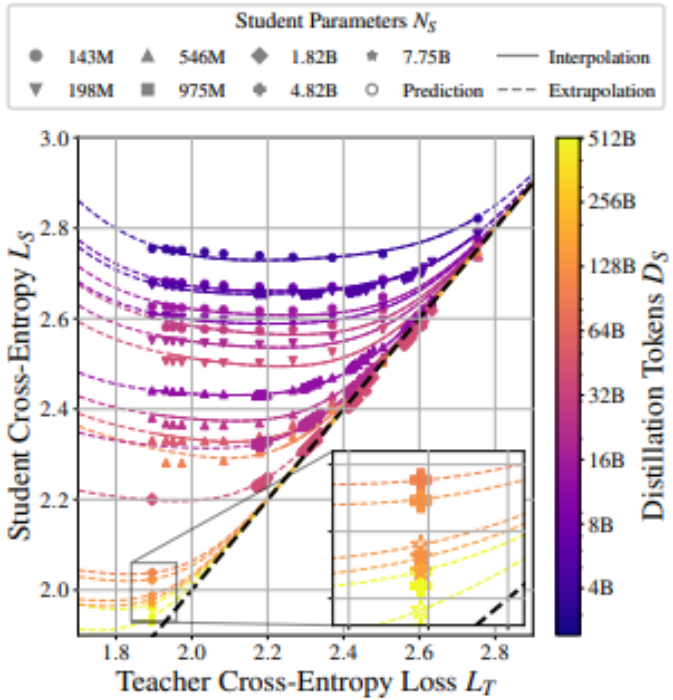


https://x.com/swyx/status/1830866865884991999/photo/1

Distillation Scaling Laws

- Predict how well a distilled student LM will perform, given a fixed compute budget.
- A **"distillation scaling law"** that predicts student cross-entropy from student size, distillation tokens, and teacher quality.

- I'll revisit this paper later!



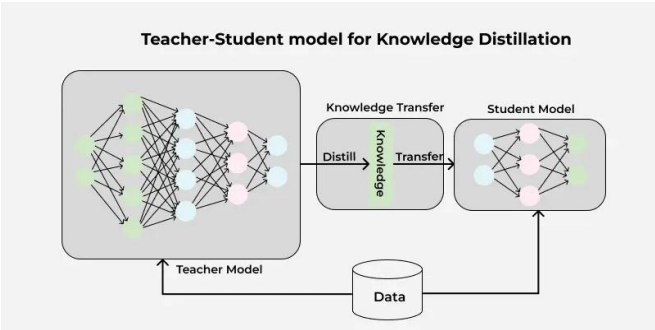
Distillation Scaling Laws

Dan Busbridge [1] Amitis Shidani [2] Floris Weers [1] Jason Ramapuram [1] Etai Littwin [1] Russ Webb [1]

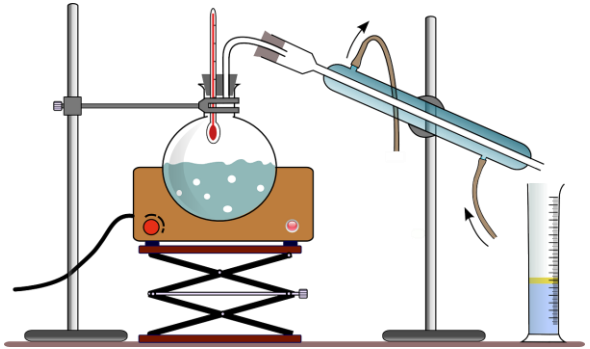
Busbridge et al. Distillation Scaling Laws

Why **Distillation** Now

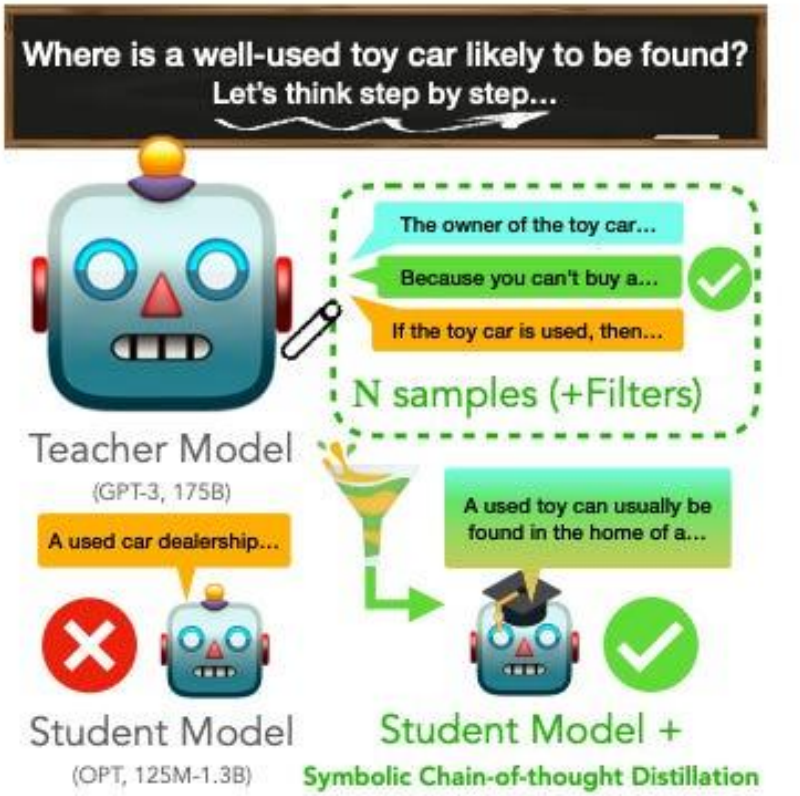
- Smaller 'mini' models are often deployed for speed and cost with modest quality gaps.
- Knowledge Distillation (KD) enables students to approach teacher quality.
- Our goal: VRAM down, speed up, minimal quality drop.



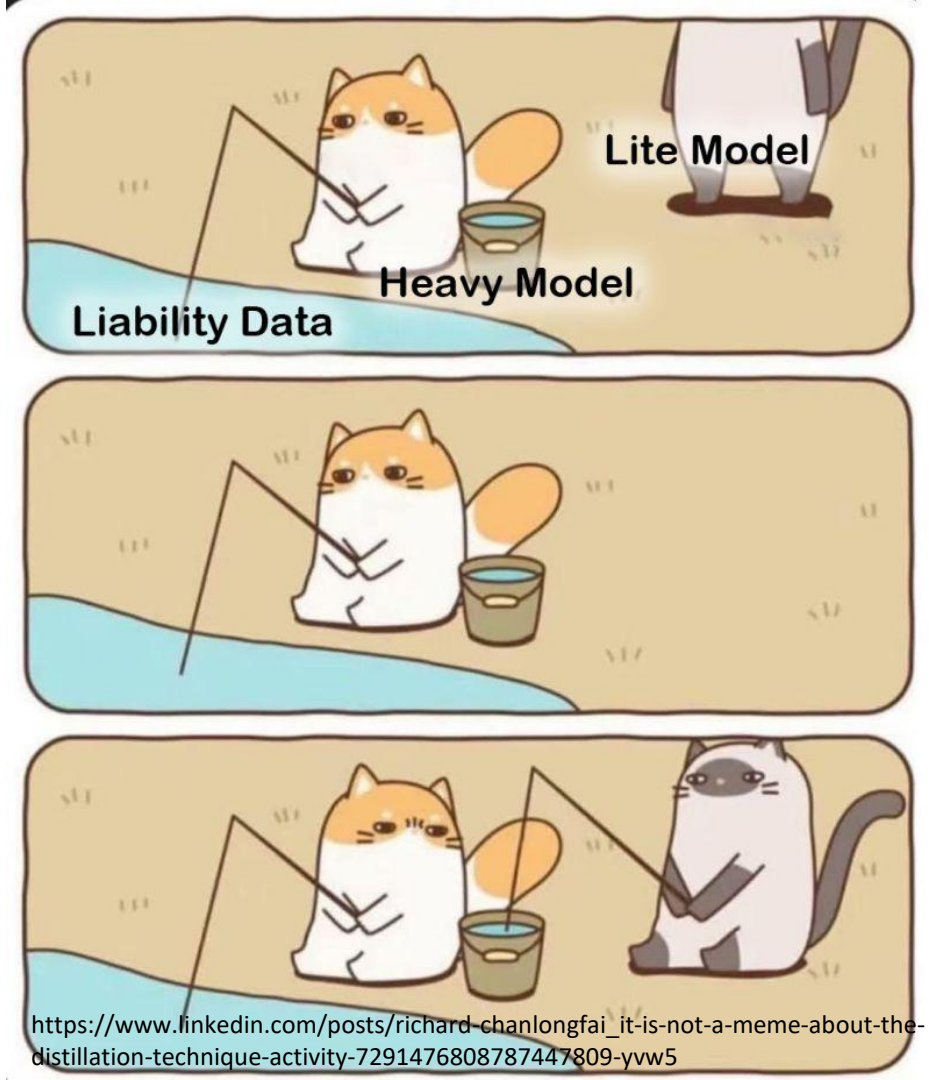
https://www.geeksforgeeks.org/machine-learning/knowledge-distillation/



Distillation in a nutshell



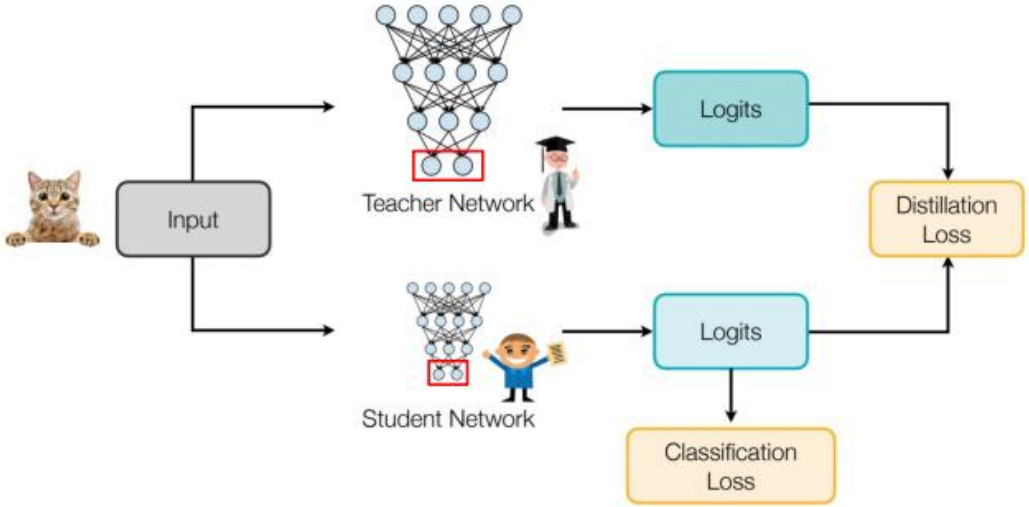
Li et al. Symbolic Chain-of-Thought Distillation: Small Models Can Also "Think" Step-by-Step



https://www.linkedin.com/posts/richard-chanlongfai_it-is-not-a-meme-about-the-distillation-technique-activity-7291476808787447809-yvw5

Knowledge Distillation

- Transfer the probability distribution (knowledge) from Teacher to Student.
- Match the next-token distribution (soft labels), typically via Forward KL.
- Soft targets convey confidence structure beyond hard labels.



Methods at a Glance

Distilling the Knowledge in a Neural Network

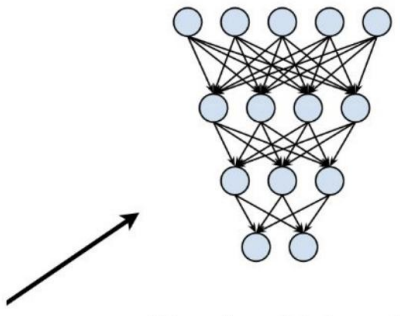
Geoffrey Hinton[*†]
Google Inc.
Mountain View
geoffhinton@google.com

Oriol Vinyals[†]
Google Inc.
Mountain View
vinyals@google.com

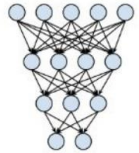
Jeff Dean
Google Inc.
Mountain View
jeff@google.com

- A. Supervised KD (Forward KL)
- B. Synthetic-Data Distillation (Sequence-level SFT on teacher outputs)
- C. On-Policy / Generalized KD (Reverse KL; student generates, teacher scores)

Supervised Distillation



Teacher Network



Student Network

	Logits	Probabilities
Cat	5	0.982
Dog	1	0.017

$$\frac{\exp(5)}{\exp(5)+\exp(1)}$$

$$\frac{\exp(1)}{\exp(5)+\exp(1)}$$

	Logits	Probabilities
Cat	3	0.731
Dog	2	0.269

The student model is less confident

Supervised Distillation for Language Models

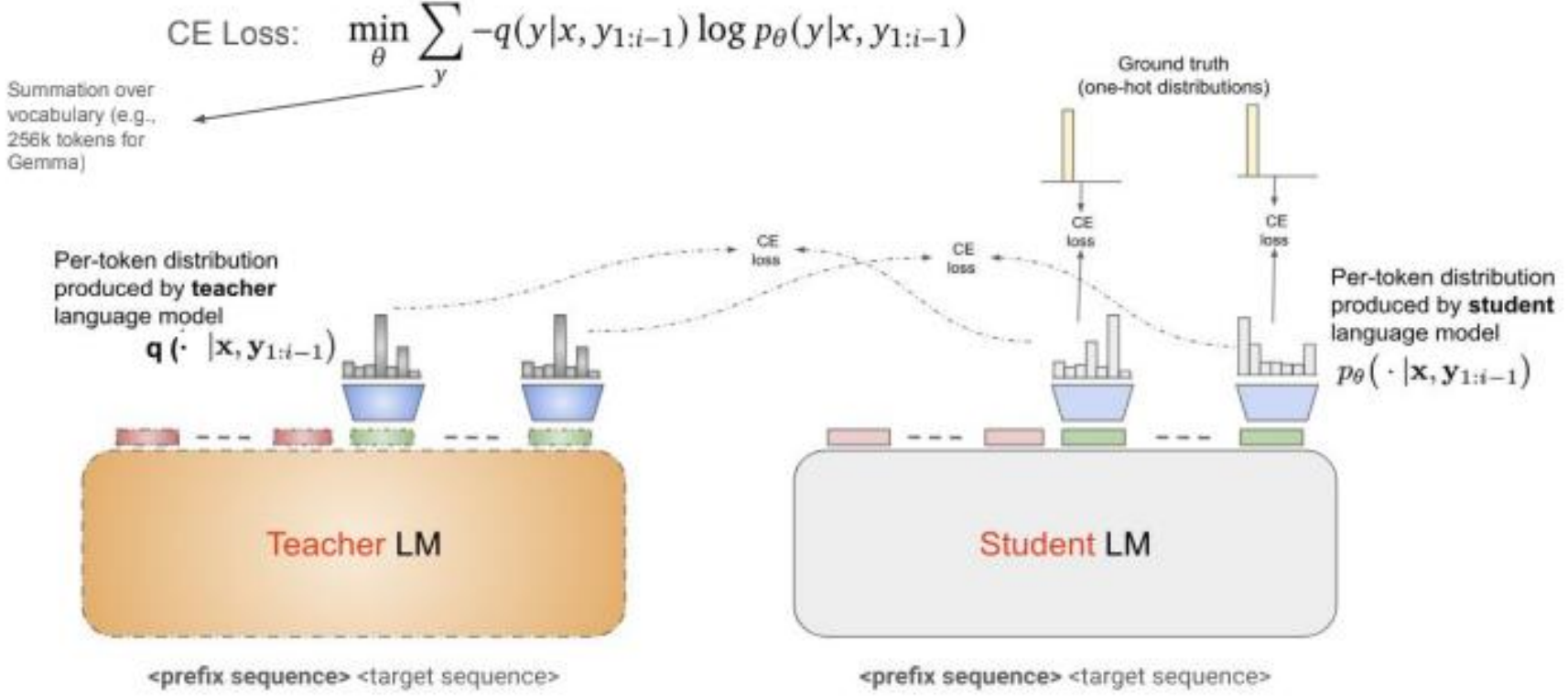


Figure Credit: Yongchao Zhou

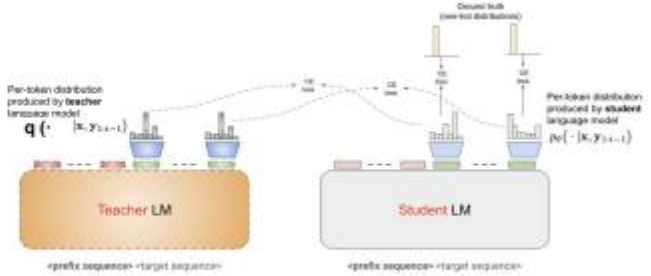
(A) Supervised KD: Mechanics & Trade-offs

- Mechanics: minimize KL(P_teacher ‖ P_student) over logits with temperature.
  - Pros: fine-grained token-level alignment; strong when tokenizers match.
  - Cons: logit tensors are large; long sequences can be memory-heavy; usually requires same tokenizer.

$$\min_\theta KL(\overbrace{q(x)}^{\text{Teacher}} \,||\, \overbrace{p_\theta(x)}^{\textit{Student}})$$

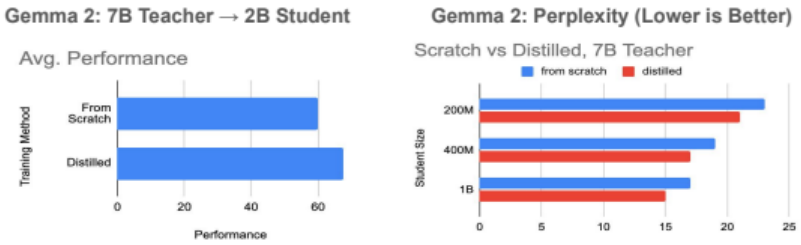
$$= \min_\theta \mathbb{E}_{y\sim q(x)}\left[\sum_i \underbrace{KL(q(\cdot|x, y_{1:i-1})||p_\theta(\cdot|x, y_{1:i-1}))}_{\text{Per-Token KL}}\right]$$

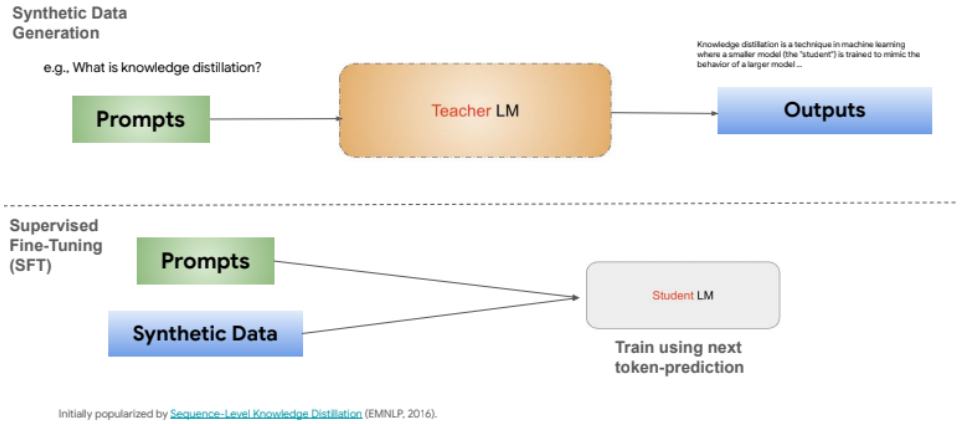
$$= \min_\theta \mathbb{E}_{y\sim q(x)}\left[\underbrace{\sum_i \sum_{y_i\in\mathcal{V}} \underbrace{-q(y_i|x, y_{1:i-1})}_{\text{Soft Labels}} \log p_\theta(y_i|x, y_{1:i-1})}_{\text{Next token Prediction Loss}}\right]$$



(B) Synthetic-Data KD: Practical Workhorse

- Pipeline: prompts >> Teacher answers >> SFT Student on (prompt, answer) pairs.
- Only needs API access; tokenizers may differ; avoids logit storage.
- Case: Gemma 2B distilled from 7B beats from-scratch and lowers perplexity.

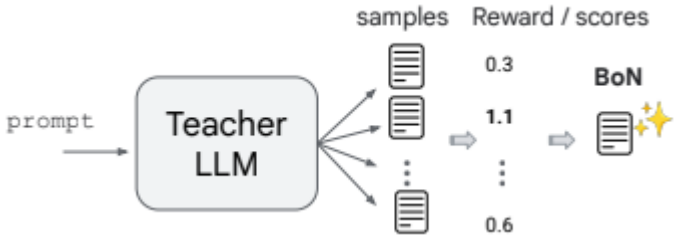




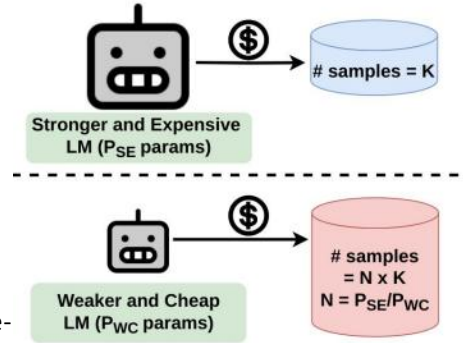
Initially popularized by Sequence-Level Knowledge Distillation (EMNLP, 2016).

Make Synthetic Data Better (BoN & Cost-Matching)

- Best-of-N (BoN): aka Rejection Sampling. sample N teacher answers; keep only the best to train the student.
    1. Sample N model generations,
    2. Score them,
    3. Keep the one with highest score



- Compute/Cost-matched: use a cheaper teacher but increase N; competitive at lower cost.
- Higher-quality teacher outputs leads better students.

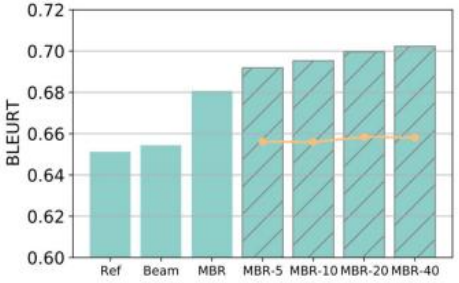


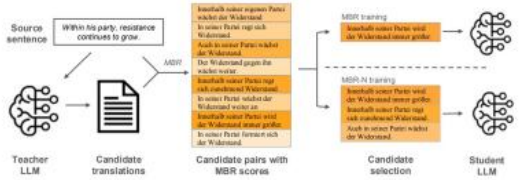
Wang et al. Don't Throw Away Data: Better Sequence Knowledge Distillation. ICLR 2025

Bansal et al. Smaller, Weaker, Yet Better: Training LLM Reasoners via Compute-Optimal Sampling. ICLR 2025.

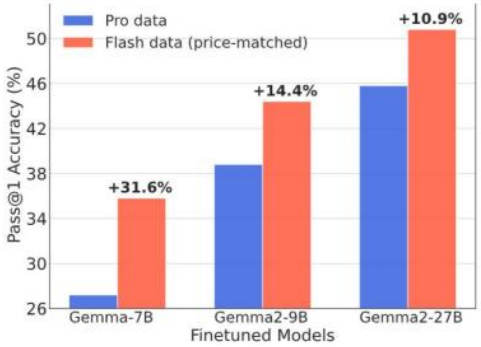
Make Synthetic Data Better (BoN & Cost-Matching)

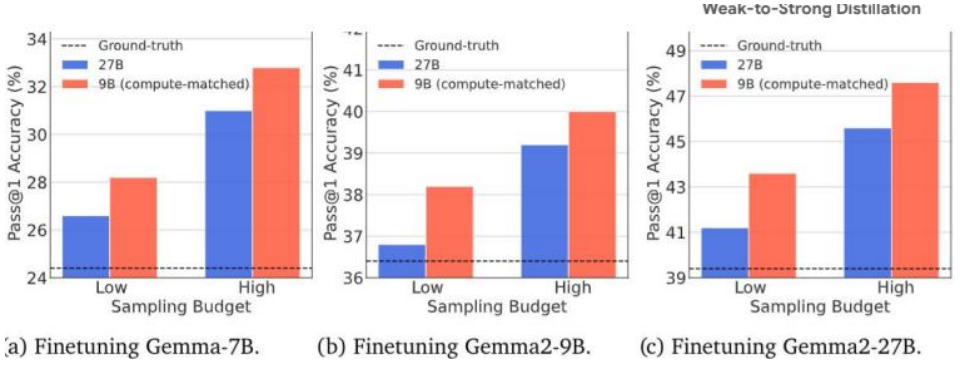
- Best-of-N (BoN): aka Rejection Sampling.





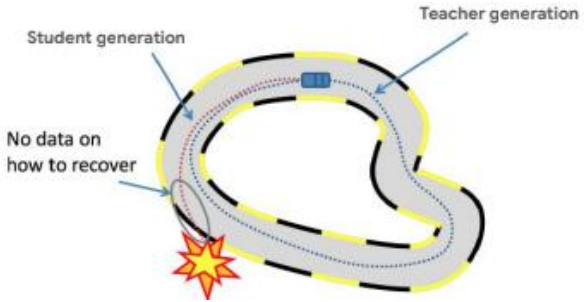
- Compute/Cost-matched

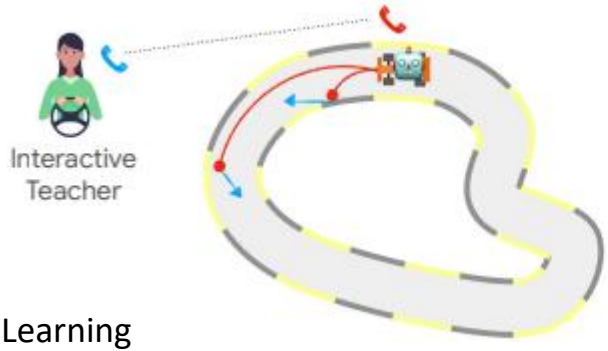




(C) On-Policy / Generalized KD (GKD)

- There is train–inference mismatch causes OOD failures when students self-generate.
- **Fix**: Student generates; Teacher provides feedback; optimize Reverse KL per token.
- Trains on on-policy samples to align training and deployment behavior.





Train / Test distribution = Student's distribution

Used for Gemma 2 post-training.

Source: CMU Lecture slides on Imitation Learning

(C) On-Policy / Generalized KD (GKD)

1. On-policy Data: Sample output sequences from the student

2. Feedback: Run inference using the teacher to get logits on student samples

3. Supervised Training: Minimize mismatch (e.g., KL-divergence) between student and teacher token-level logits.

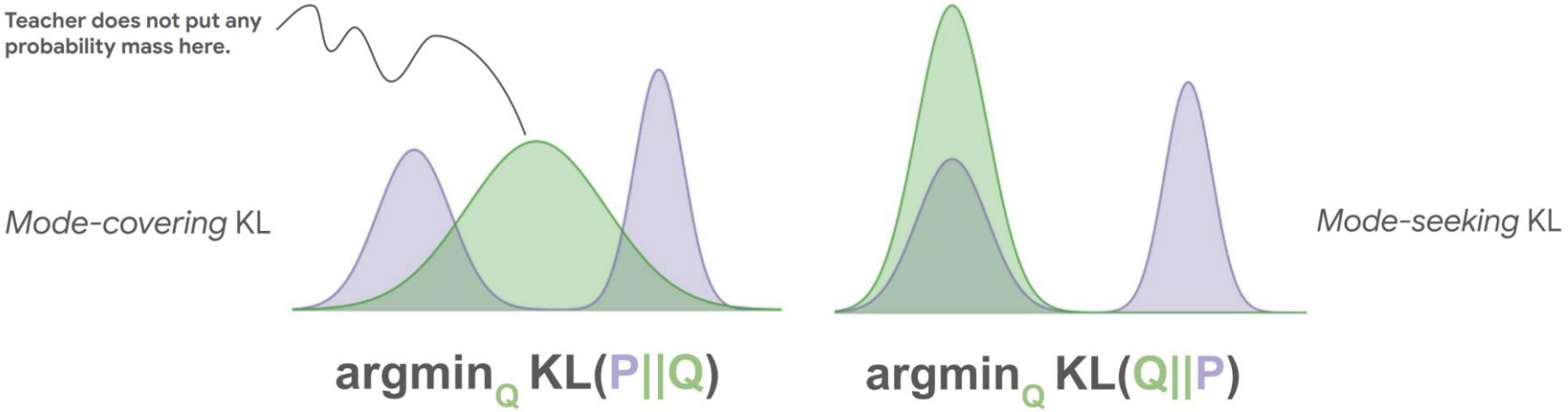
$$\min_\theta D_{KL}\big(\overbrace{p_\theta(x)}^{\textit{Student}} \,||\, \overbrace{q(x)}^{\text{Teacher}}\big)$$

$$= \min_\theta \mathbb{E}_{\underbrace{y \sim p_\theta(x)}_{\text{On-policy samples}}} \left[\sum_i \underbrace{D_{KL}\big(p_\theta(\cdot|x, y_{1:i-1})\big)||q(\cdot|x, y_{1:i-1})}_{\text{Per-Token Reverse KL}} \right]$$

GKD generalizes this to other f-divergences (JSD, Jeffreys)

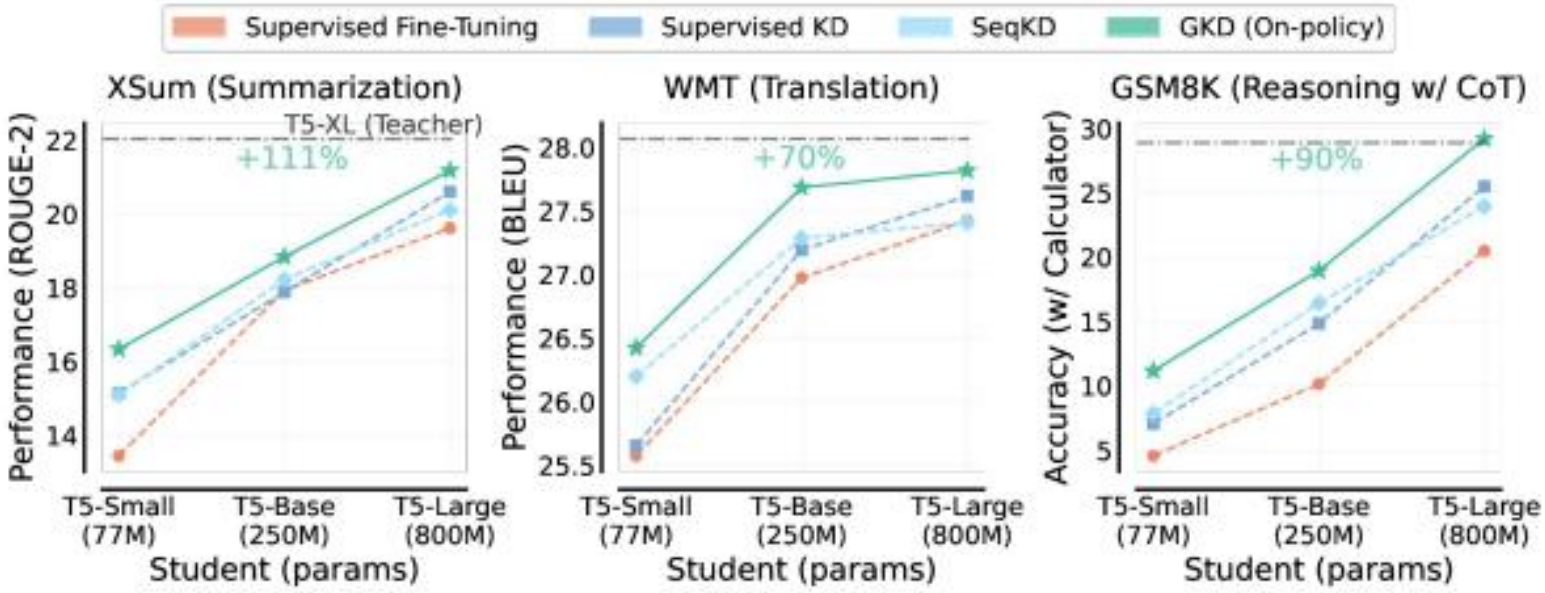
Mode-Covering vs. Mode-Seeking (Why Mix KLs)

- Forward KL ⇒ mode-covering; Reverse KL ⇒ mode-seeking.
- Small-capacity students often benefit from some mode-seeking to avoid bizarre tokens.
- Empirically, mixing (e.g., Jeffreys 0.5/0.5) worked best in experiments.



On-Policy Distillation

- On-Policy GKD outperforms other distillation approaches on T5 models



Agarwal et al. On-Policy Distillation of Language Models: Learning from Self-Generated Mistakes. ICLR 2024

On-Policy Distillation

- Qwen3-8B on par with Deepseek-R1.
- On-policy distillation is 10x more compute efficient than reinforcement learning

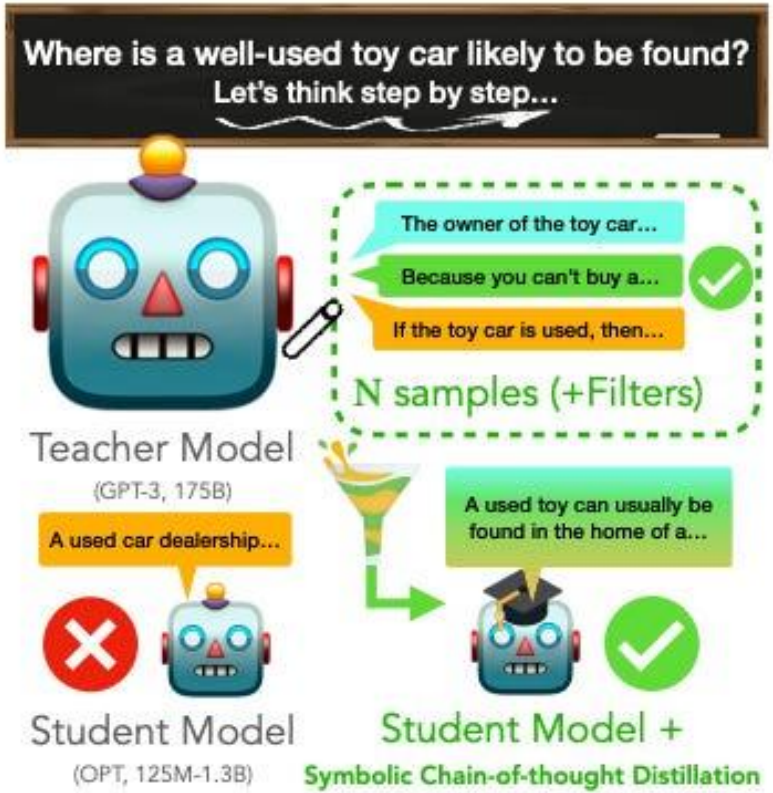
Table 21: Comparison of reinforcement learning and on-policy distillation on Qwen3-8B. Numbers in parentheses indicate pass@64 scores.

Method	AIME'24	AIME'25	MATH500	LiveCodeBench v5	MMLU -Redux	GPQA -Diamond	GPU Hours
Off-policy Distillation	55.0 (90.0)	42.8 (83.3)	92.4	42.0	86.4	55.6	-
+ Reinforcement Learning	67.6 (90.0)	55.5 (83.3)	94.8	52.9	86.9	61.3	17,920
+ On-policy Distillation	**74.4 (93.3)**	**65.5 (86.7)**	**97.0**	**60.3**	**88.3**	**63.3**	1,800

Agarwal et al. On-Policy Distillation of Language Models: Learning from Self-Generated Mistakes. ICLR 2024

Symbolic Chain of Thought Distillation

- small models can also "think" step-by-step" presented a method that enables smaller language models to learn step-by-step reasoning capabilities
- A smaller student model is trained on CoT samples from a much larger teacher model. (>50B parameters.)



Symbolic Chain-of-Thought Distillation: Small Models Can Also "Think" Step-by-Step

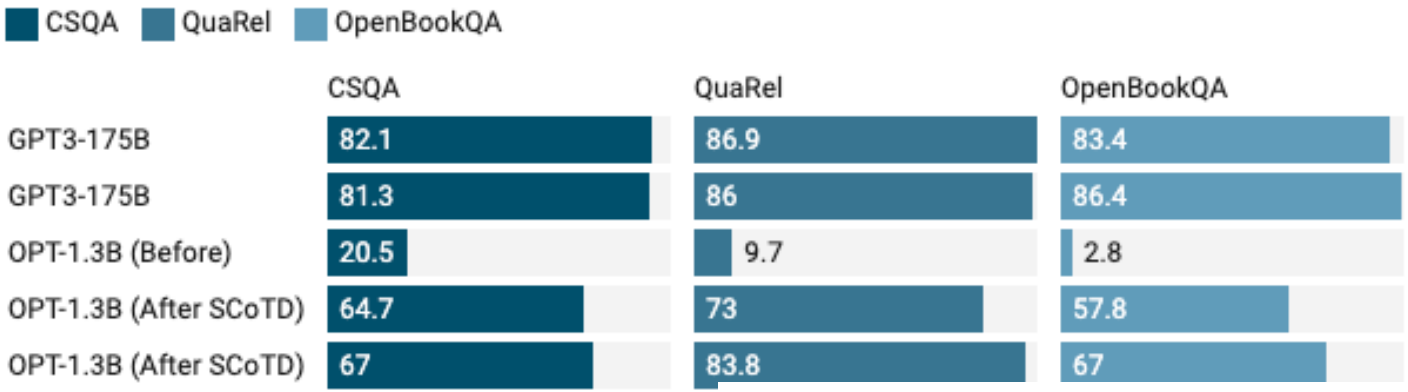
Liunian Harold Li[†♡], Jack Hessel[♣], Youngjae Yu[◇], Xiang Ren[◦], Kai-Wei Chang[†] & Yejin Choi[♣♡]

[†]University of California, Los Angeles, [♣]Allen Institute for Artificial Intelligence
[◦]University of Southern California, [◇] Yonsei University, [♡]University of Washington

Li et al. Symbolic Chain-of-Thought Distillation: Small Models Can Also "Think" Step-by-Step

Symbolic Chain of Thought Distillation

- Teacher Model: Large language model (e.g., GPT-3 175B)
- Student Model: Smaller model (e.g., OPT 125M-1.3B)

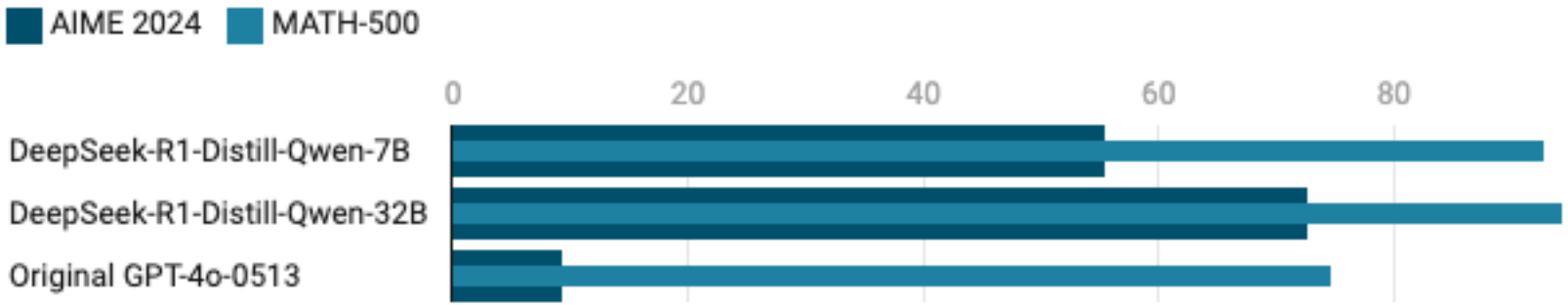


Data Amount	Performance Impact
Few-Shot	60-70% accuracy
Full Supervision	70-80% accuracy

RL-enhanced distillation, DeepSeek

The teacher model provides not just output probabilities but also rewards that help shape the student's behavior

- Direct RL distillation through DeepSeek-R1-Zero, achieving 71.0% on AIME 2024 without supervised fine-tuning
- Hybrid approach with DeepSeek-R1, combining cold-start data with iterative RL fine-tuning, reaching 79.8% on AIME 2024



https://labs.adaline.ai/p/llm-distillation-explained

A Scaling Law for Distilled Students

This paper extends that idea to distillation:

$$\underbrace{L(N,D)}_{\text{Model Cross-Entropy}} = \underbrace{E}_{\text{Irreducible Error}} + \underbrace{\left(\frac{A}{N^{\alpha}}+\frac{B}{D^{\beta}}\right)^{\gamma}}_{\text{Model ability to mimic data}}, \quad (1)$$

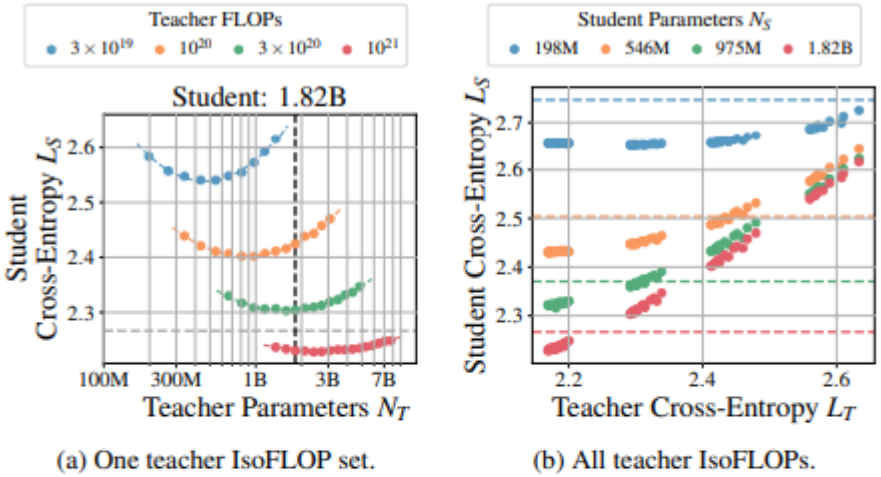
- Student loss L_S is a function of
  - student parameters N_S,
  - distillation tokens D_S,
  - teacher cross-entropy L_T.

we propose that student cross-entropy follows a broken power law in L_T and a power law in N_S and D_S:

$$\underbrace{L_S(N_S,D_S,L_T)}_{\text{Student cross-entropy}} = \underbrace{L_T}_{\text{Teacher cross-entropy}} + \underbrace{\frac{1}{L_T^{c_0}}\left(1+\left(\frac{L_T}{\widetilde{L}_S d_1}\right)^{1/f_1}\right)^{-c_1 f_1}\left(\frac{A}{N_S^{\alpha'}}+\frac{B}{D_S^{\beta'}}\right)^{\gamma'}}_{\text{Student ability to mimic teacher}} \quad (8)$$

- Teacher size and training tokens matter **only through** the resulting teacher loss L_T.
- Student loss follows a power law in N_S and D_S ,and a **broken power law** in L_T

Key Empirical Findings



(a) One teacher IsoFLOP set. (b) All teacher IsoFLOPs.

Figure 3. **IsoFLOP Teacher/Fixed M Students. (a)** One of four students with a token-to-parameter ratio $M_S = D_S/N_S \approx 20$ is distilled from teachers with four IsoFLOP profiles defined by compute budgets $C_T \in \{3 \times 10^{19}, 10^{20}, 3 \times 10^{20}, 10^{21}\}$ FLOPs. For all four student sizes $N_S \in \{546M, 975M, 1.82B, 7.75B\}$, see Appendix E.4, Figure 38b. **(b)** All profiles are plotted against teacher cross-entropy L_T. Horizontal (vertical) dashed lines show student supervised cross-entropy $\widetilde{L}_S$ (student size N_S).

Teacher Cross-entropy Alone Predicts Student Performance

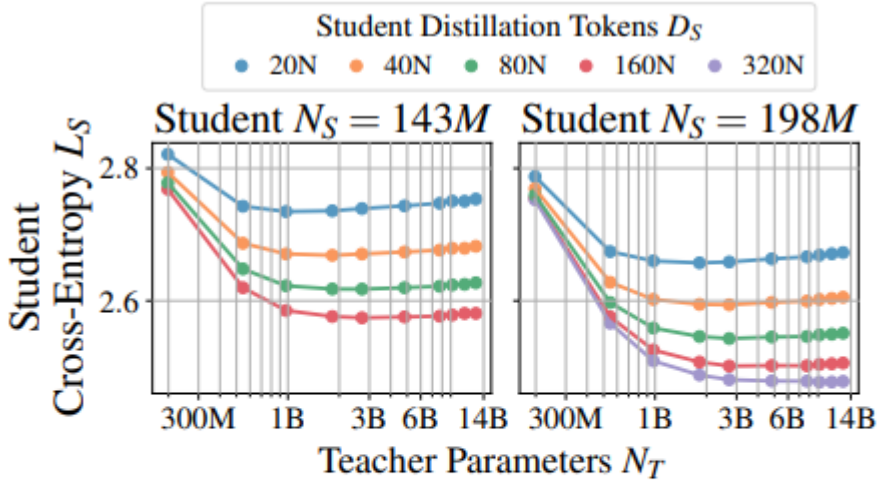


Figure 4. **Fixed M Teacher/Fixed M Student.** Students of two sizes trained with different token-to-parameter ratios $M_S = D_S/N_S \in \{20, 40, 80, 160, 320\}$ are distilled from teachers of various sizes with a token-to-parameter ratio $M_T = D_T/N_T \approx 20$. The capacity gap is visible: student cross-entropy decreases to an optimum and then increases with increasing teacher size N_T.

Compute-Optimal Distillation vs. Supervised Training

- The authors compare four compute scenarios
  - best case (teacher fully amortized),
  - teacher inference only,
  - teacher pretraining only,
  - teacher pretraining plus inference.
- Given enough compute, supervised training always matches the best possible distillation for a fixed student size.
- Distillation is more compute-efficient than supervised training only when
  - ➢ student compute stays below a size-dependent threshold, and
  - ➢ a suitable teacher already exists or will be reused for many students.

Why Distillation Scaling Laws Are Important

- Provides the first large-scale, controlled distillation scaling law for language models.
- Clarifies the capacity gap phenomenon and when stronger teachers stop helping students.
- Gives compute-optimal recipes for
  - choosing teacher strength,
  - allocating tokens between T and S
  - deciding whether to distill or train

Table 3. Optimal compute allocation trends.

Student size	Compute (FLOPs)	Allocation
Small ($\lesssim 3B$)	Small ($\lesssim 10^{21}$)	Mostly teacher pretraining.
Small ($\lesssim 3B$)	Large ($\gtrsim 10^{25}$)	Evenly divided between student training and teacher inference, much less on teacher pretraining.
Large ($\gtrsim 10B$)	Small ($\lesssim 10^{21}$)	Mostly standard student training.
Large ($\gtrsim 10B$)	Large ($\gtrsim 10^{25}$)	Equally divided between student training, teacher inference, and teacher pretraining.

Why Distillation Scaling Laws Are Important

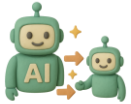
- Practical takeaway:
- Distillation is best when you
  - have or plan to reuse a strong teacher, and
  - want many capable small models under limited compute.
- For single models at very large compute, direct supervised training is simpler and optimal.

When Distillation Beats Supervised Training

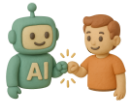
- Distillation이 supervised pretraining보다 좋은 경우
  - teacher가 이미 존재할 때
  - student에 사용할 compute나 tokens가 '너무 많지 않을 때'
  - student가 teacher보다 훨씬 작아서 capacity gap의 왼쪽 영역에 있을 때
- Distillation이 손해가 되는 경우
  - teacher를 새로 학습해야 하고, student는 1명뿐일 때
  - student compute가 충분히 클 때
  - teacher가 너무 강하거나 너무 약할 때

1. An *infinitely capable student* should be able to model *any* teacher: $\lim_{N_S, D_S \to \infty} L_S(N_S, D_S, L_T) \to L_T$.
2. A *random* teacher produces *random* students *independent* of how capable those students are: $\lim_{L_T \to \infty} L_S(N_S, D_S, L_T) \to L_T$.
3. There is a *capacity gap*: making a teacher too capable eventually reduces the student performance.

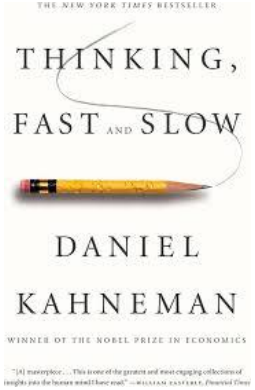
Human-like Communication, Computational Social Science



System 2 reasoning **distillation** for **small LLMs**



Human-in-the-loop, Agents collaborating with human



Dialogue CoT Distillation **EMNLP 2023**

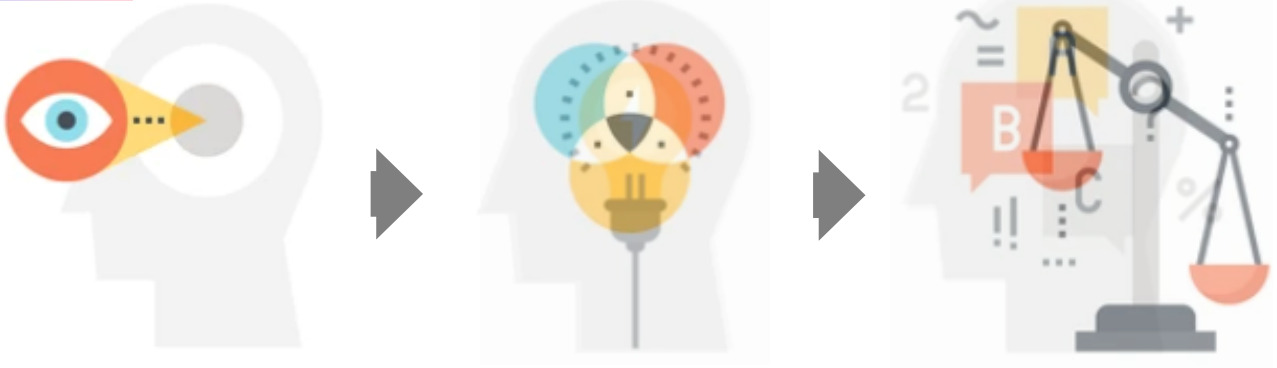
Symbolic CoT Distillation **ACL 2023**

NormLens : GroundedNorms **EMNLP 2023**

Automatic Chart Generation **NAACL 2025**

Commonsense-aware Navigation **ICRA 2025**

Disambiguating Robot Commands **ICRA 2024**



Perception Intuition Reasoning

[Ongoing] Transfer Knowledge from VLM/VLA to sVLA

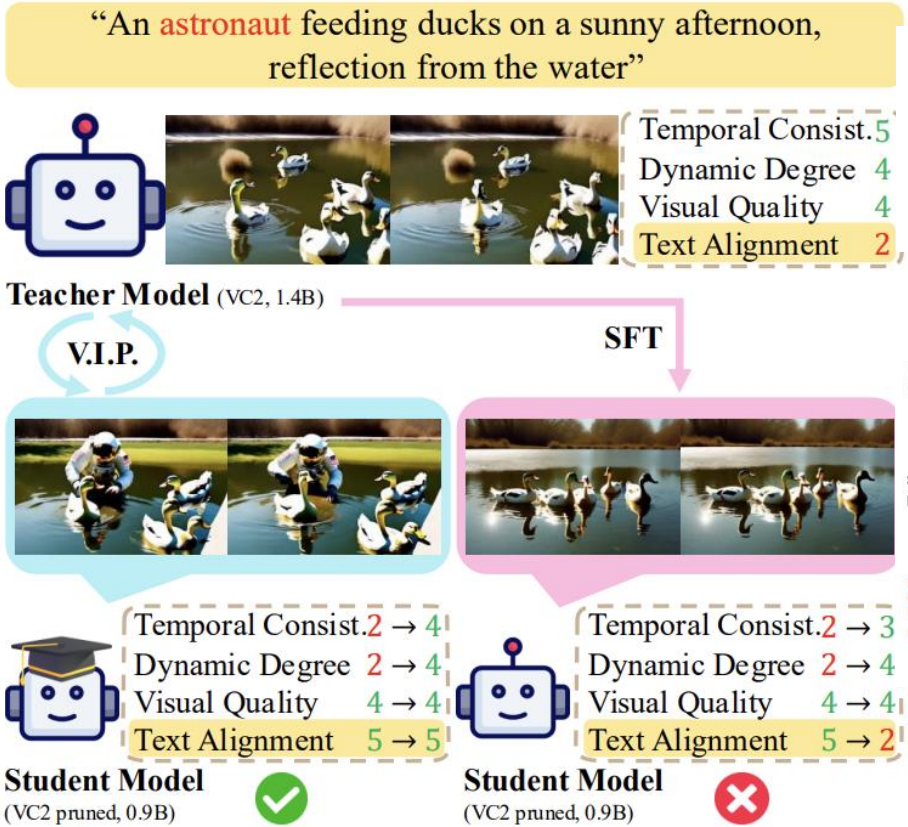
Vision Language Action model distillation / Lightweighting

- Evolutionary distillation on simulated environment

Dialogue CoT Distillation **EMNLP 2023**	Symbolic CoT Distillation **ACL 2023**
Commonsense-aware Navigation **ICRA 2025**	Disambiguating Robot Commands **ICRA 2024**

VIP : Video Gen Distillation
ICCV 2025

Distillation for Multimodal Gen AI



V.I.P. : Iterative Online Preference Distillation for Efficient Video Diffusion Models

Jisoo Kim Wooseok Seo Junwan Kim Seungho Park Sooyeon Park Youngjae Yu
Yonsei University
{jisoo6687, justin_seo, jw1510, gomi0904, dreamyou070, yjy}@yonsei.ac.kr

Conclusion : Distillation, Pros and Cons

Computational Efficiency

- Up to 90% parameter reduction while preserving core reasoning
- Large reductions in storage, memory footprint, and deployment-time energy usage

Performance Improvements

- Smoothed KD reduces hallucinations
- Stronger generalization across diverse tasks and domains

Conclusion : Distillation, Pros and Cons

Practical Applications

- Enables real-time inference on mobile and edge devices
- Reduces infrastructure needs for broad accessibility
- Cost-effective scaling for production deployment

Limitations & Challenges

- Hard reasoning tasks still show a performance gap
- Requires expertise in tuning (temperature, loss balancing)
- Optimal settings vary significantly by task

Conclusion : Key Takeaways & What's Next

- Distillation is the fastest path to small, fast, affordable LLMs with solid quality.
- Start with synthetic KD; add on-policy GKD if robustness is required.
- Distillation Scaling Law, Optimal Teacher & Student strategy
- Inference acceleration (e.g., speculative decoding) and newer KD variants.
- Multimodal Distillation, VLA Distillation are next promising challenge

Reference

- The Magic of LLM Distillation - Rishabh Agarwal, Google DeepMind
- Scaling Laws for Autoregressive Generative Modeling https://arxiv.org/pdf/2010.14701.pdf
- Scaling Laws for Neural Language Models https://arxiv.org/pdf/2001.08361.pdf
- Neural Scaling Laws and GPT-3, https://www.youtube.com/watch?v=QMqPAM_knrE
- Beyond neural scaling laws- data pruning https://arxiv.org/abs/2206.14486

School of Computer Science & Engineering

Thanks for listening!!

youngjaeyu@snu.ac.kr